

6_Rscript_spatialData_analysis

V. R. KnightSchrijver

September 3, 2026

1 Rscript for the Kuppe2022 spatial dataset

1.1 Libraries and Data

```
[1]: library(anndata)
      library(EBImage)
      library(matrixStats)
      library(conover.test)
```

```
[2]: setwd("../")
      getwd()
```

"/home/vincent/Documents/Projects/Endothelins/Manuscript2025/Analysis'

```
[3]: # functions etc
      func.dir=~"/Documents/Rlib/functions"
      source("~"/Documents/Rlib/functions/load_functions.r")
      # additional functions
      source("scripts/spatialDataProcessing.r")
```

Loading required package: RColorBrewer

Loading required package: wordcloud

The following functions contained errors and were not loaded properly, please check the code:

cell_types.r colour_scale_map.r

```
[4]: datasets = list.files("data/datasets", full=T)
      datasets
```

```
1. 'data/datasets/control_P1_10X001.h5ad' 2. 'data/datasets/control_P17_ACH002.h5ad'
3. 'data/datasets/control_P7_ACH003.h5ad' 4. 'data/datasets/control_P8_ACH004.h5ad'
5. 'data/datasets/GT_IZ_P9_ACH0012.h5ad' 6. 'data/datasets/IZ_P10_ACH0014.h5ad'
7. 'data/datasets/IZ_P15_ACH0015.h5ad' 8. 'data/datasets/IZ_P16_ACH0016.h5ad'
9. 'data/datasets/IZ_P3_10X0017.h5ad' 10. 'data/datasets/RZ_P3_10X0020.h5ad'
```

11. 'data/datasets/RZ_P6_ACH0021.h5ad' 12. 'data/datasets/RZ_P6_ACH0022.h5ad'
 13. 'data/datasets/RZ_P9_ACH0023.h5ad'

```
[5]: # HARDCODE sample IDs here?
      sample_id_list <- c( # these were manually identified in the h5ad
        ↪objects
          "AKK006_II_healthycontrol_V19T05022_rep2", # "datasets/
        ↪control_P1_10X001.h5ad"
          "Visium_1_CK279", # "data/spatialdatasets/control_P17_ACH002.
        ↪h5ad"
          "Visium_3_CK281", # "data/spatialdatasets/control_P7_ACH003.h5ad"
          "Visium_4_CK282", # "data/spatialdatasets/control_P8_ACH004.h5ad"

          "Visium_9_CK287", # "data/spatialdatasets/GT_IZ_P9_ACH0012.h5ad"
          "Visium_7_CK285", # "data/spatialdatasets/IZ_P10_ACH0014.h5ad"
          "Visium_20_CK298", # "data/spatialdatasets/IZ_P15_ACH0015.h5ad"
          "Visium_15_CK293", # "data/spatialdatasets/IZ_P16_ACH0016.h5ad"
          "AKK002_Infarct_MI_acute_V19T05025_rep2", # "datasets/
        ↪IZ_P3_10X0017.h5ad"

          "AKK002_Normal_MI_acute_V19T05026_rep1", # 'data/spatial/datasets/
        ↪RZ_P3_10X0020.h5ad'
          "Visium_2_CK280", # 'data/spatial/datasets/RZ_P6_ACH0021.h5ad'
          "Visium_17_CK295", # 'data/spatial/datasets/RZ_P6_ACH0022.h5ad'
          "Visium_6_CK284" # 'data/spatial/datasets/RZ_P9_ACH0023.h5ad'
        )
```

1.2 Plotting the genes of interest

Again, as these functions were part of an ongoing shiny app code base, I just pull up the functions that I was working on to access the pre-processed data, and plot the values over the images.

```
[6]: # Where all the functions are (also a bunch of redundant or "in-progress"
      ↪functions)..
      source("scripts/plot_spatialGene.r")
```

```
[7]: gc()
```

		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2 × 6 of type dbl	Ncells	2178246	116.4	4284382	228.9	2960177	158.1
	Vcells	406501974	3101.4	614138087	4685.6	425138367	3243.6

```
[8]: #####inputs from ui.r:
      input = list(
        "variable.list" = c("fine_grain", "week"),
        "genes_input" = c("MYH7"),
        "genes_LR" = c("RSP01"),
        "genes_selected" = c("LGR6"),
```

```

"gene.red" = "MYH7",
"gene.blue" = "MYH11",
"gene.green" = "MYH6",
"annotations2keep" = NULL,
"rescale" = 1,
"light_or_dark" = 2,
"plot_size" = 0.5,
"dot_or_not" = 2,
"direction" = "send",
"genes_selected" = list(),
"display_spatial" = TRUE,
"opacity" = 1,
"min.col" = 0.1,
"mask_overlay" = T,
"spatial_channels" = c("red", "green", "blue"),
"opacity_count_dependent" = T,
"red_on" = 1,
"green_on" = 1,
"blue_on" = 1
)

```

1.2.1 EDNRA + ECE1 + EDNRB

```

[10]: # RGB color plots for some select samples ("representative?").
      gene.r = "EDNRA"
      gene.g = "ECE1"
      gene.b = "EDNRB"
      options(repr.plot.width = 20)
      options(repr.plot.height = 20)

      for(i in c(1,2,3,4,5,6,7,9,10,11,12,13)){
        samples2plot = sample_id_list[i]
        pdf(paste("figures/panels/results/Kuppe2022/EDNRA_ECE1_EDNRB_RGB_", gsub("^.",
↪ *datasets/|.h5ad", "", datasets[i]), sample_id_list[i], ".pdf", sep=""),
↪ height=20, width=20)

        plotVisium2(
          rgb.samples_list = VisiumData2(gene.r, gene.g, gene.
↪ b, sample_id_list = samples2plot),
          light_or_dark = 2,
          plot_size = 1,
          mask_overlay = input$mask_overlay,
          min.col = input$min.col,
          red_on = sum("red" %in% input$spatial_channels),
          green_on = sum("green" %in% input$spatial_channels),
          blue_on = sum("blue" %in% input$spatial_channels),

```

```

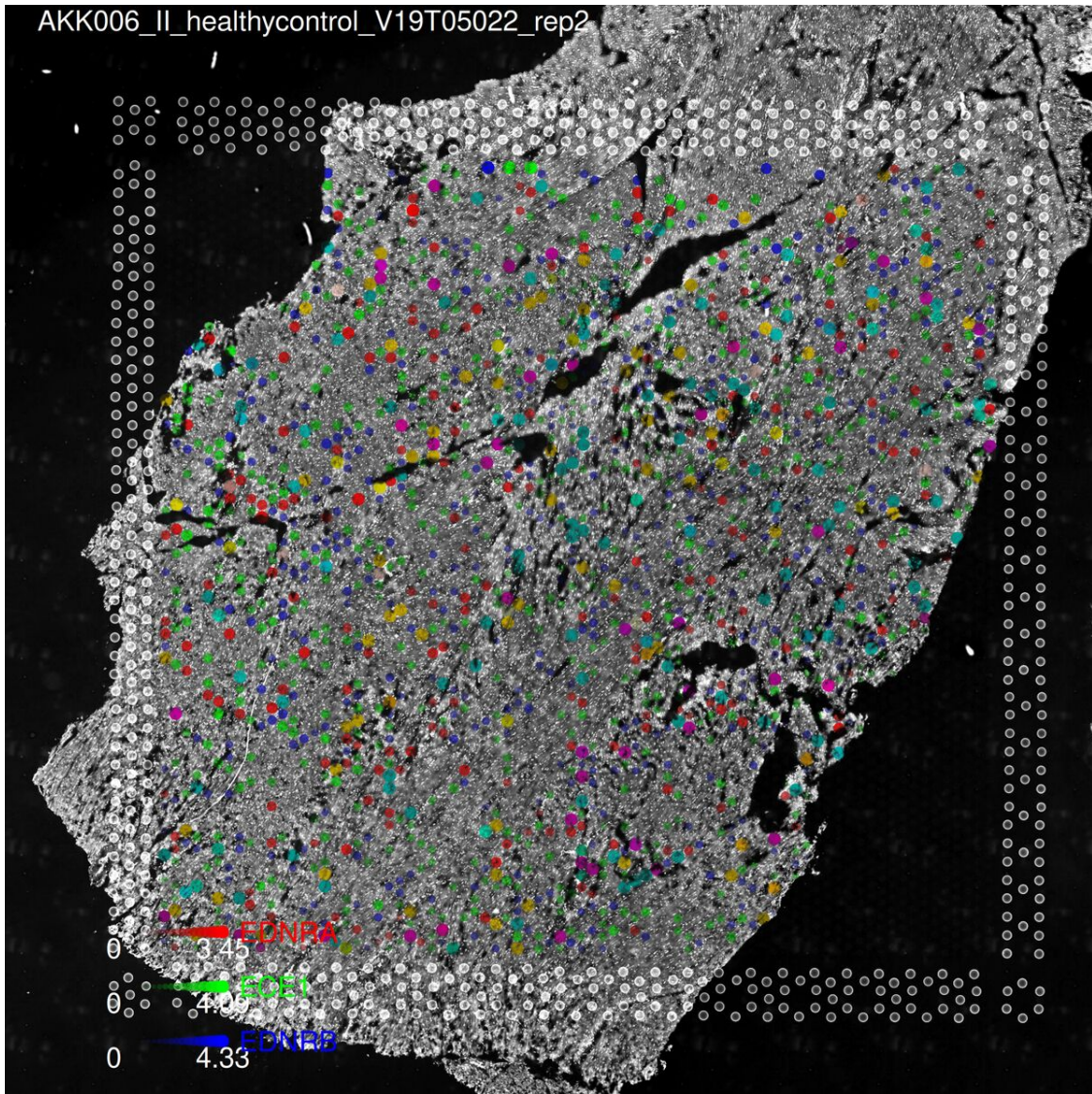
        opacity_count_dependent = □
↪sum(input$opacity_count_dependent),
        opacity = input$opacity,
        dot_or_not = 2,
        sample_id_list = samples2plot
    )

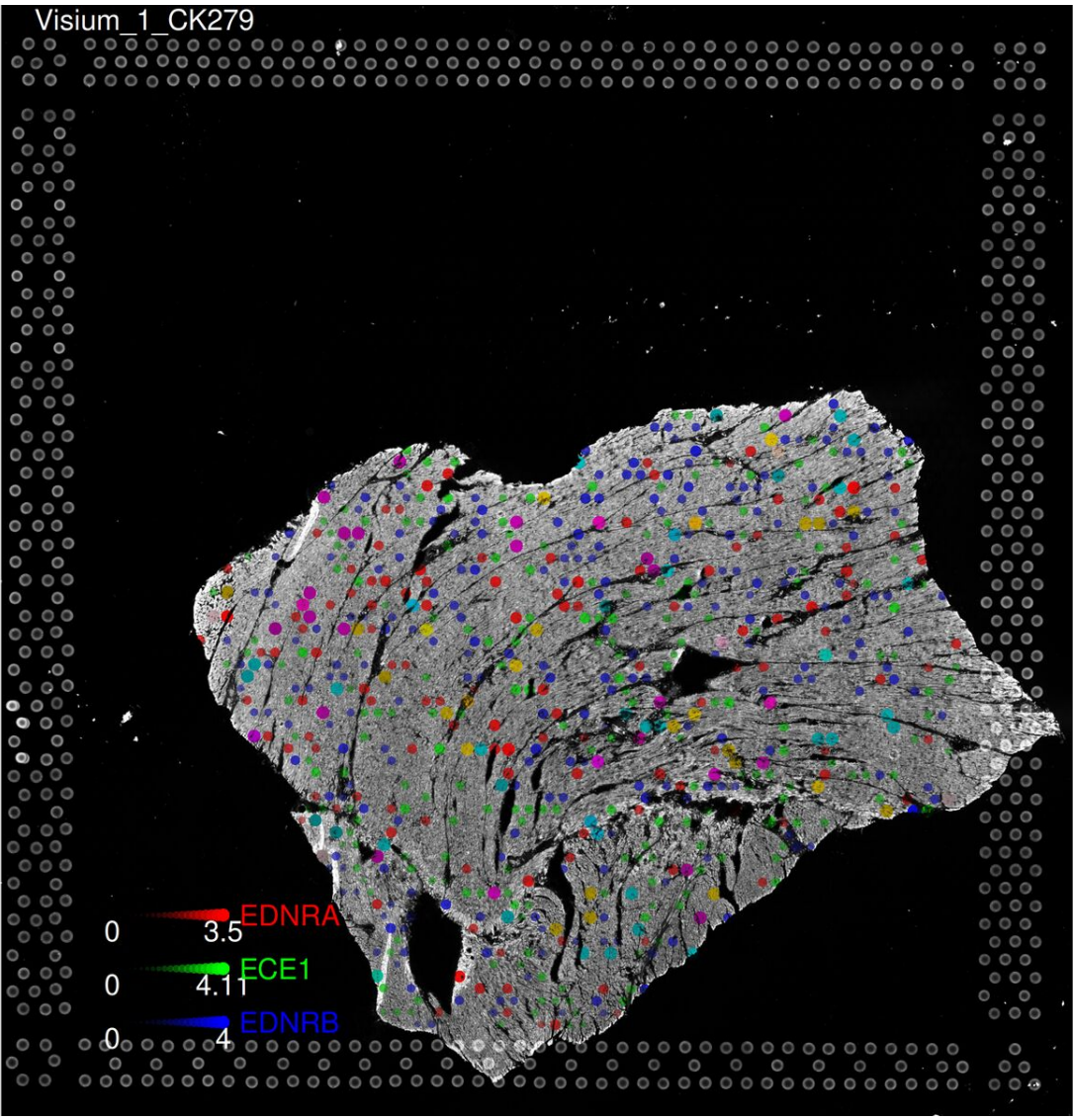
    dev.off()

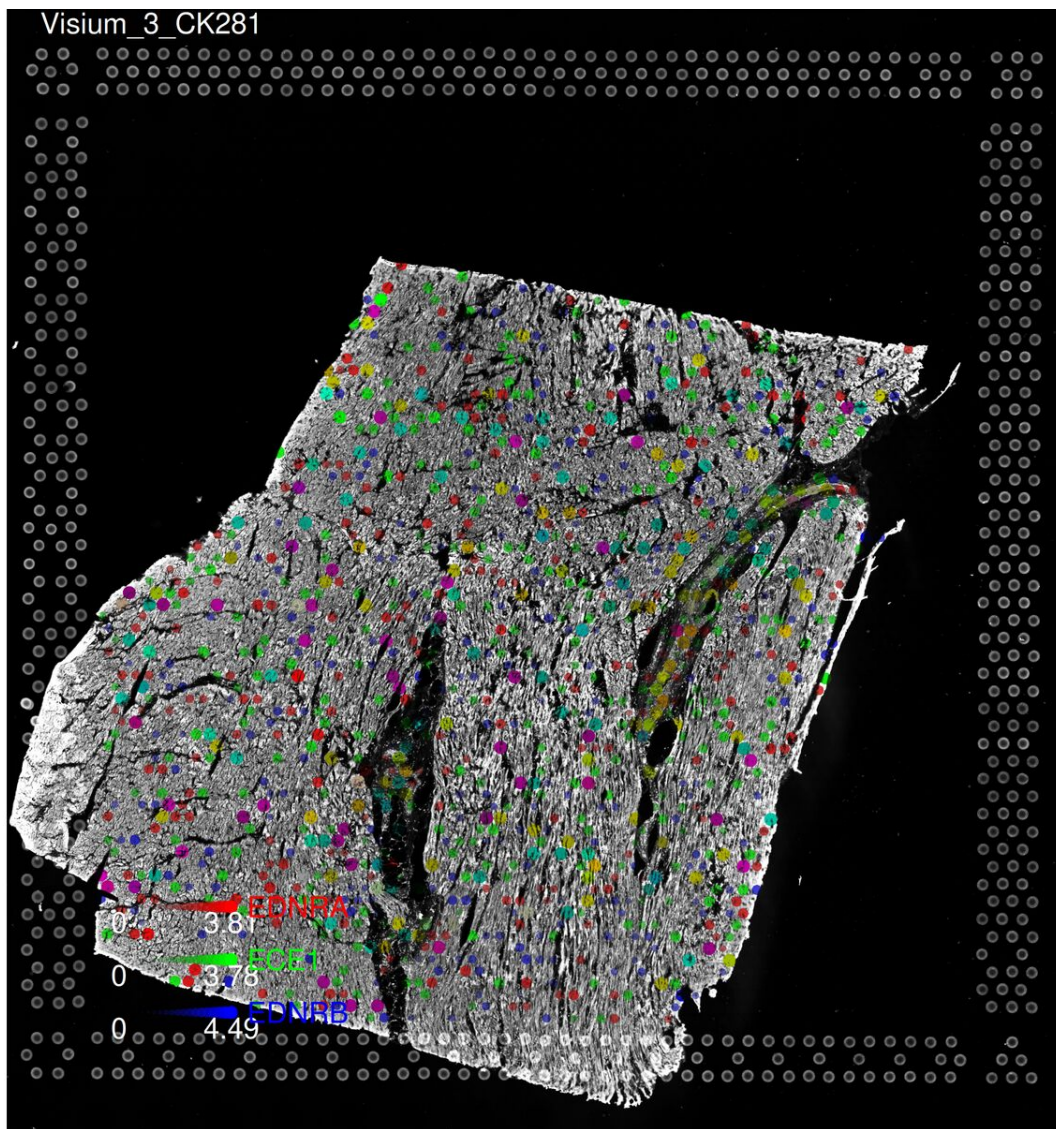
    plotVisium2(
        rgb.samples_list = VisiumData2(gene.r, gene.g, gene.
↪b, sample_id_list = samples2plot),
        light_or_dark = 2,
        plot_size = 1,
        mask_overlay = input$mask_overlay,
        min.col = input$min.col,
        red_on = sum("red" %in% input$spatial_channels),
        green_on = sum("green" %in% input$spatial_channels),
        blue_on = sum("blue" %in% input$spatial_channels),
        opacity_count_dependent = □
↪sum(input$opacity_count_dependent),
        opacity = input$opacity,
        dot_or_not = 2,
        sample_id_list = samples2plot
    )

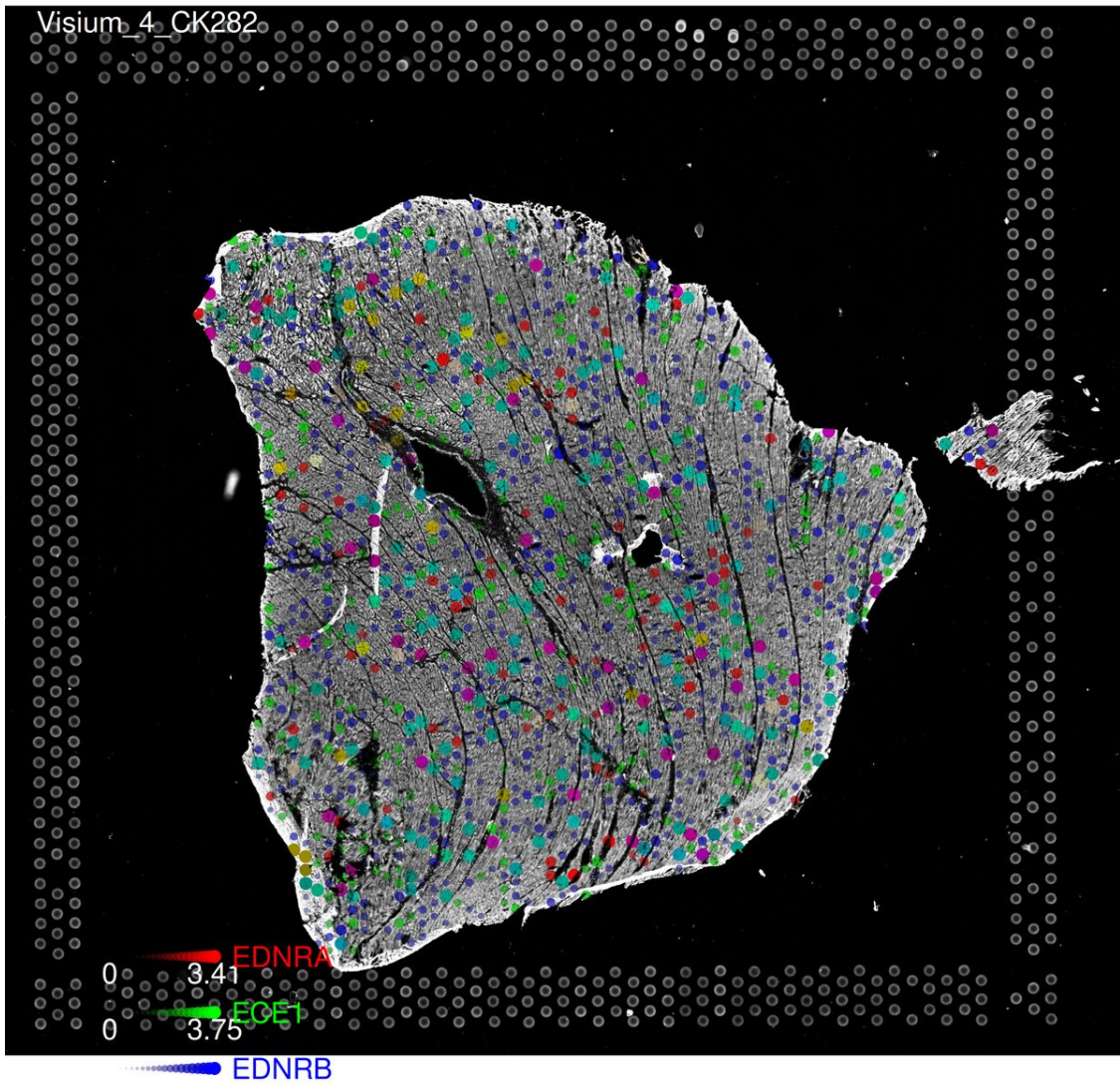
}

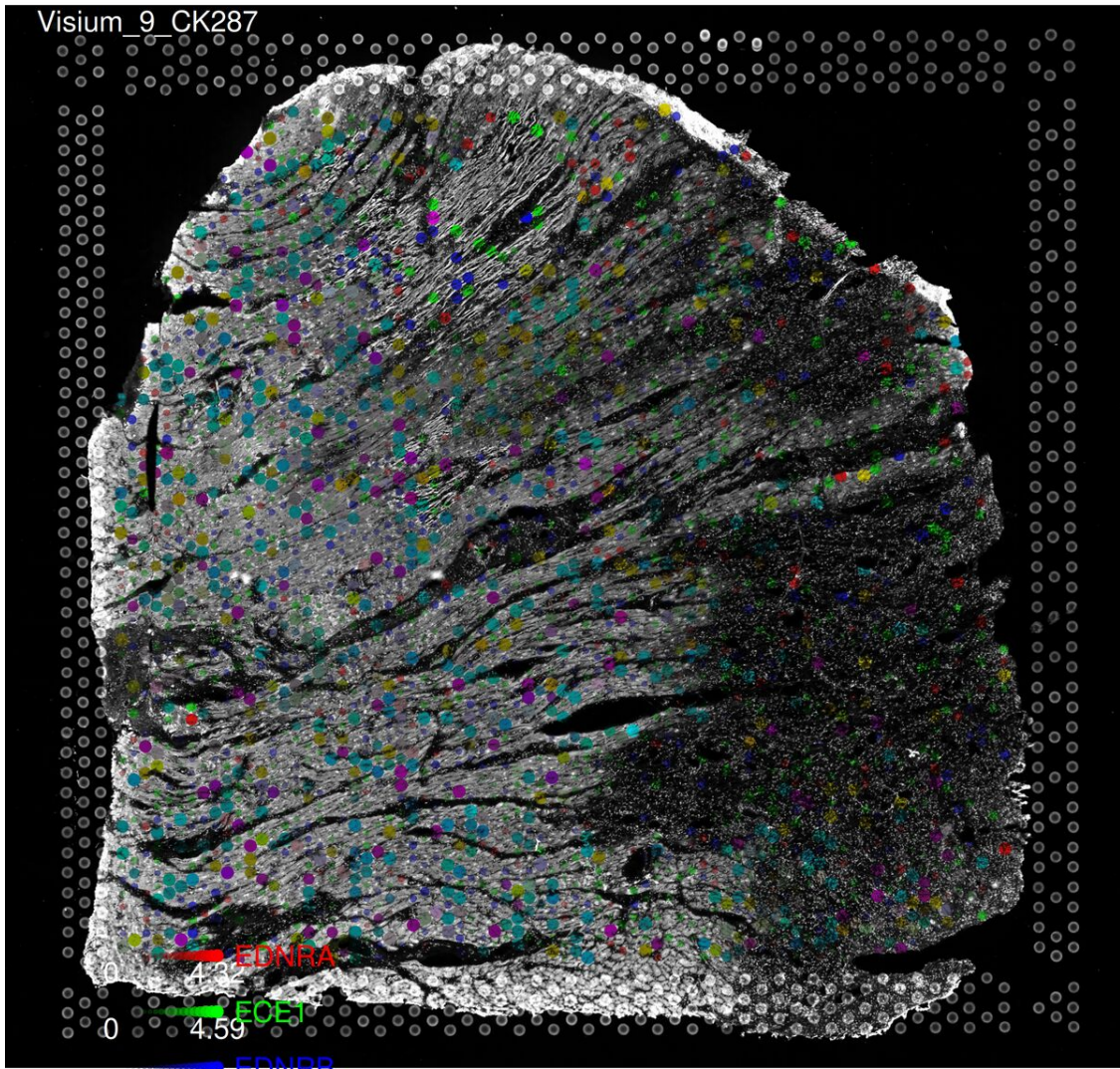
```

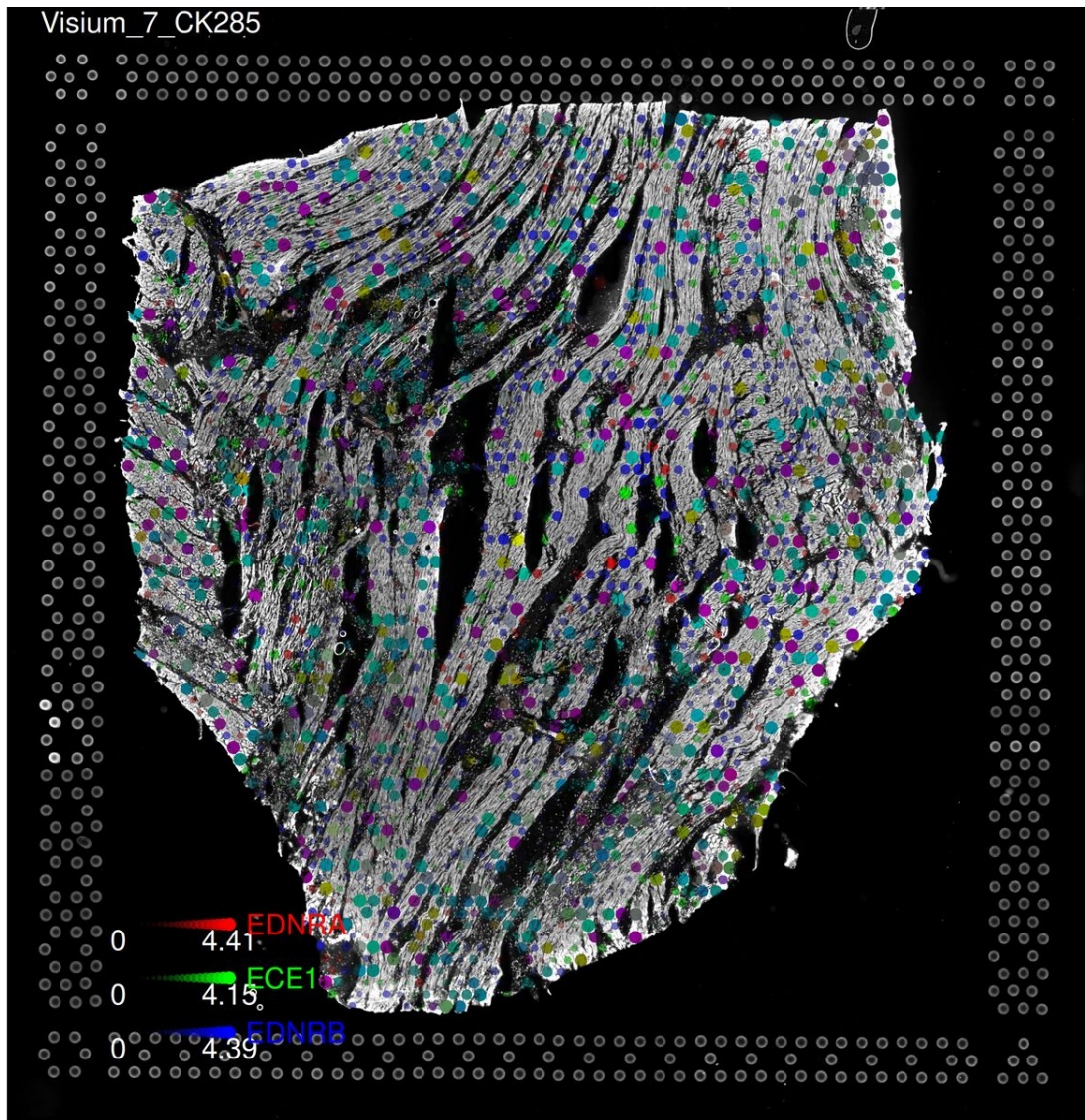



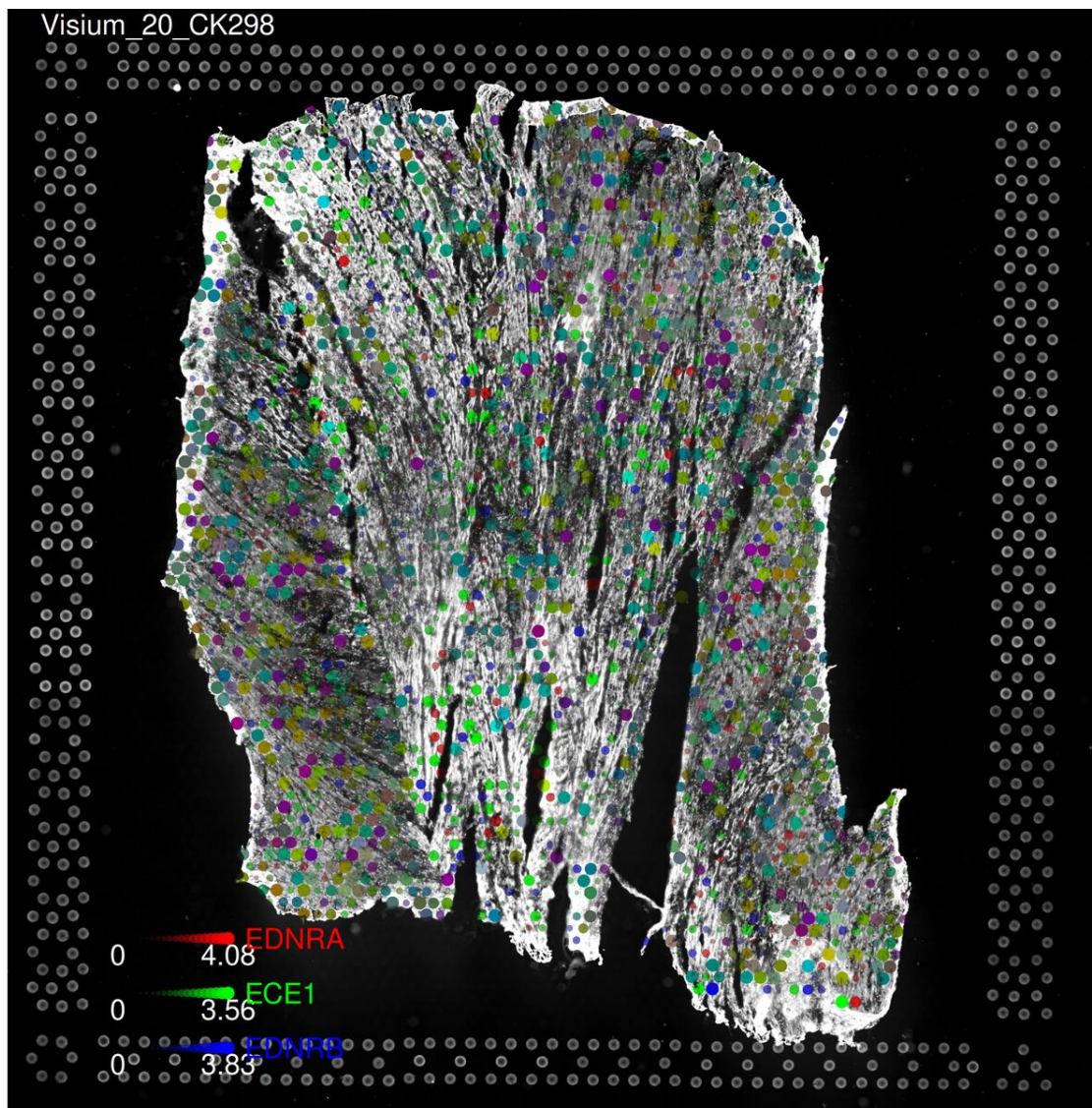




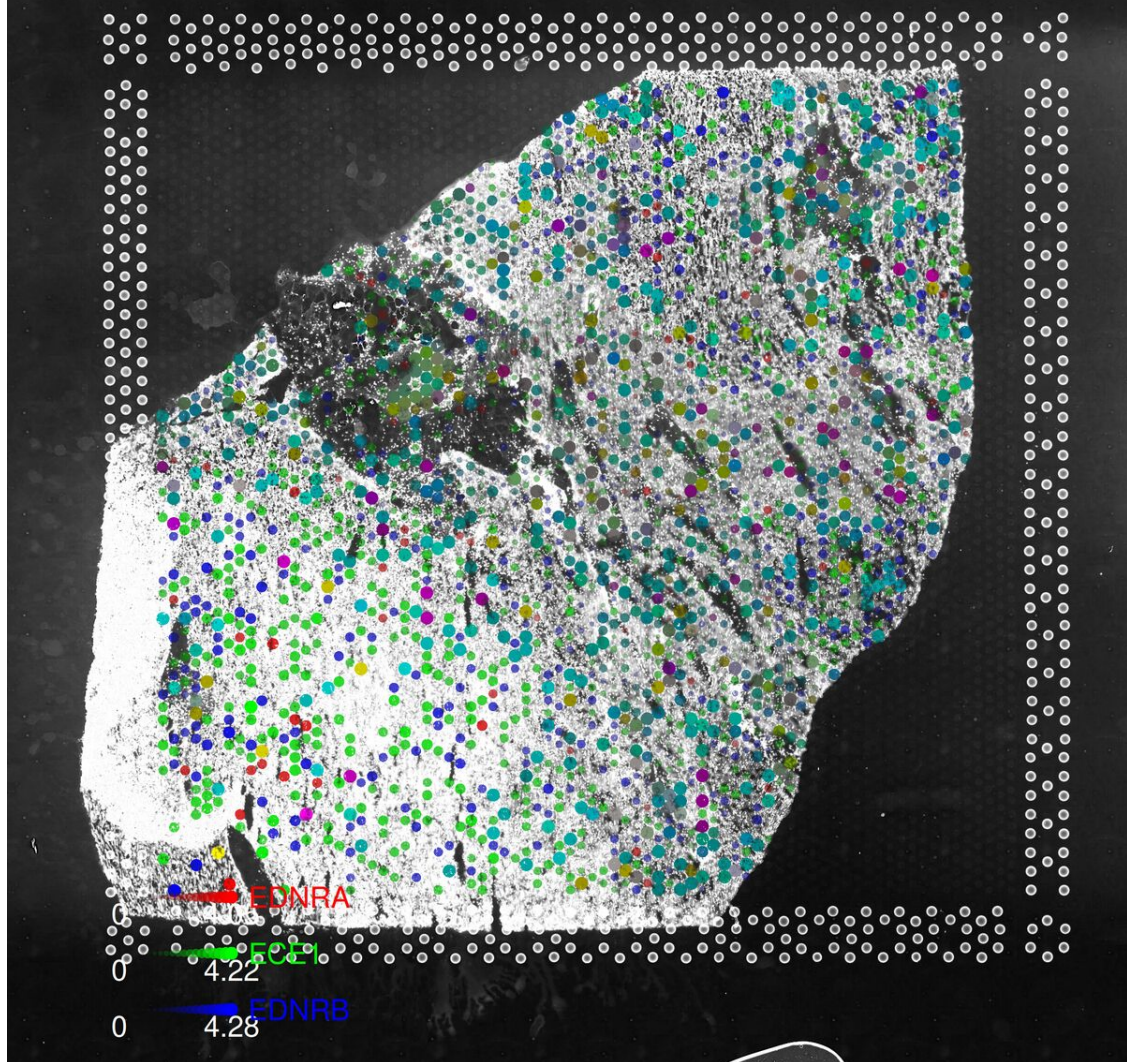




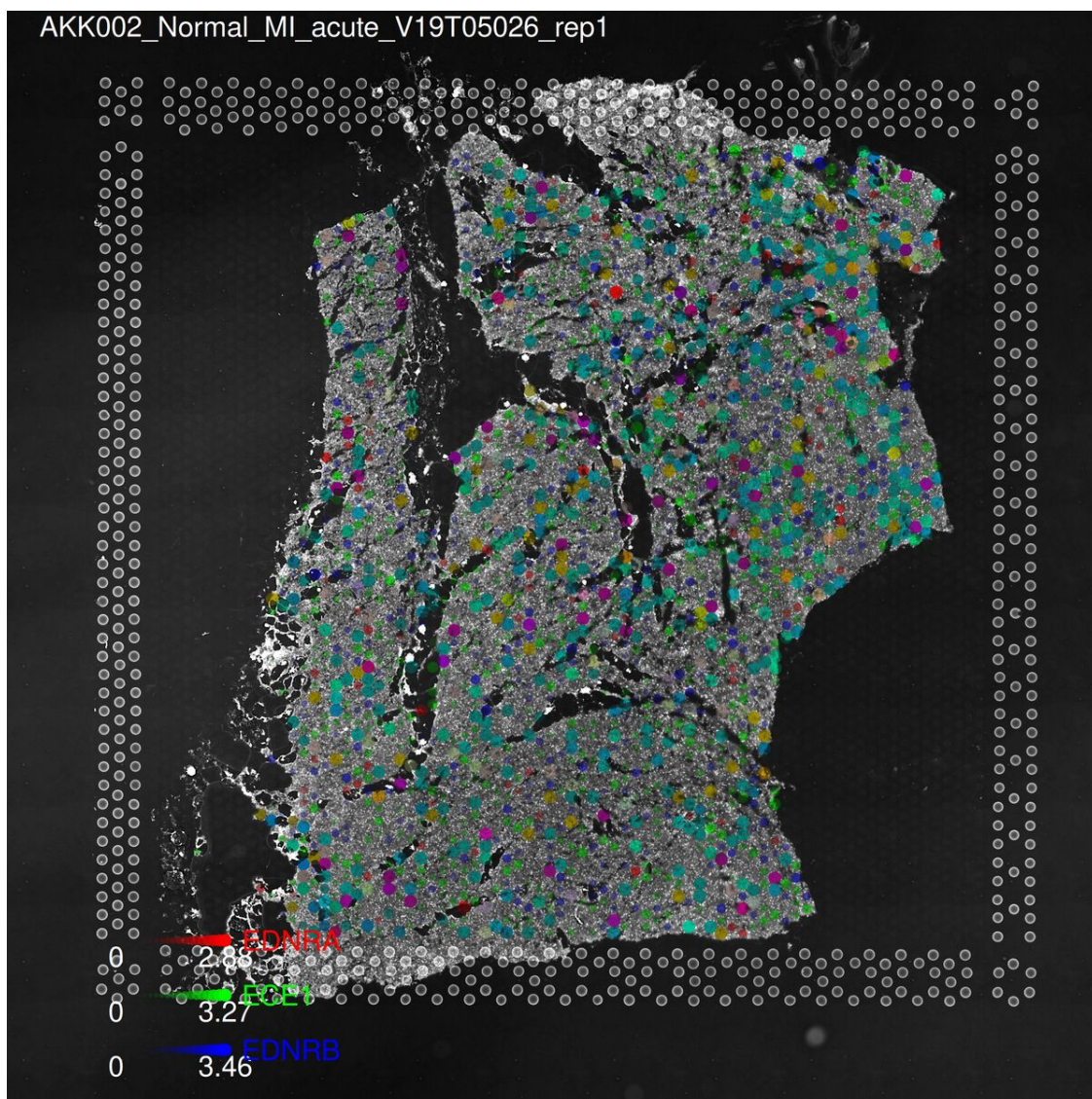


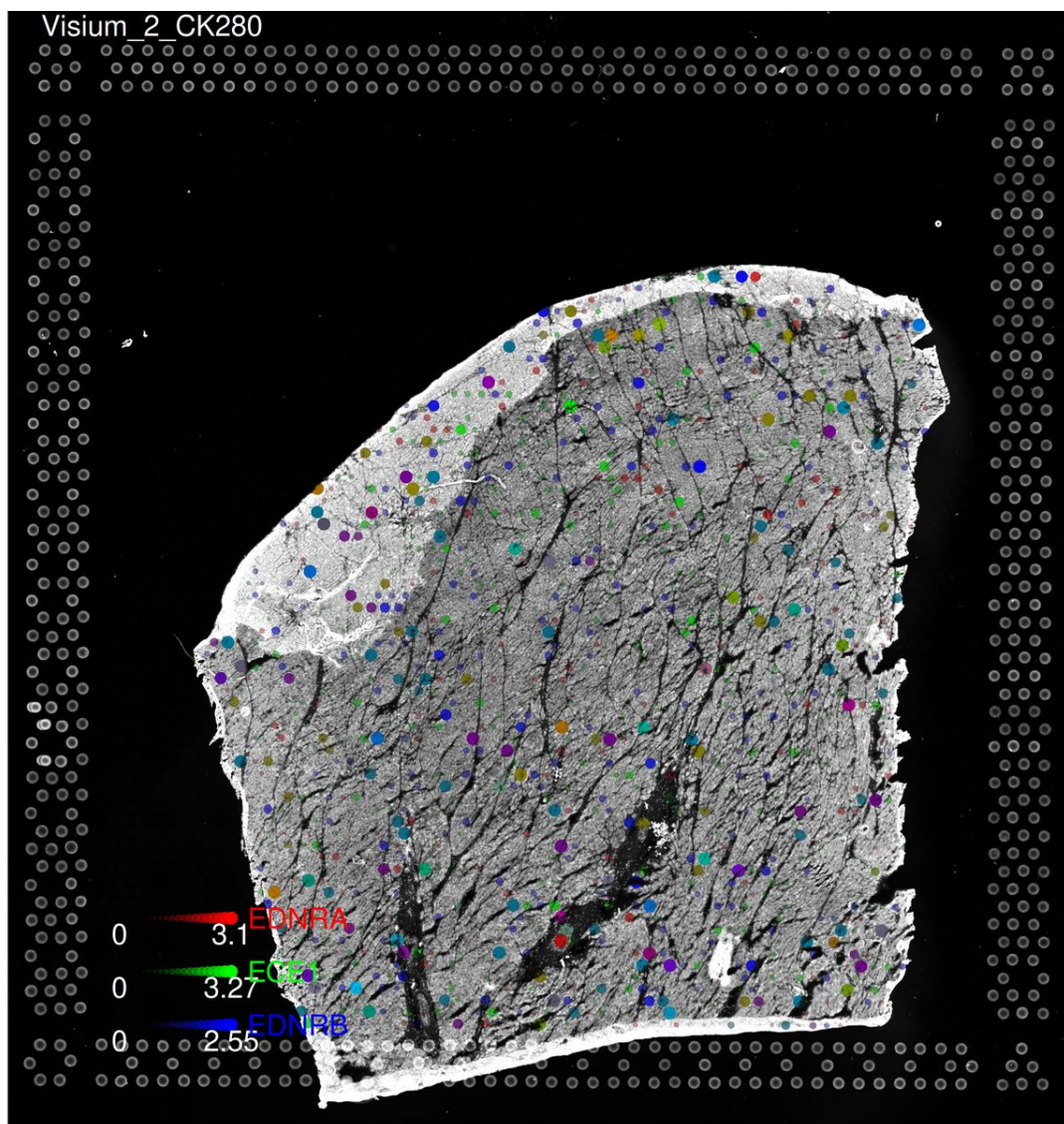


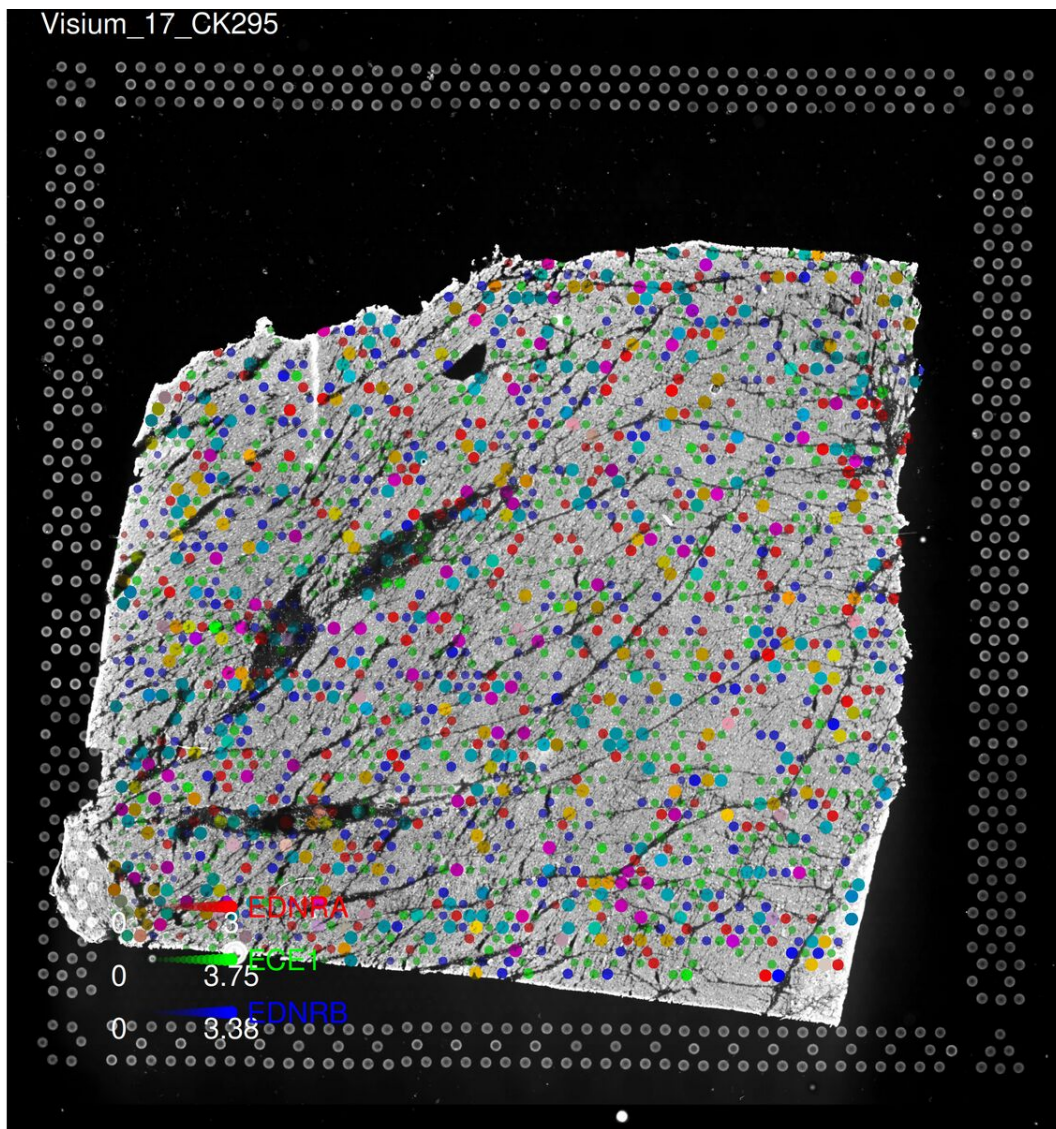
AKK002_Infarct_MI_acute_V19T05025_rep2

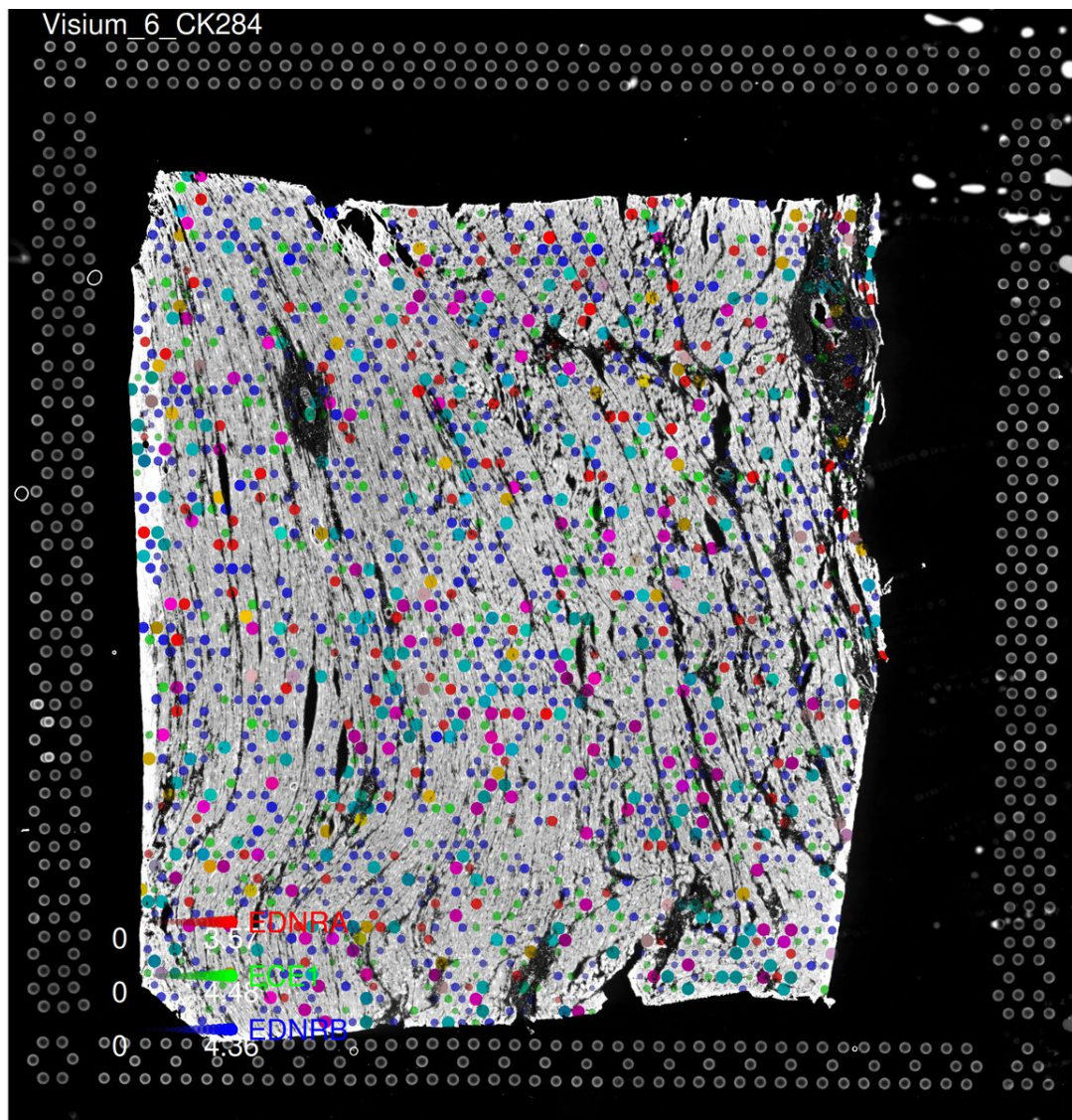


AKK002_Normal_MI_acute_V19T05026_rep1









1.2.2 ECE1 + TNNT2 + MYH11

```
[11]: gene.r = "ECE1"
      gene.g = "TNNT2"
      gene.b = "MYH11"
      options(repr.plot.width = 20)
      options(repr.plot.height = 20)

      gc()

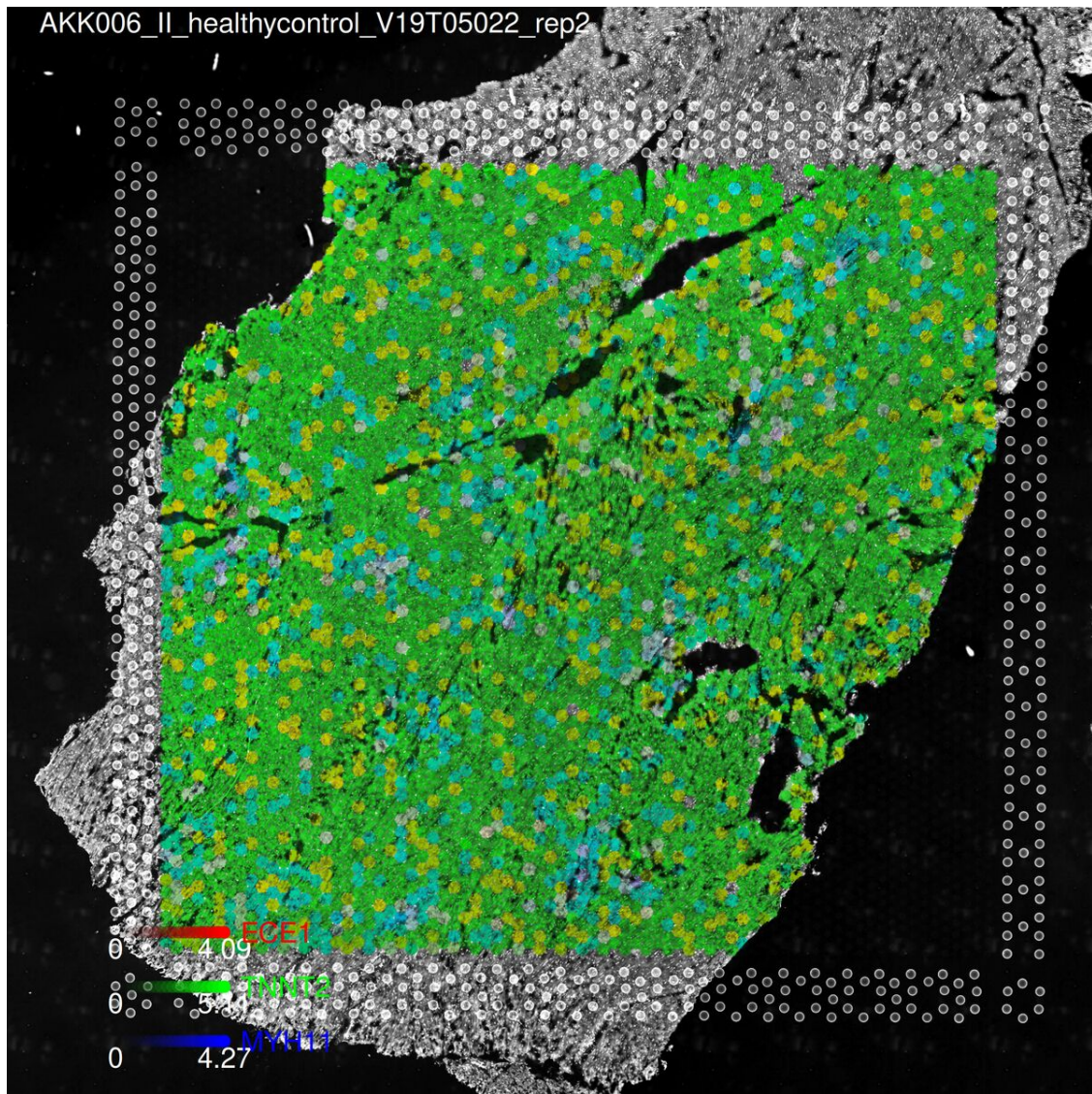
      for(i in c(1,2,3,4,5,6,7,9,10,11,12,13)){
        samples2plot = sample_id_list[i]
```

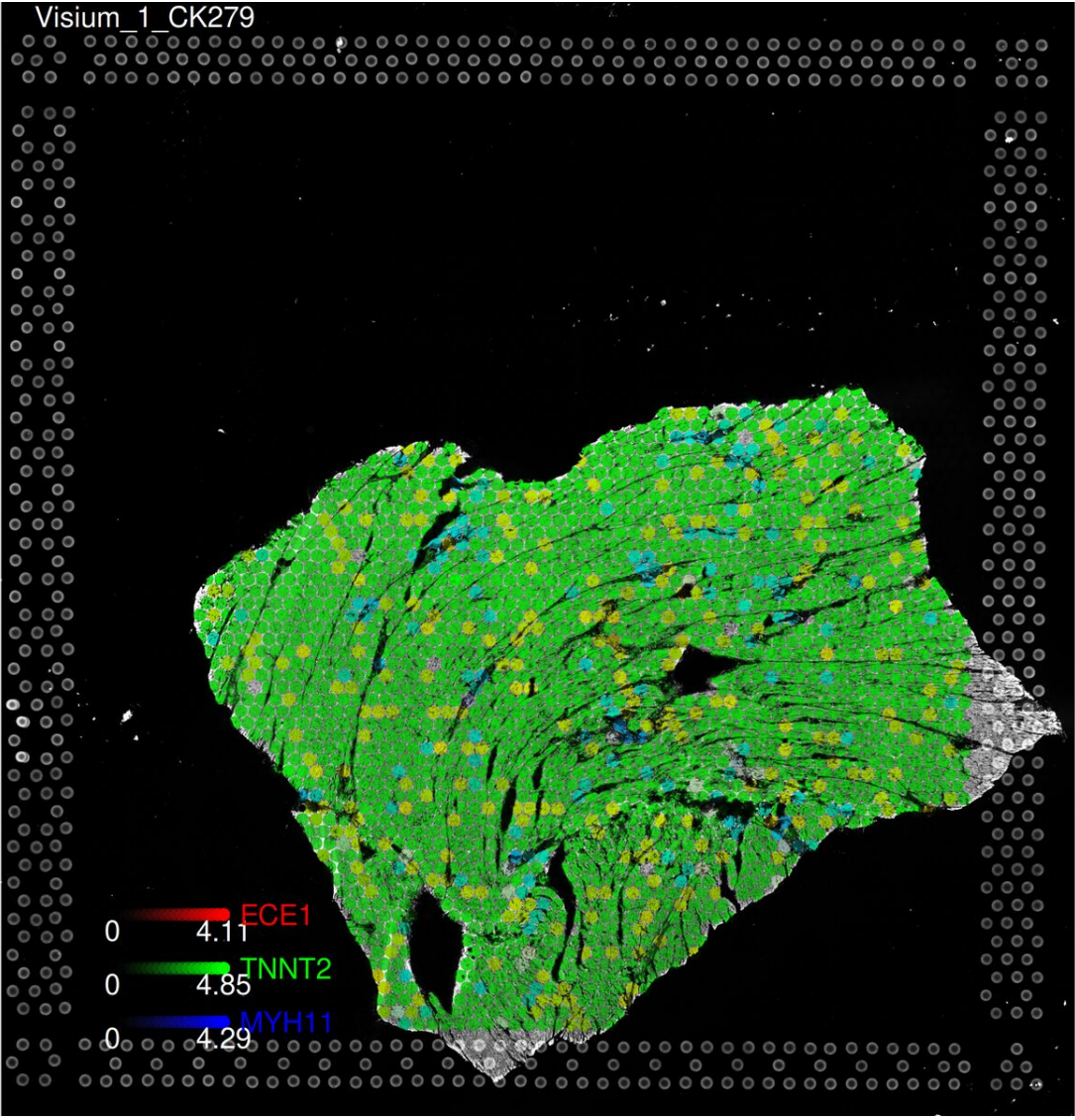
```

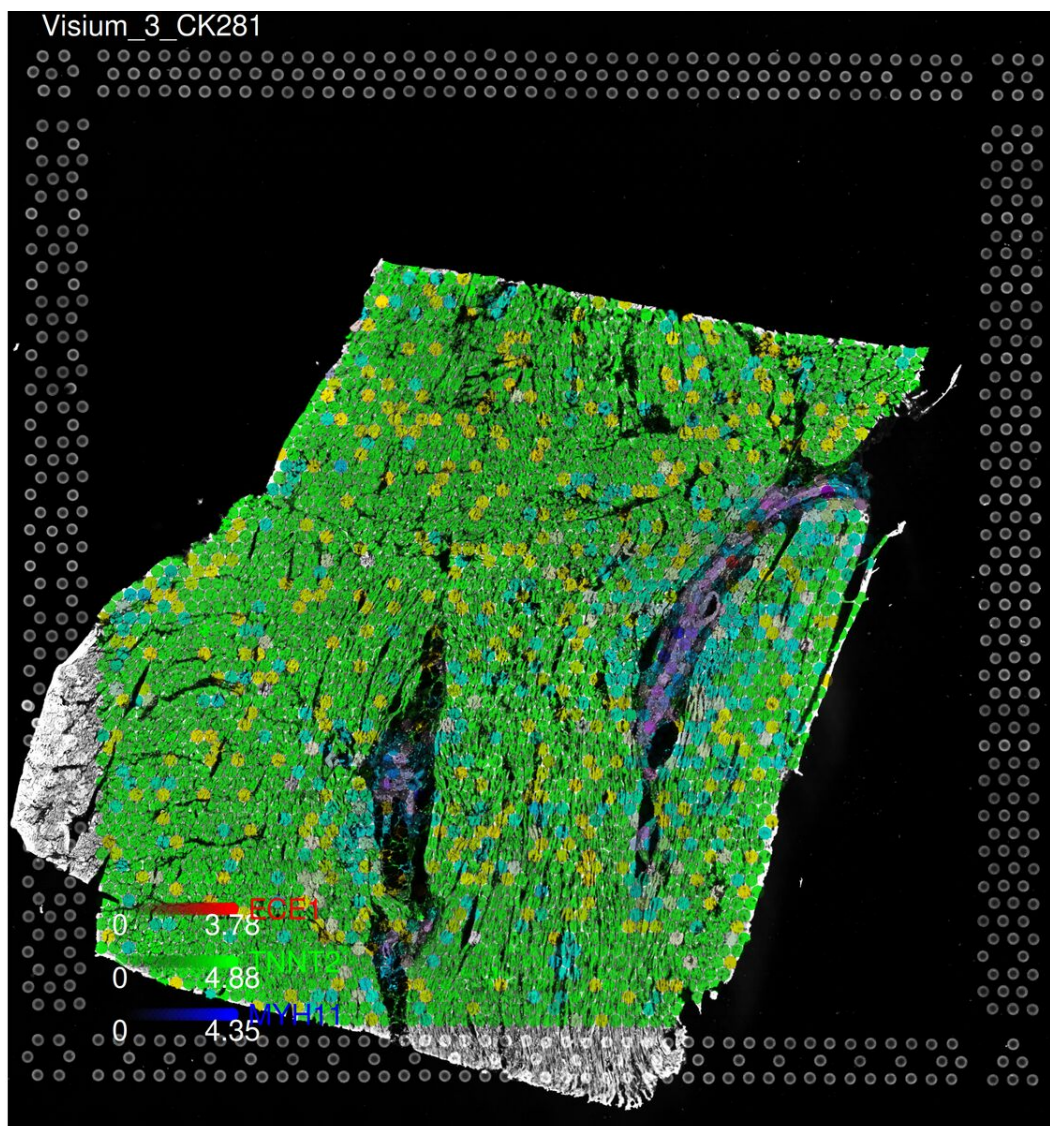
pdf(paste("figures/panels/results/Kuppe2022/ECE1_MYH7_MYH11_RGB_", gsub("^.",
↪*datasets/|.h5ad", "", datasets[i]), sample_id_list[i], ".pdf", sep=""),
↪height=20, width=20)
    plotVisium2(
        rgb.samples_list = VisiumData2(gene.r, gene.g, gene.b,
↪sample_id_list = samples2plot),
        light_or_dark = input$light_or_dark,
        plot_size = 1,
        mask_overlay = input$mask_overlay,
        min.col = input$min.col,
        red_on = sum("red" %in% input$spatial_channels),
        green_on = sum("green" %in% input$spatial_channels),
        blue_on = sum("blue" %in% input$spatial_channels),
        opacity_count_dependent =
↪sum(input$opacity_count_dependent),
        opacity = input$opacity,
        dot_or_not = 1,
        sample_id_list = samples2plot
    )
    dev.off()
    plotVisium2(
        rgb.samples_list = VisiumData2(gene.r, gene.g, gene.b,
↪sample_id_list = samples2plot),
        light_or_dark = input$light_or_dark,
        plot_size = 1,
        mask_overlay = input$mask_overlay,
        min.col = input$min.col,
        red_on = sum("red" %in% input$spatial_channels),
        green_on = sum("green" %in% input$spatial_channels),
        blue_on = sum("blue" %in% input$spatial_channels),
        opacity_count_dependent =
↪sum(input$opacity_count_dependent),
        opacity = input$opacity,
        dot_or_not = 1,
        sample_id_list = samples2plot
    )
}

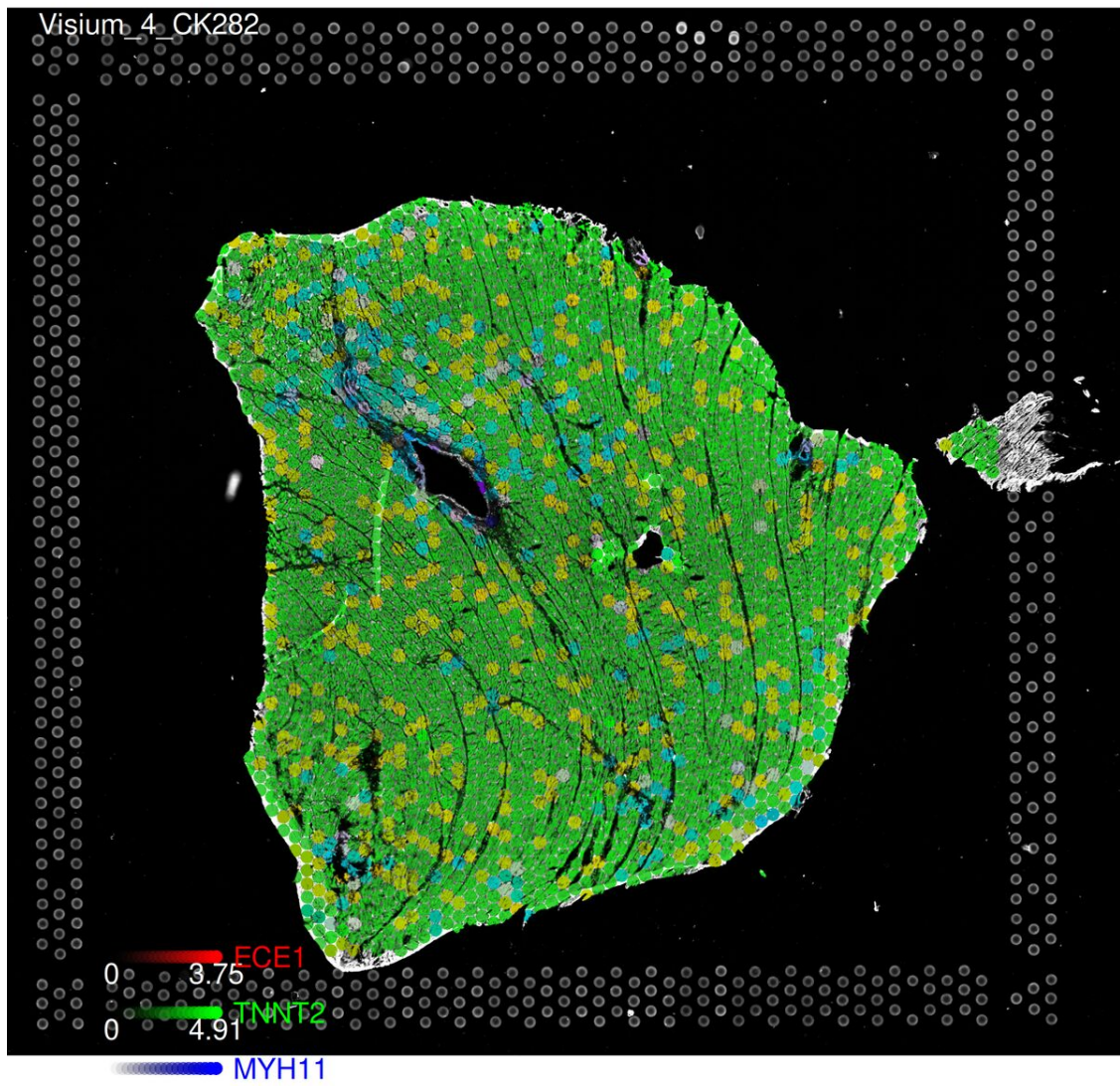
```

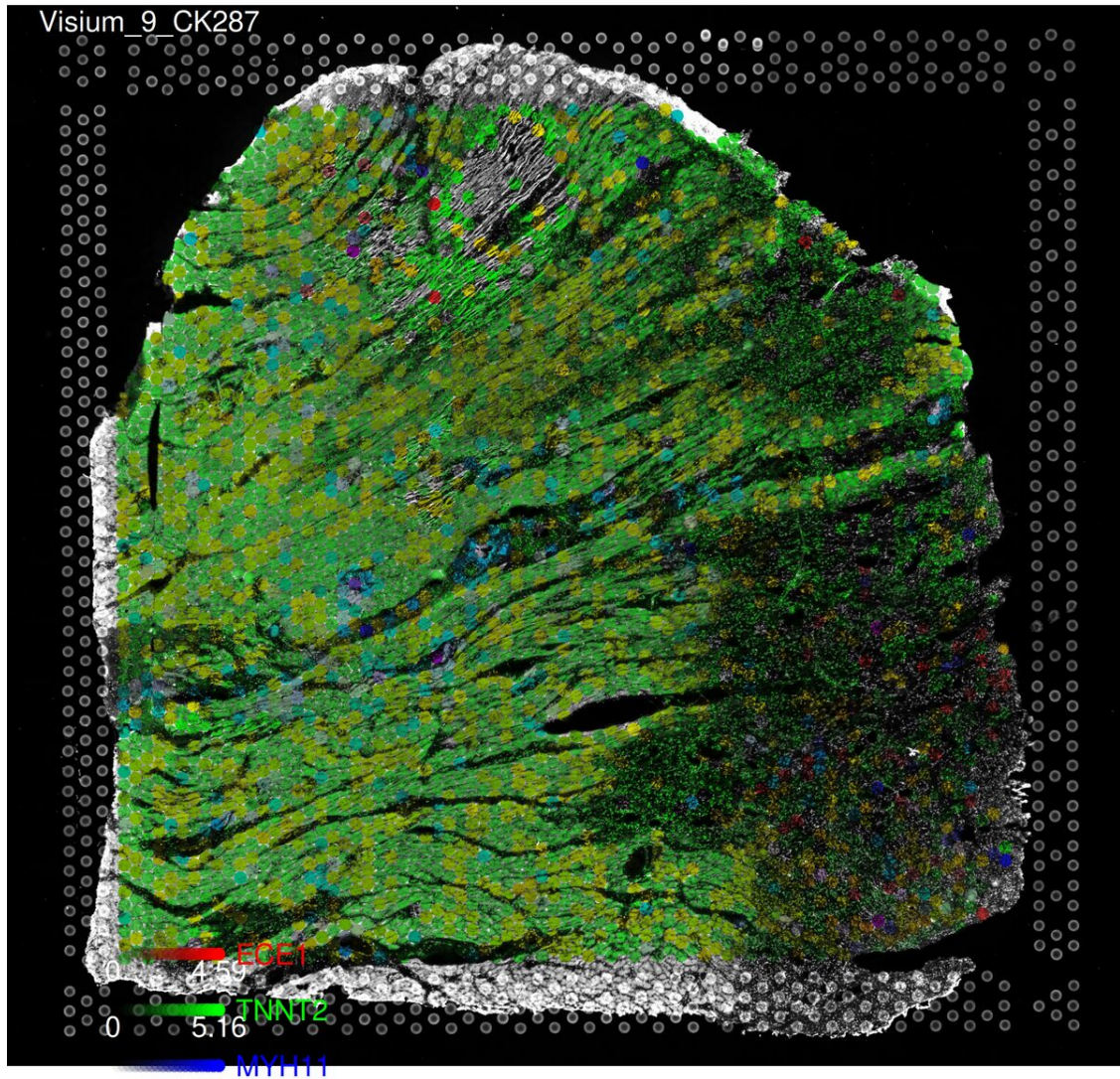
		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2 × 6 of type dbl	Ncells	2187741	116.9	4262370	227.7	4262370	227.7
	Vcells	406539845	3101.7	751169477	5731.0	751169477	5731.0

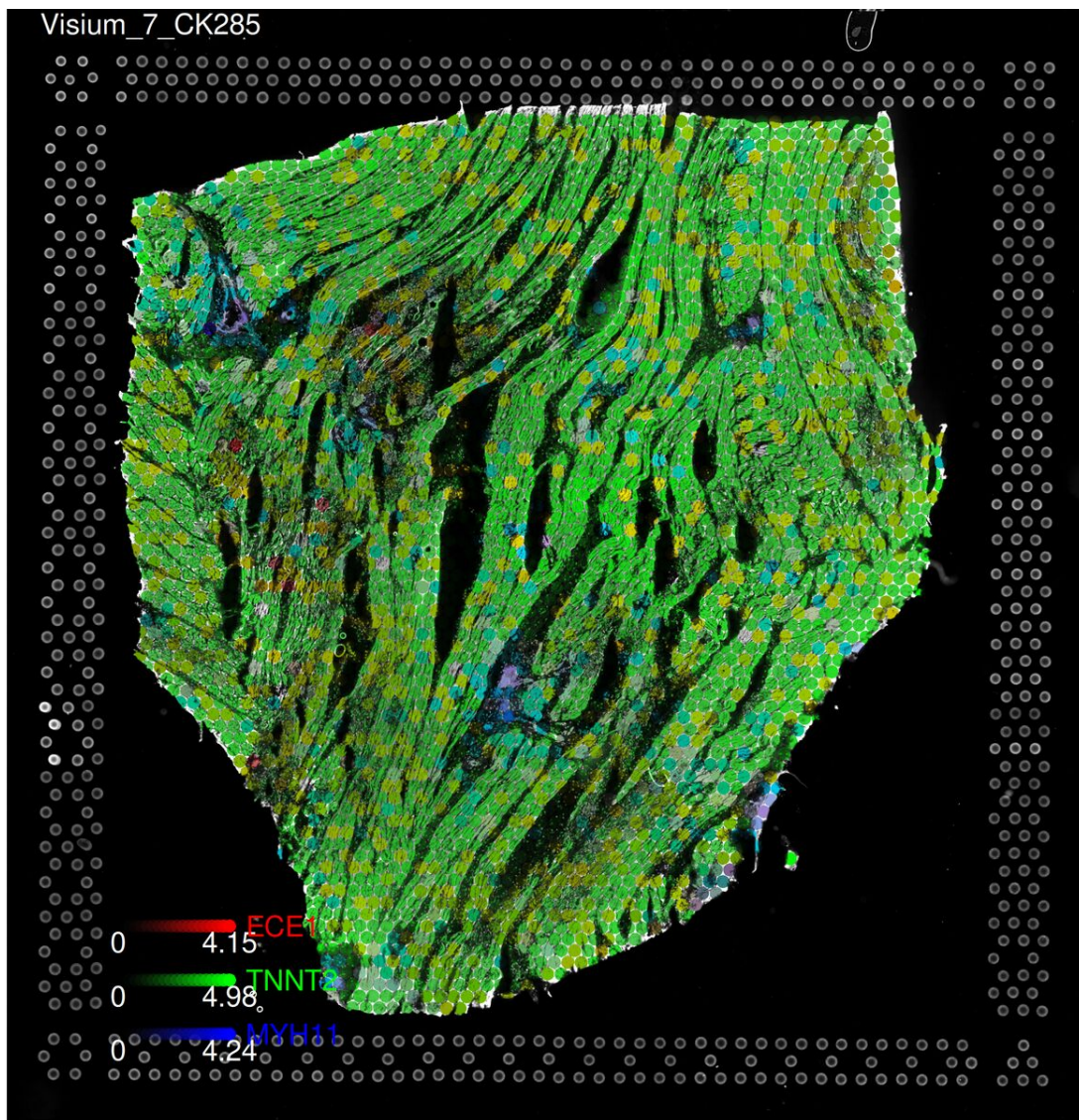


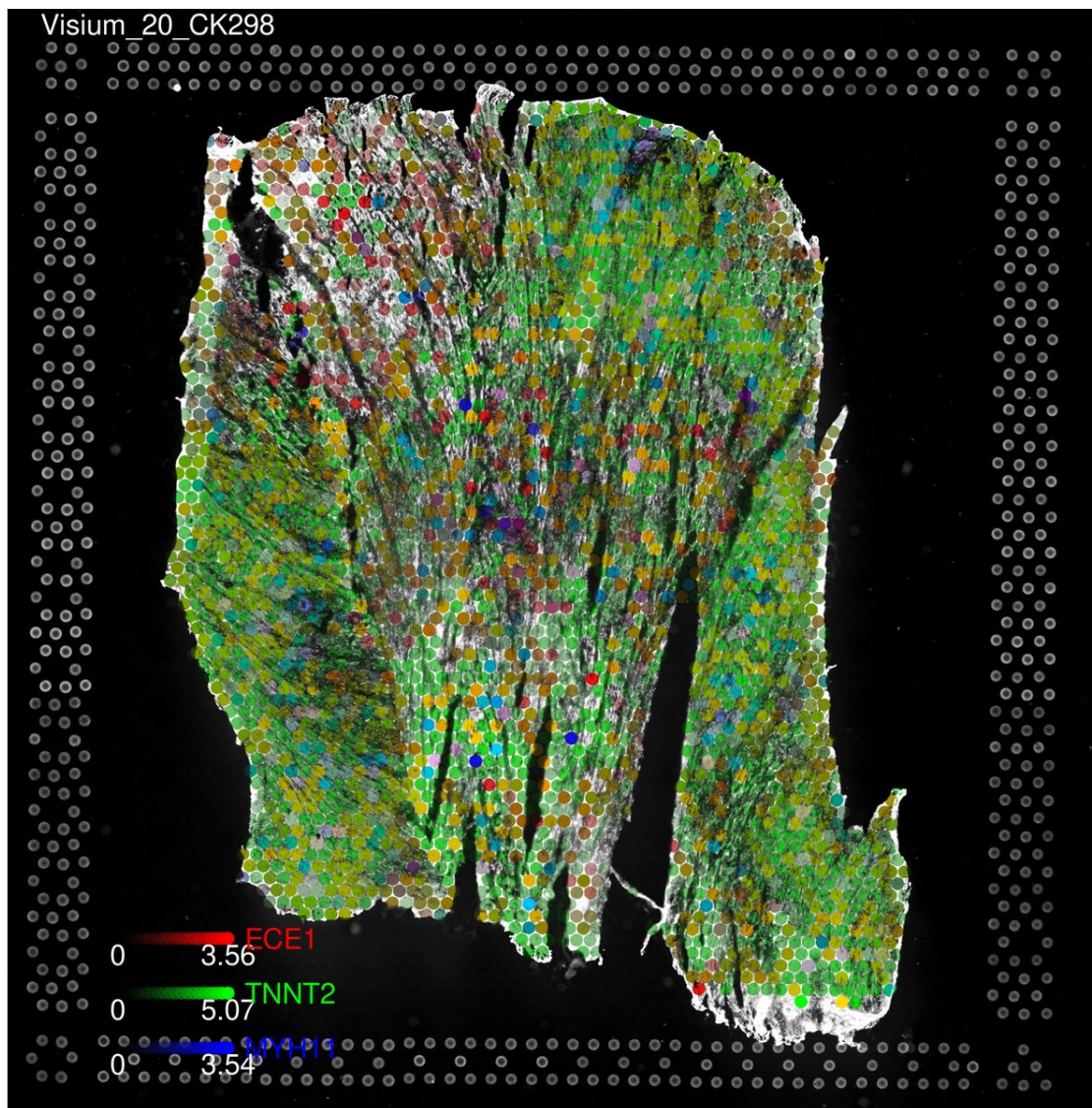




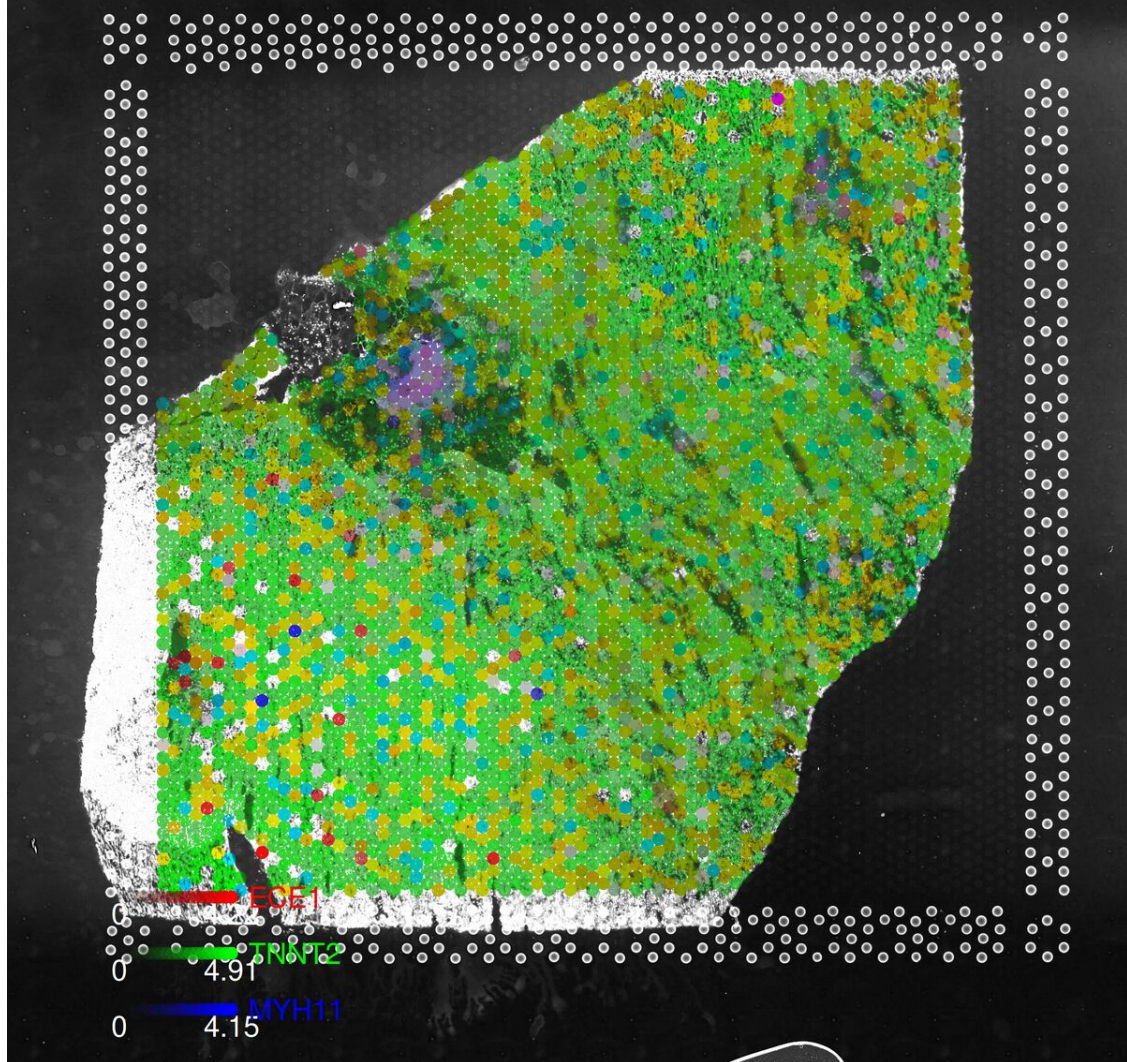




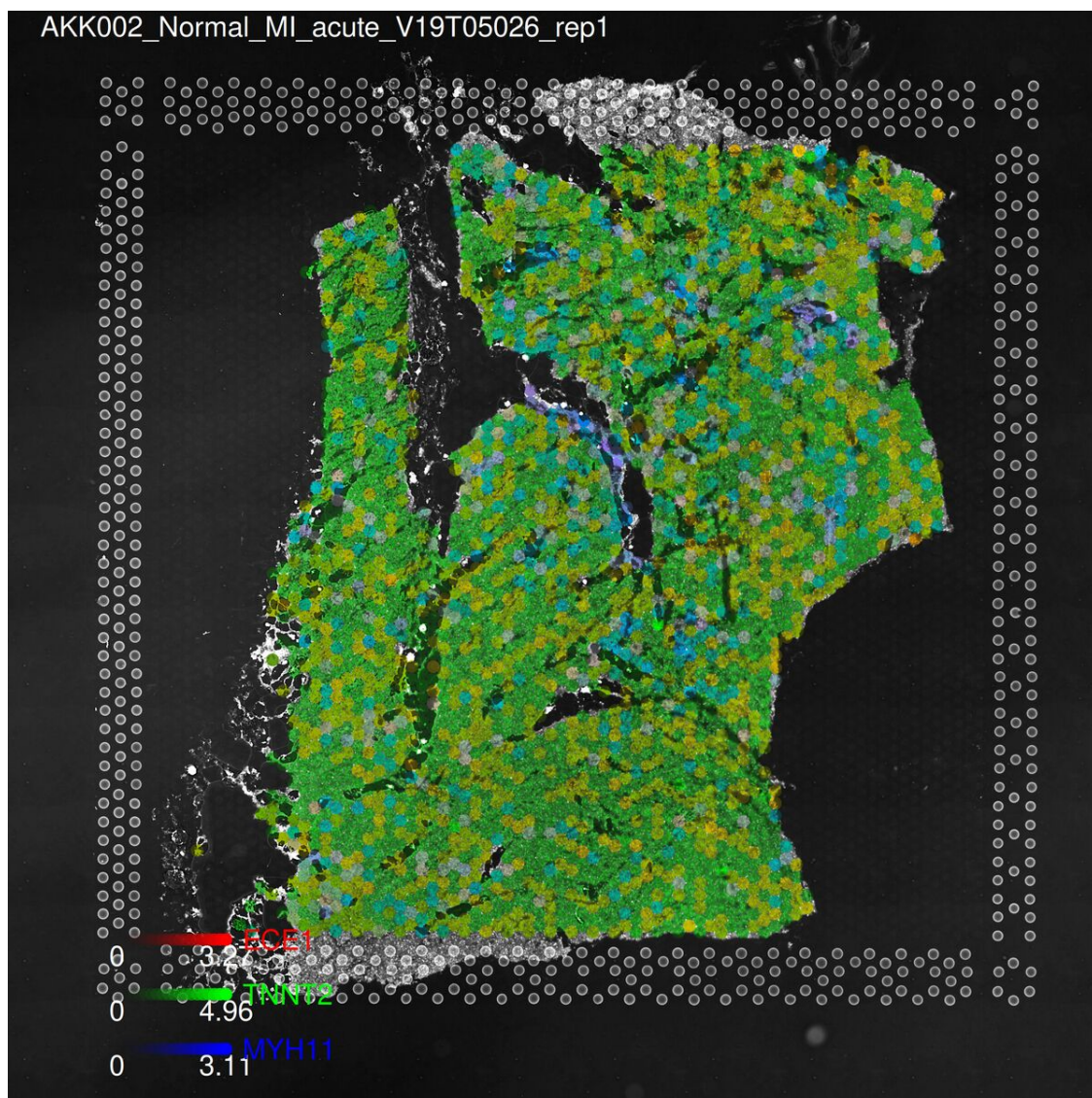


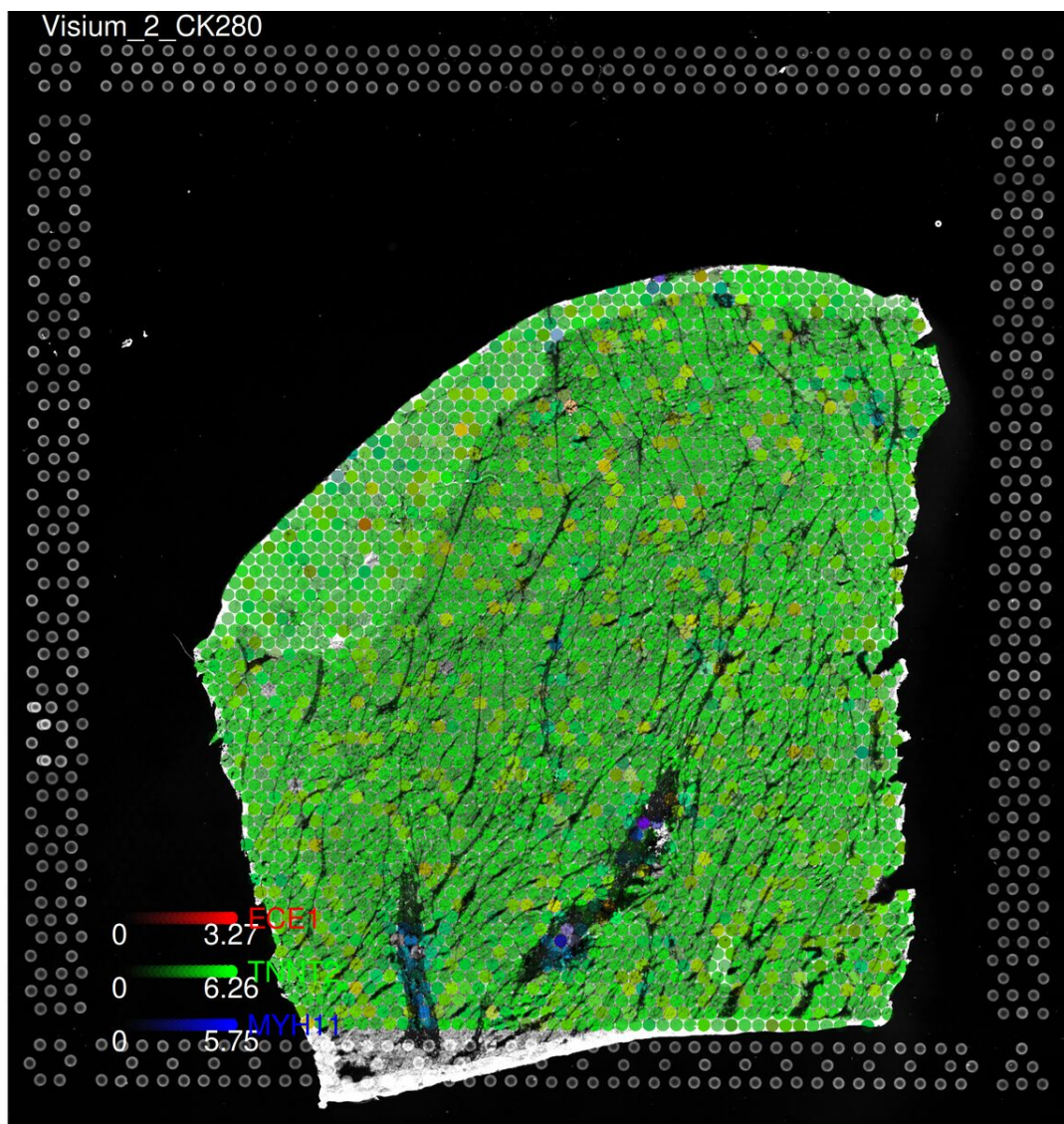


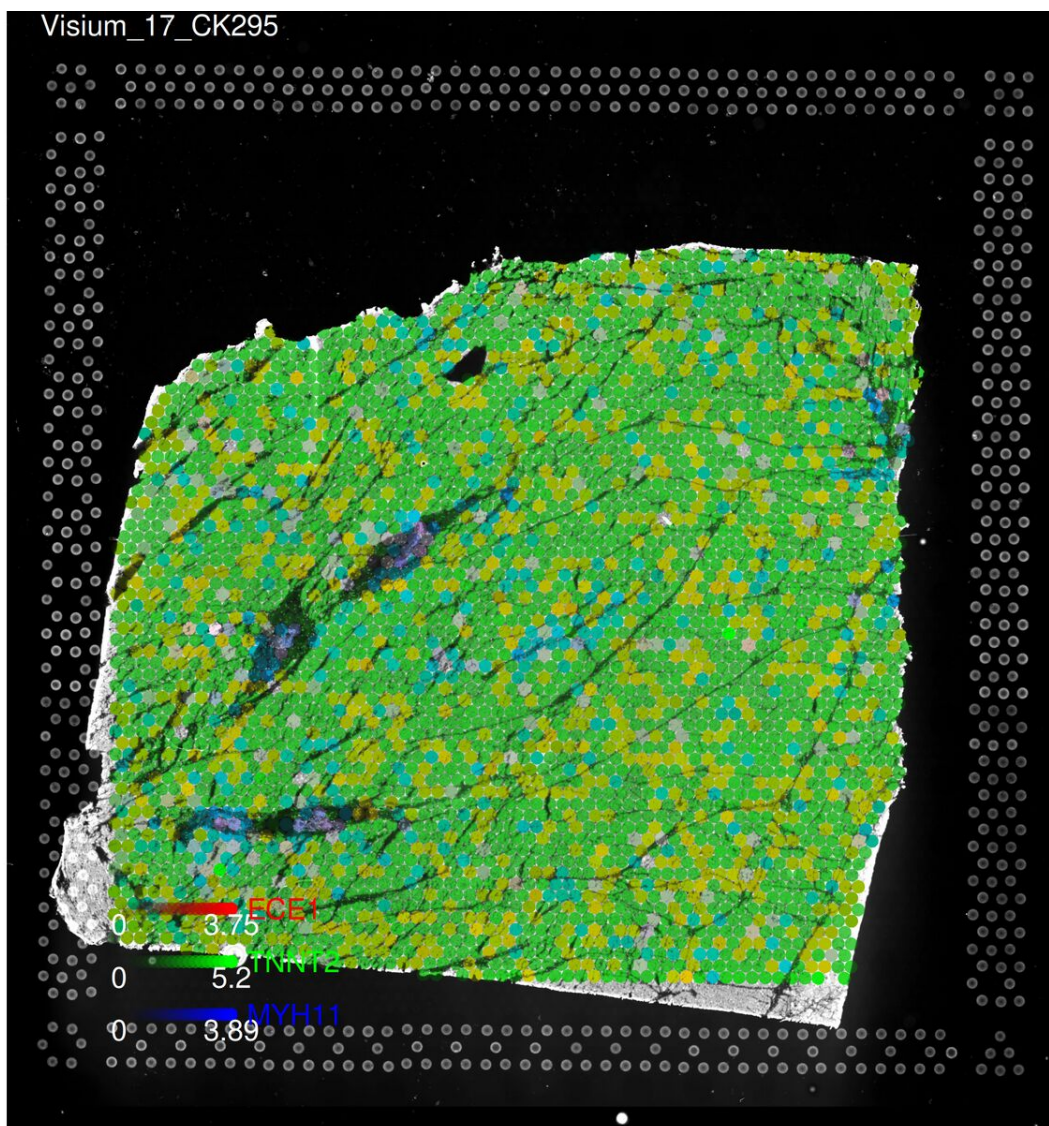
AKK002_Infarct_MI_acute_V19T05025_rep2

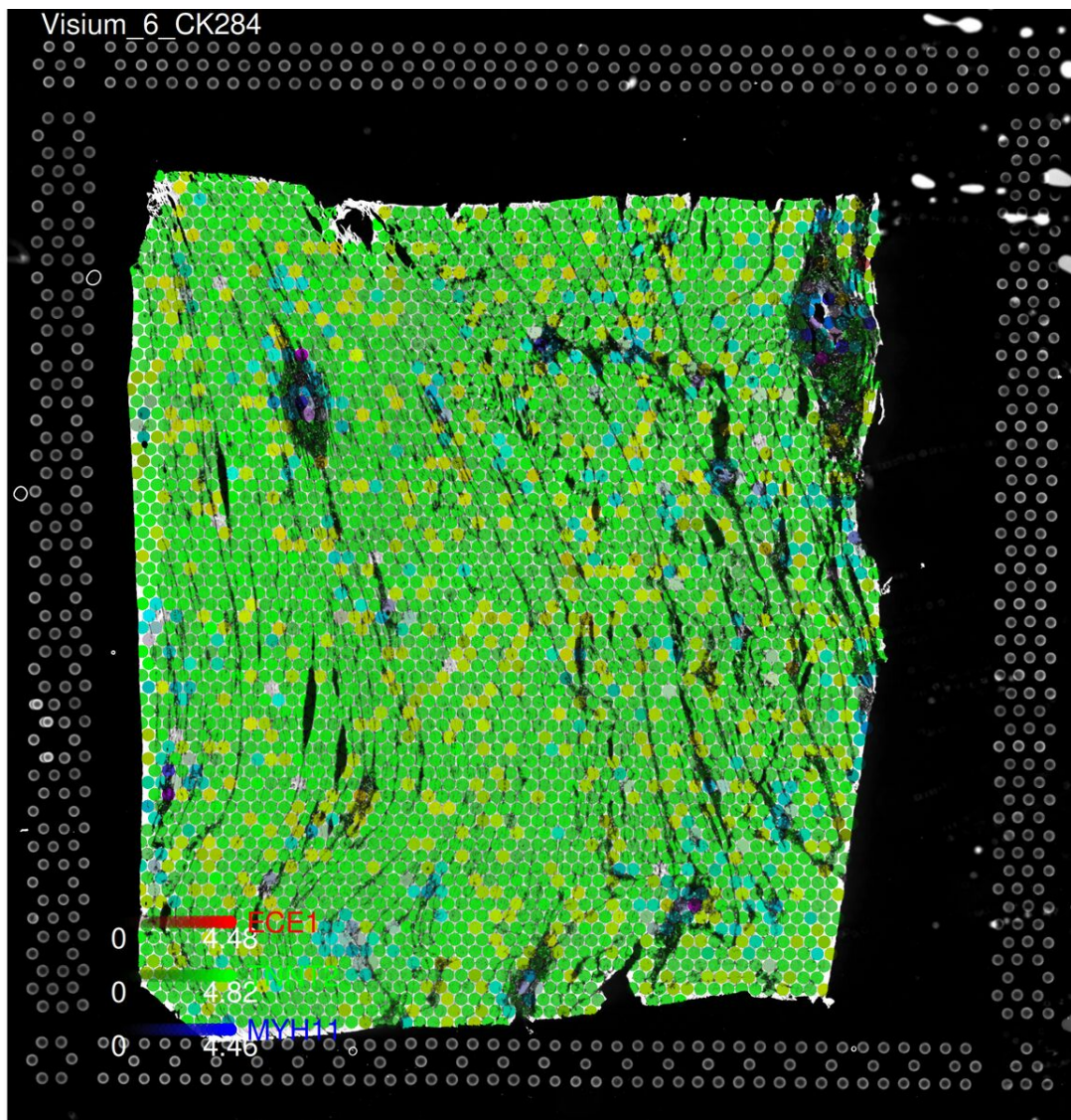


AKK002_Normal_MI_acute_V19T05026_rep1









1.2.3 ECE1 + CYGB + MYH11

```
[11]: options(repr.plot.width = 20)
      options(repr.plot.height = 20)

      for(i in c(1,2,3,4,5,6,7,9,10,11,12,13)){
        samples2plot = sample_id_list[i]
        pdf(paste("figures/panels/results/Kuppe2022/ECE1_CYGB_MYH11_RGB_", gsub("^.",
          ↳*datasets/|.h5ad", "", datasets[i]), sample_id_list[i], ".pdf", sep=""),
          ↳height=20, width=20)

        gene.r = "ECE1"
        gene.g = "CYGB"
```

```

gene.b = "MYH11"

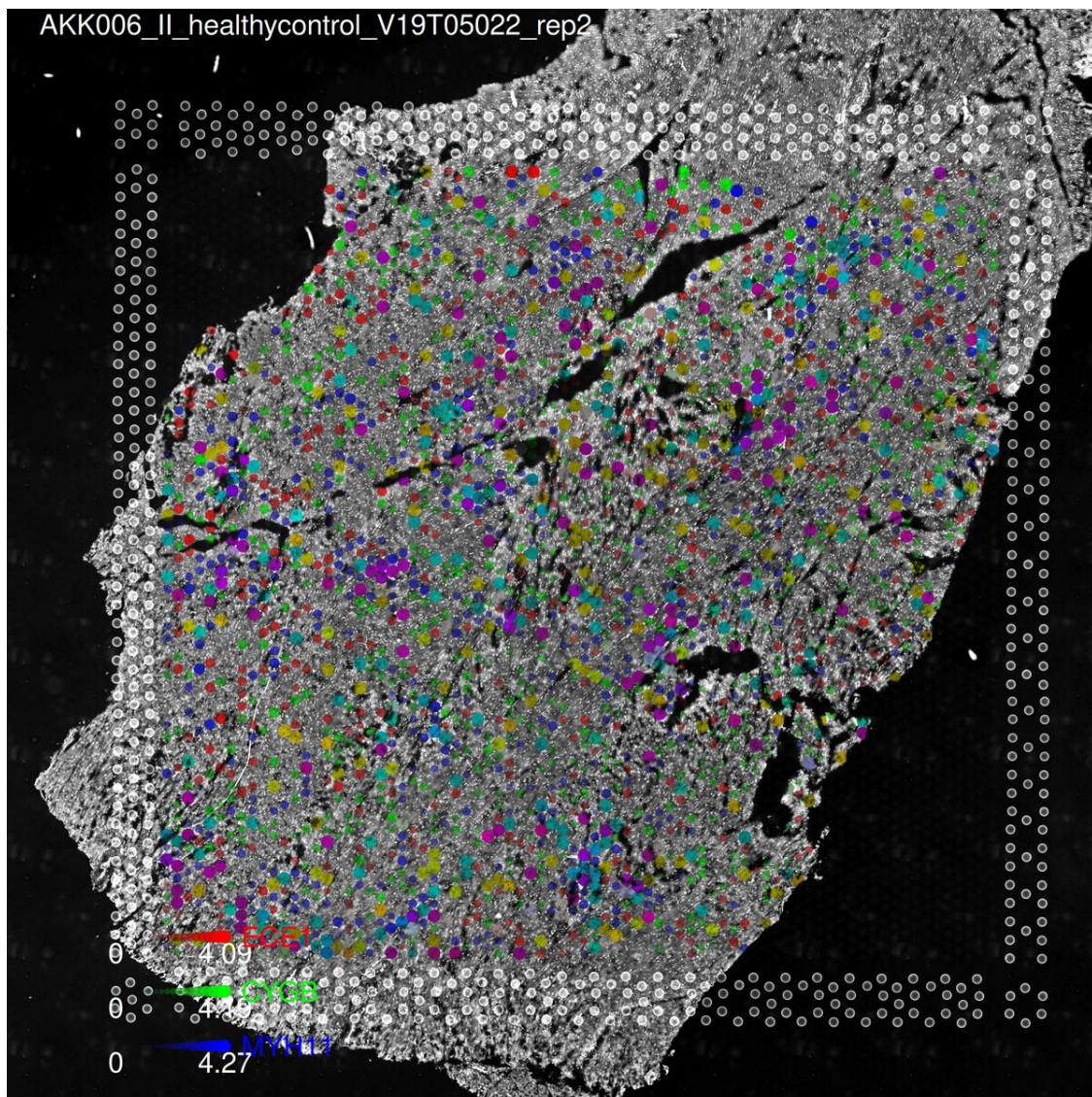
plotVisium2(
    rgb.samples_list = VisiumData2(gene.r, gene.g, gene.b,
    ↪sample_id_list = samples2plot),
    light_or_dark = 2,
    plot_size = 1,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    red_on = sum("red" %in% input$spatial_channels),
    green_on = sum("green" %in% input$spatial_channels),
    blue_on = sum("blue" %in% input$spatial_channels),
    opacity_count_dependent =
    ↪sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = 2,
    sample_id_list = samples2plot
)

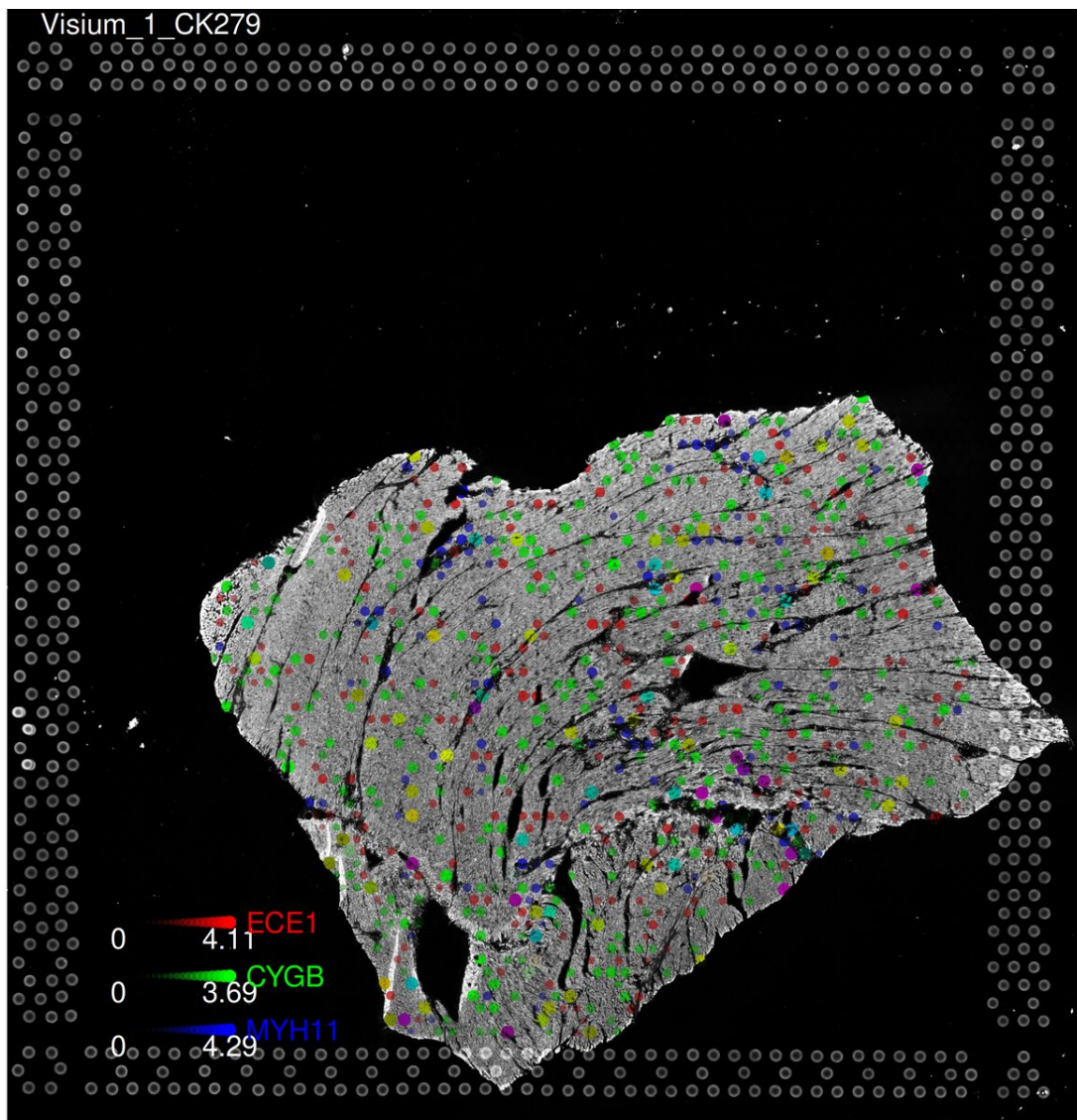
dev.off()

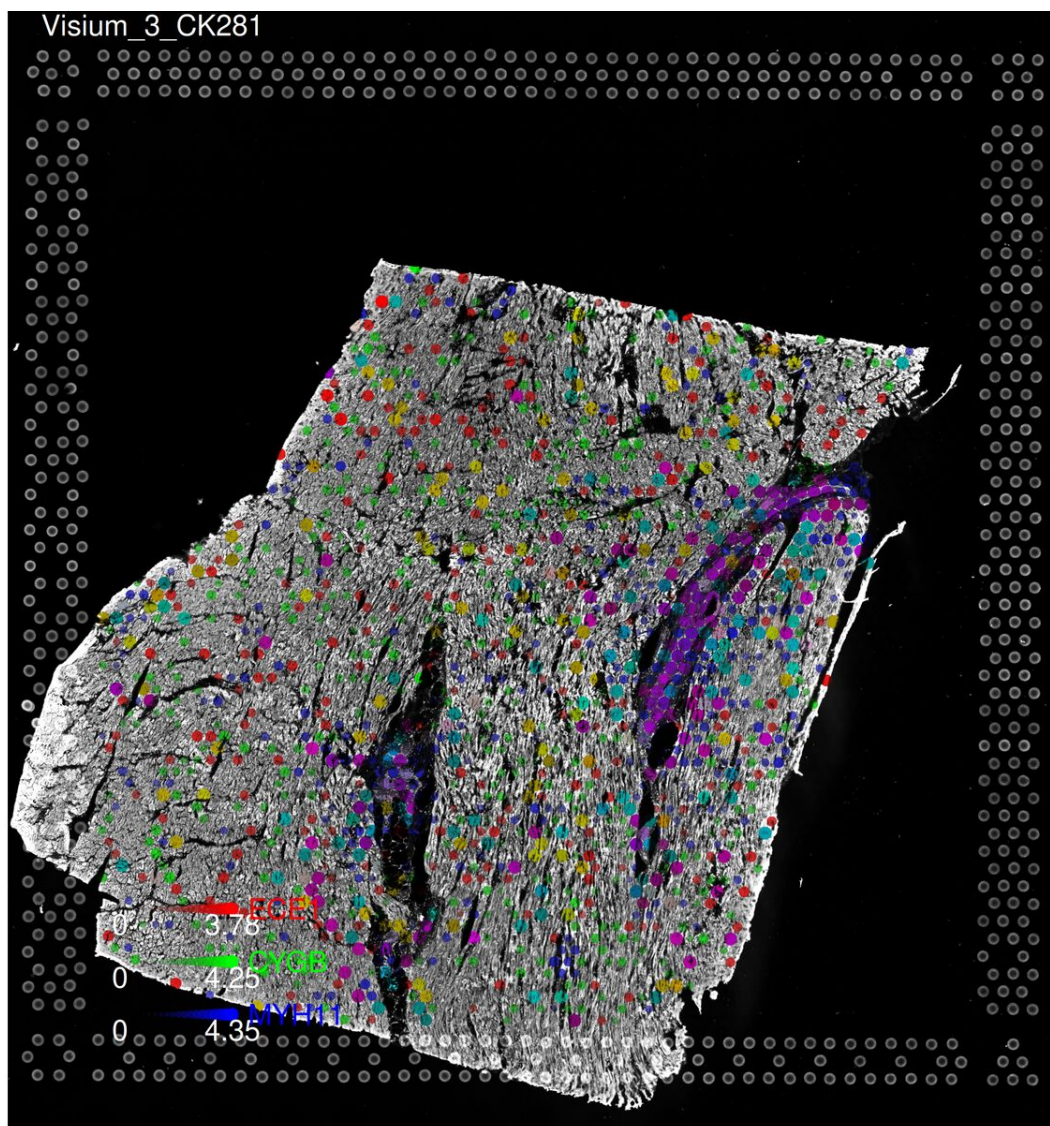
plotVisium2(
    rgb.samples_list = VisiumData2(gene.r, gene.g, gene.b,
    ↪sample_id_list = samples2plot),
    light_or_dark = 2,
    plot_size = 1,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    red_on = sum("red" %in% input$spatial_channels),
    green_on = sum("green" %in% input$spatial_channels),
    blue_on = sum("blue" %in% input$spatial_channels),
    opacity_count_dependent =
    ↪sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = 2,
    sample_id_list = samples2plot
)

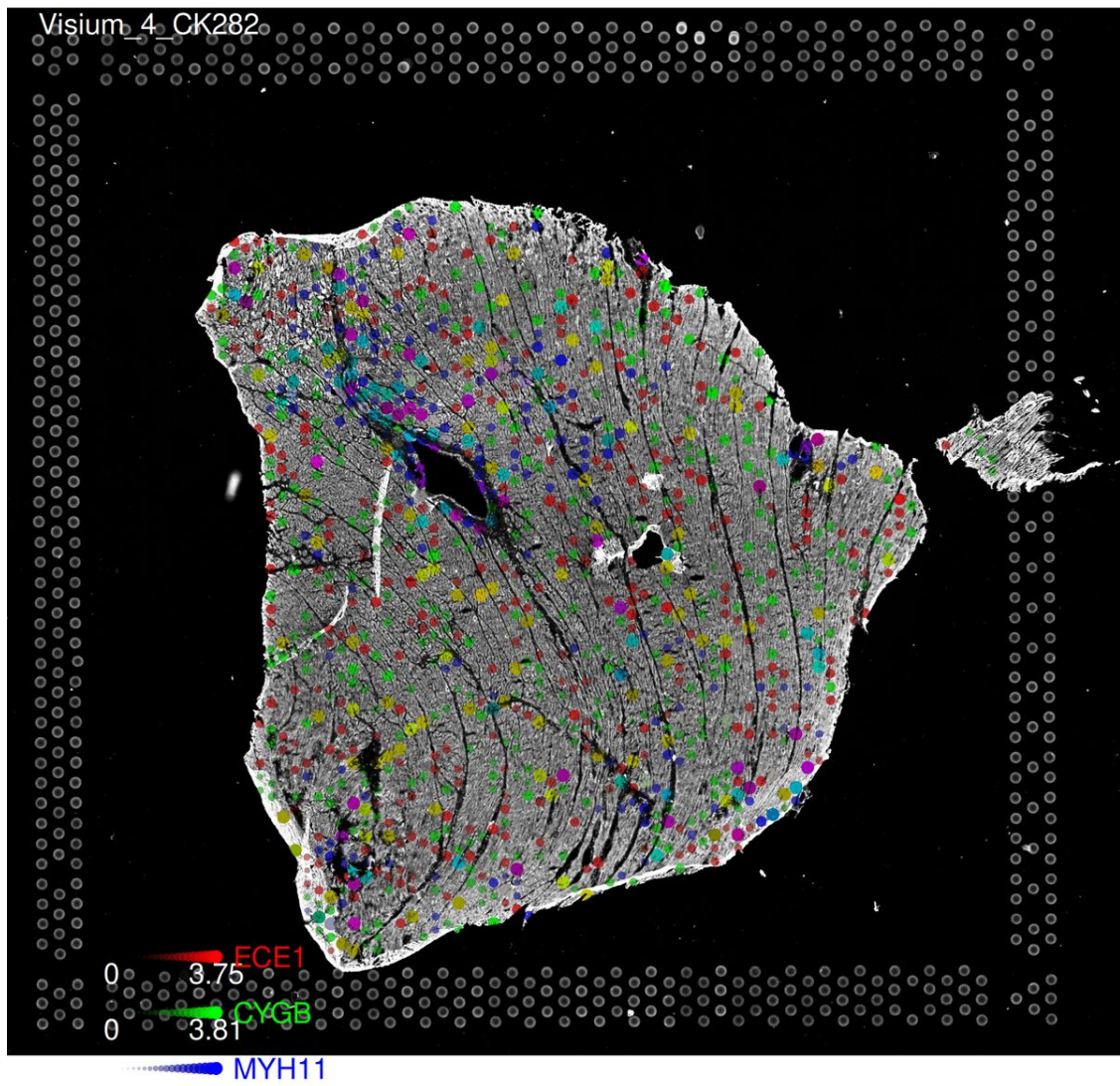
}

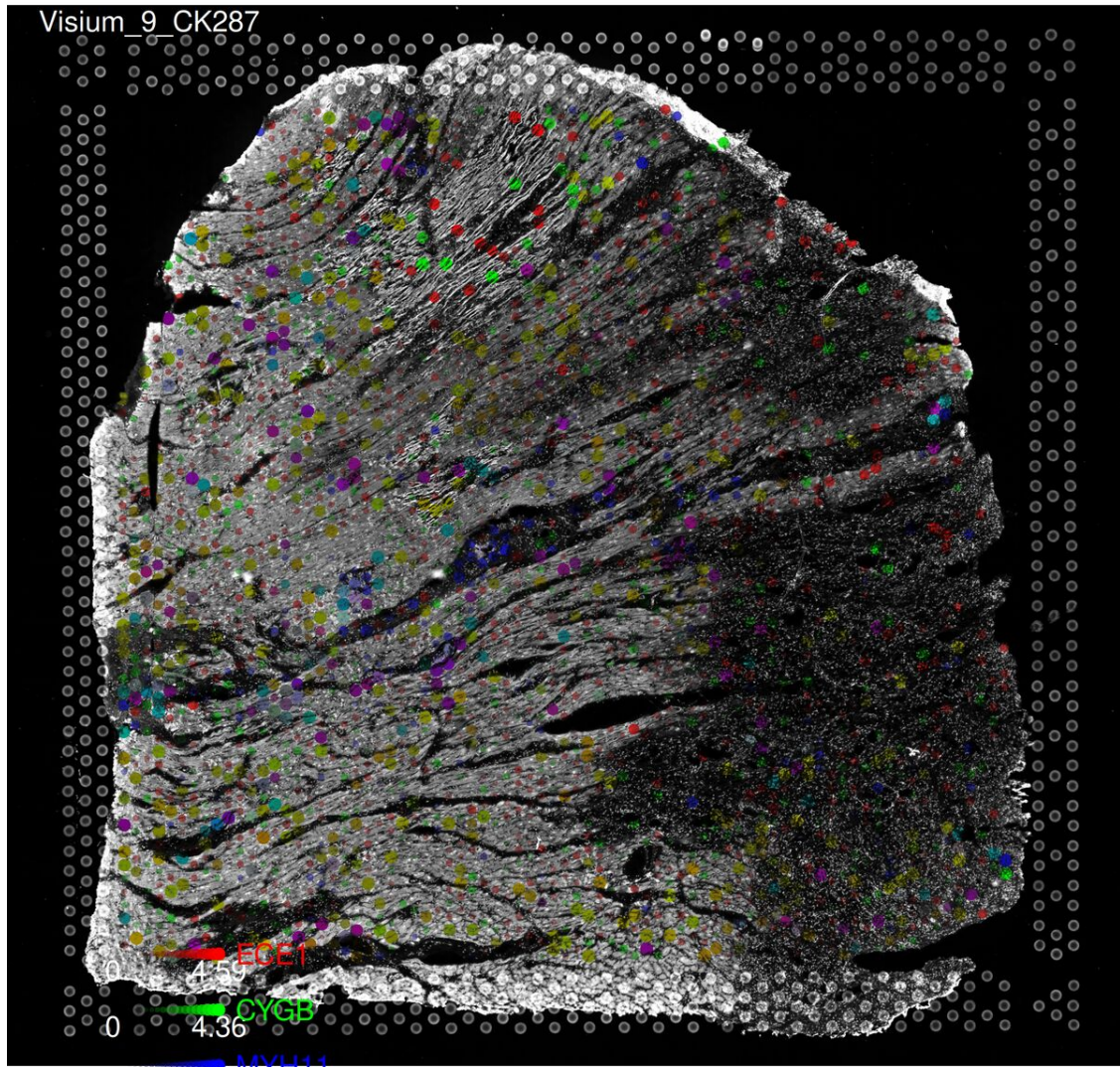
```

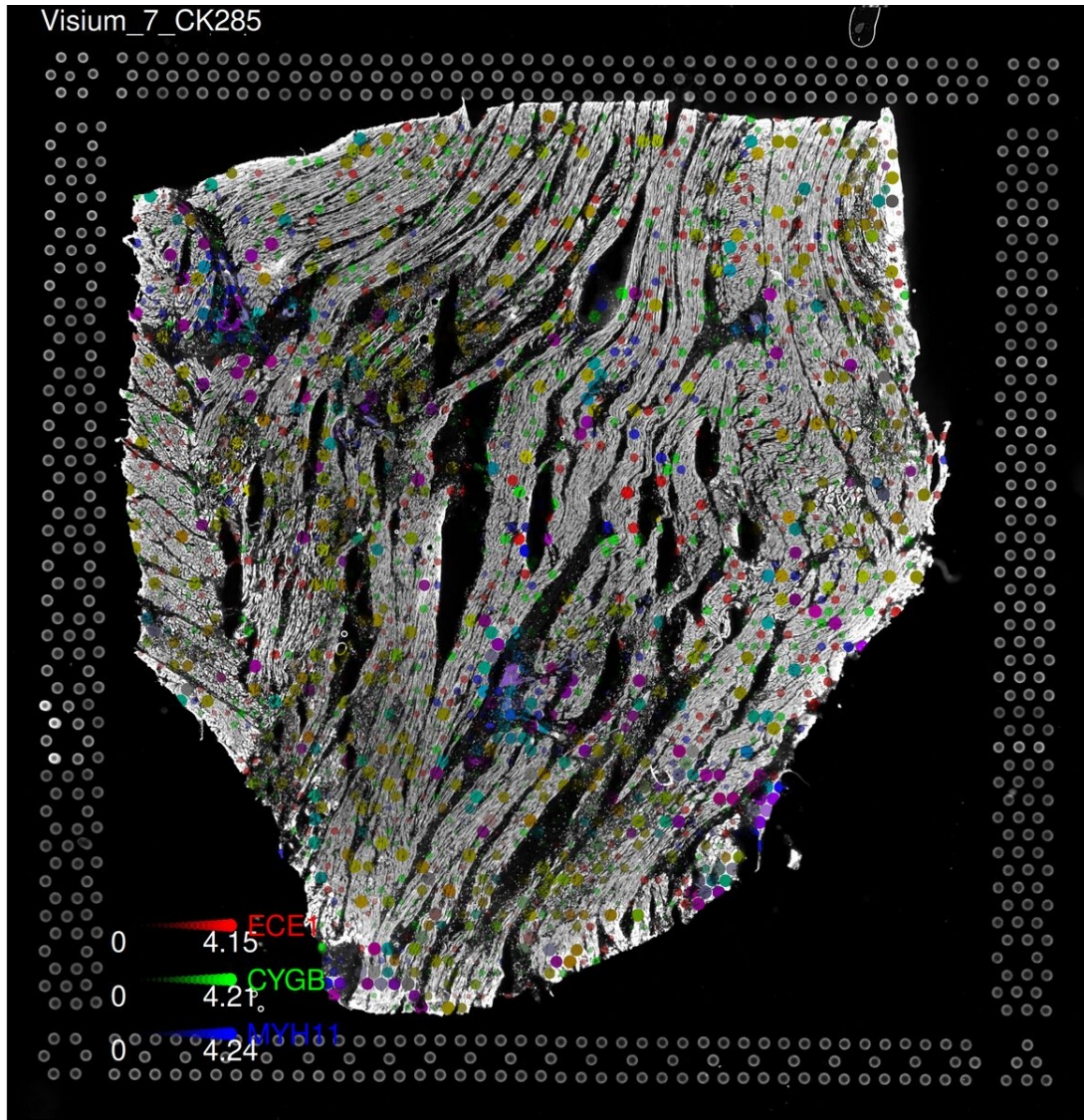



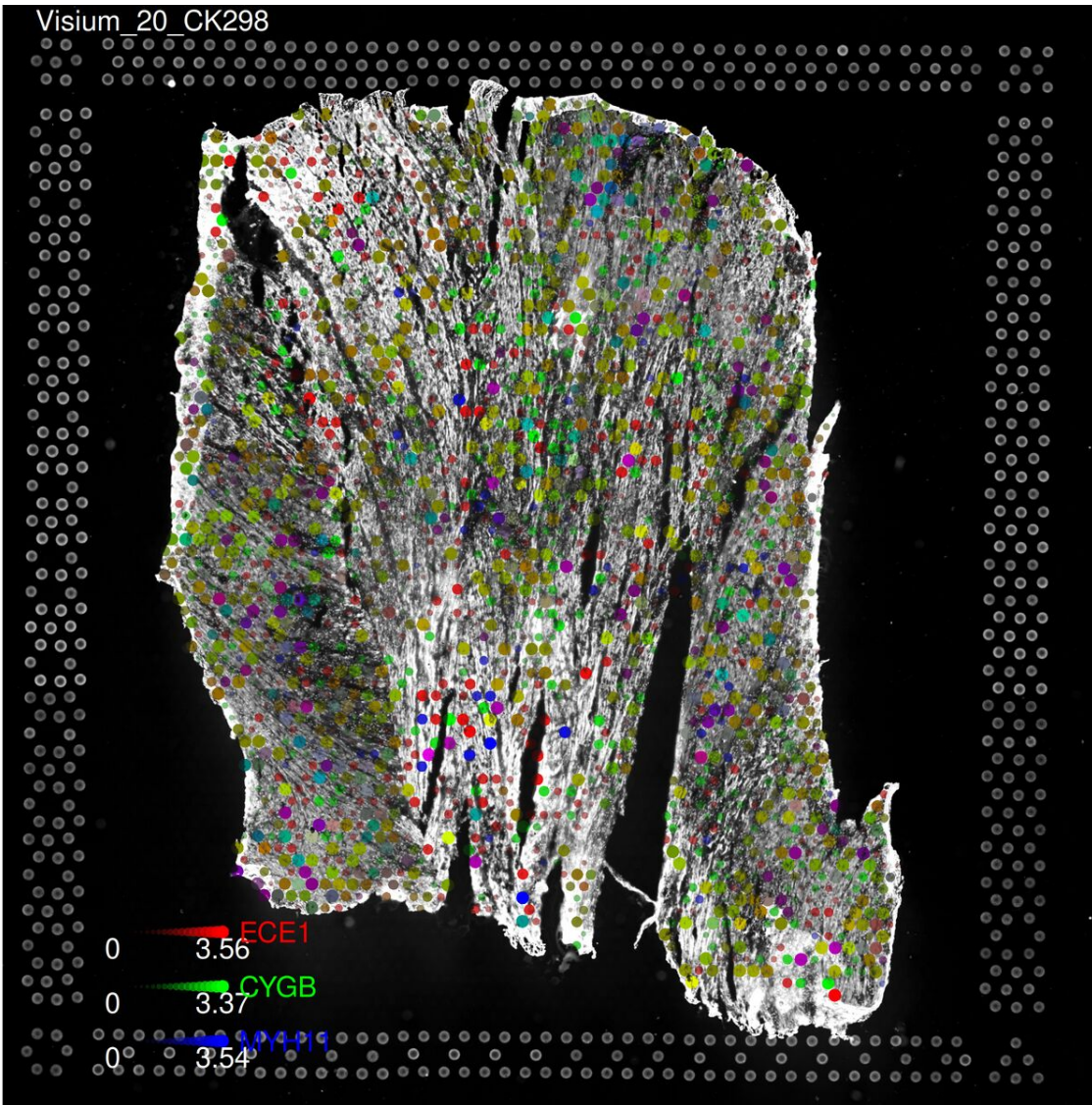




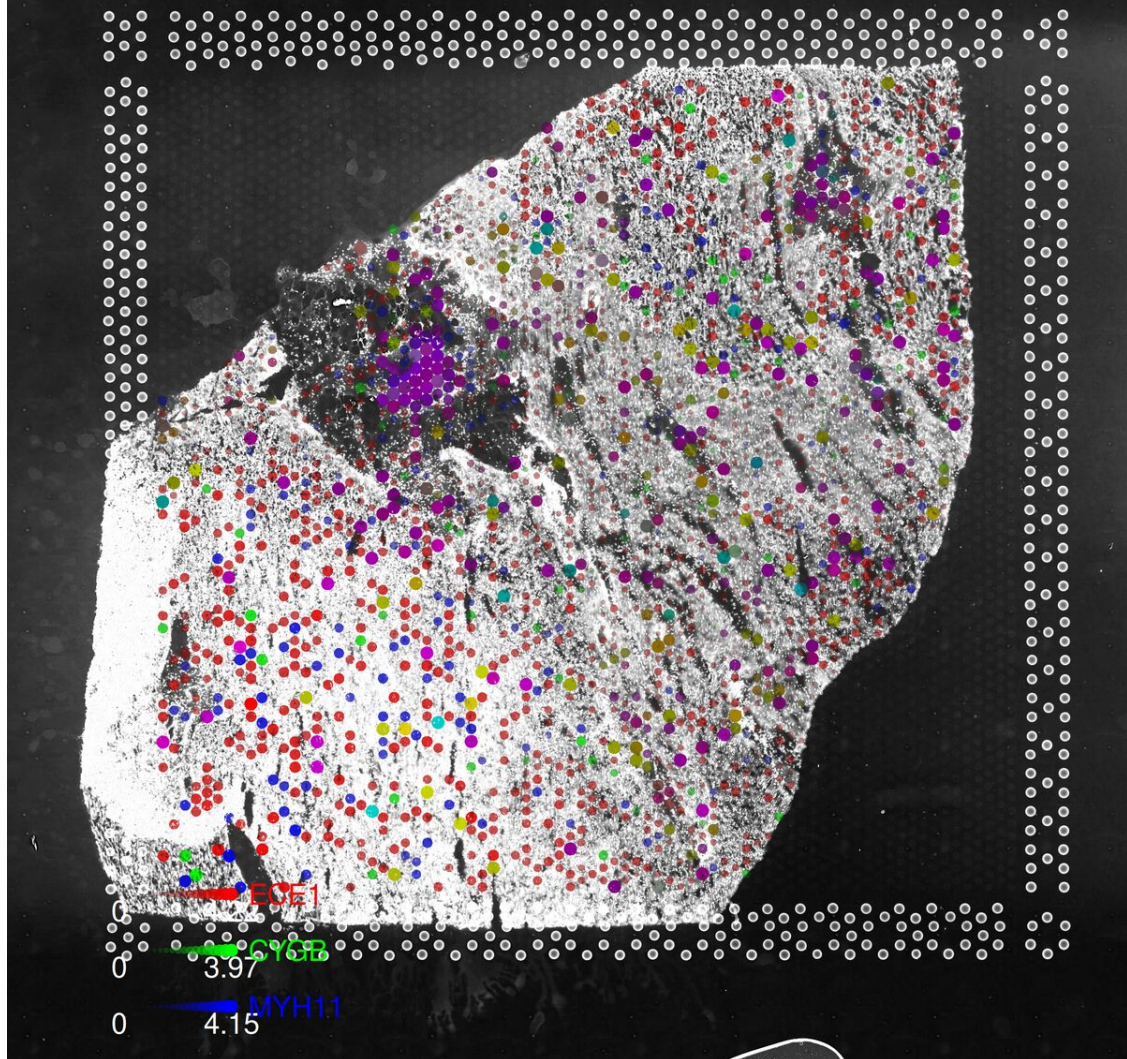




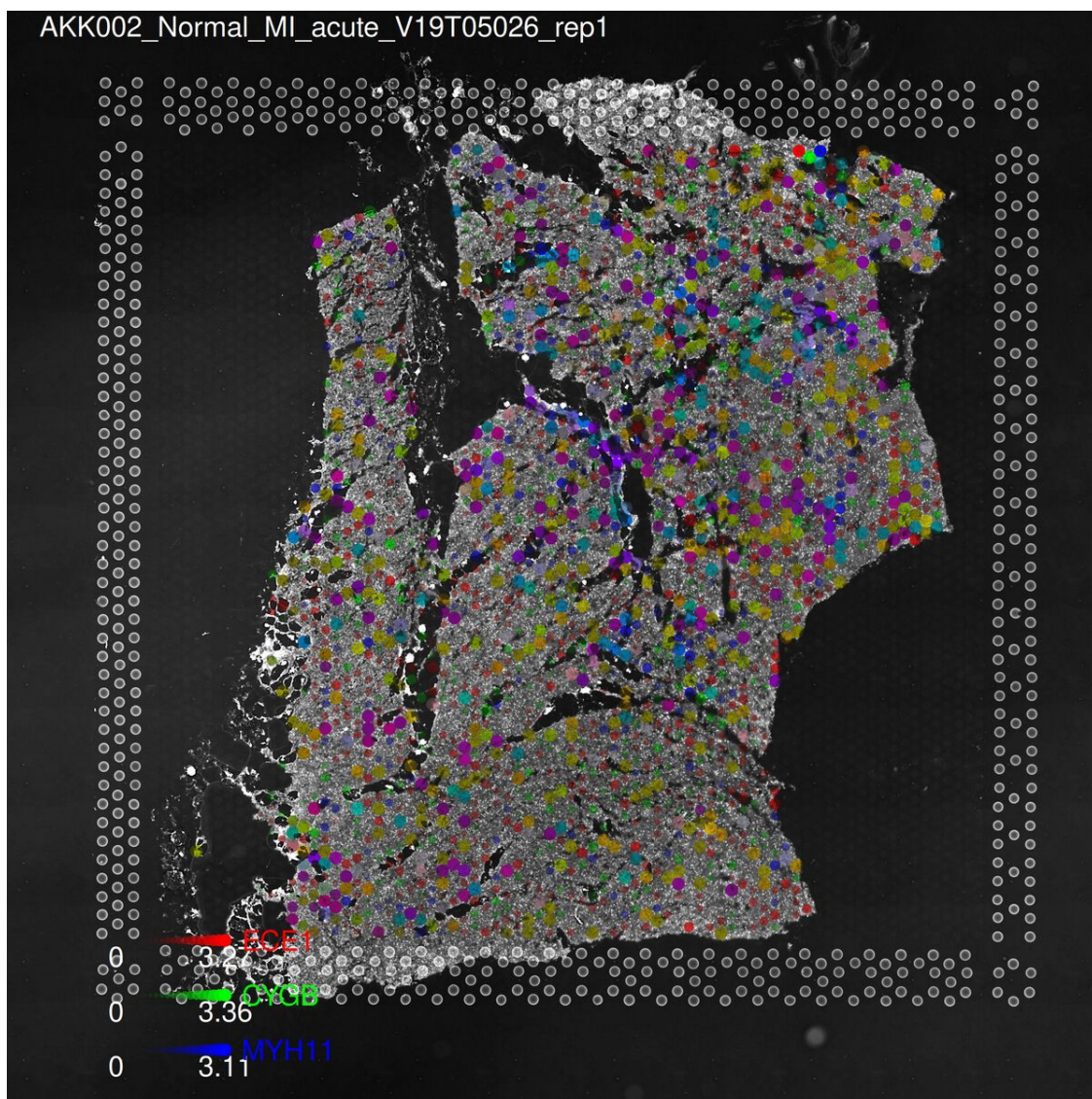


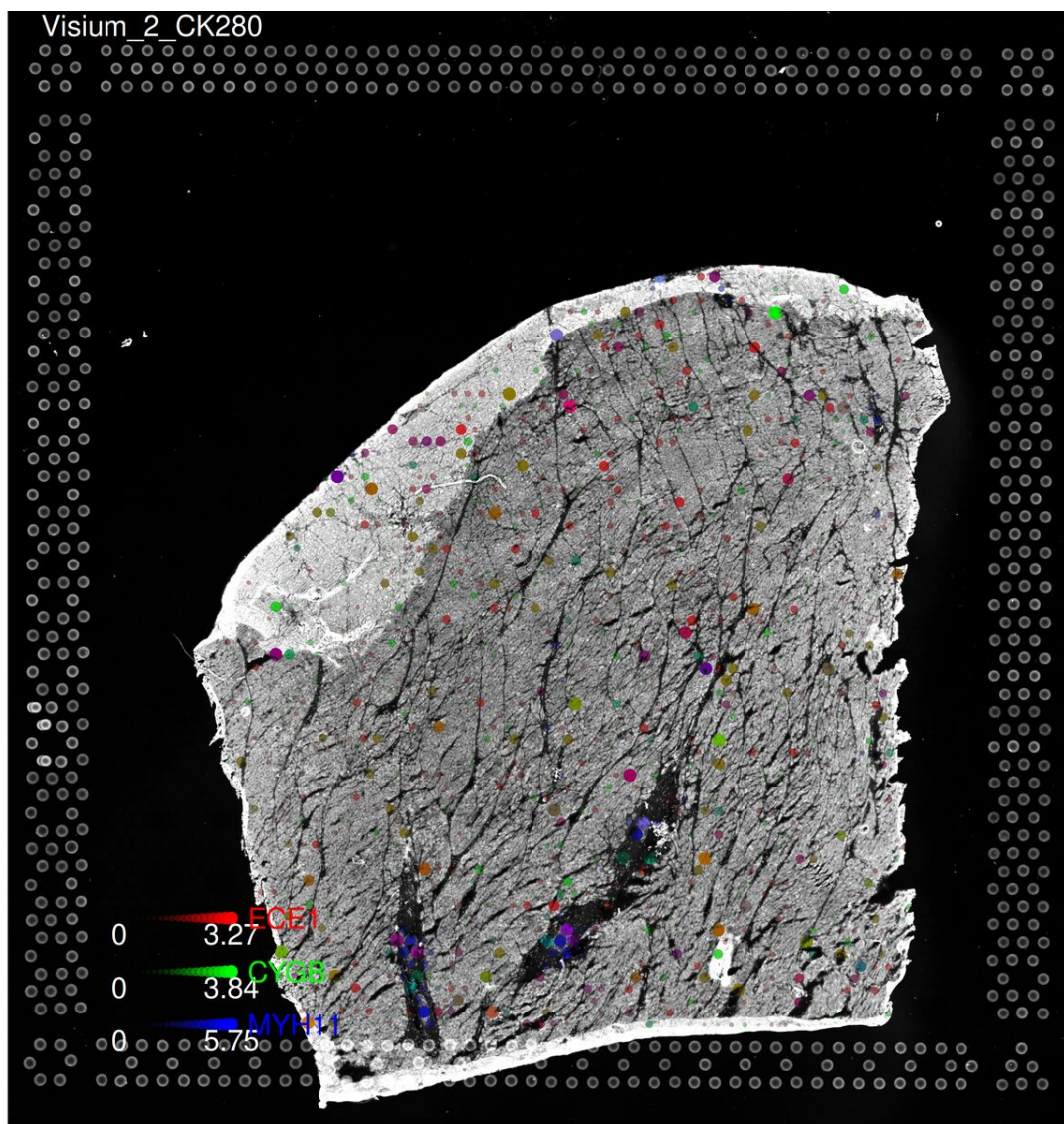


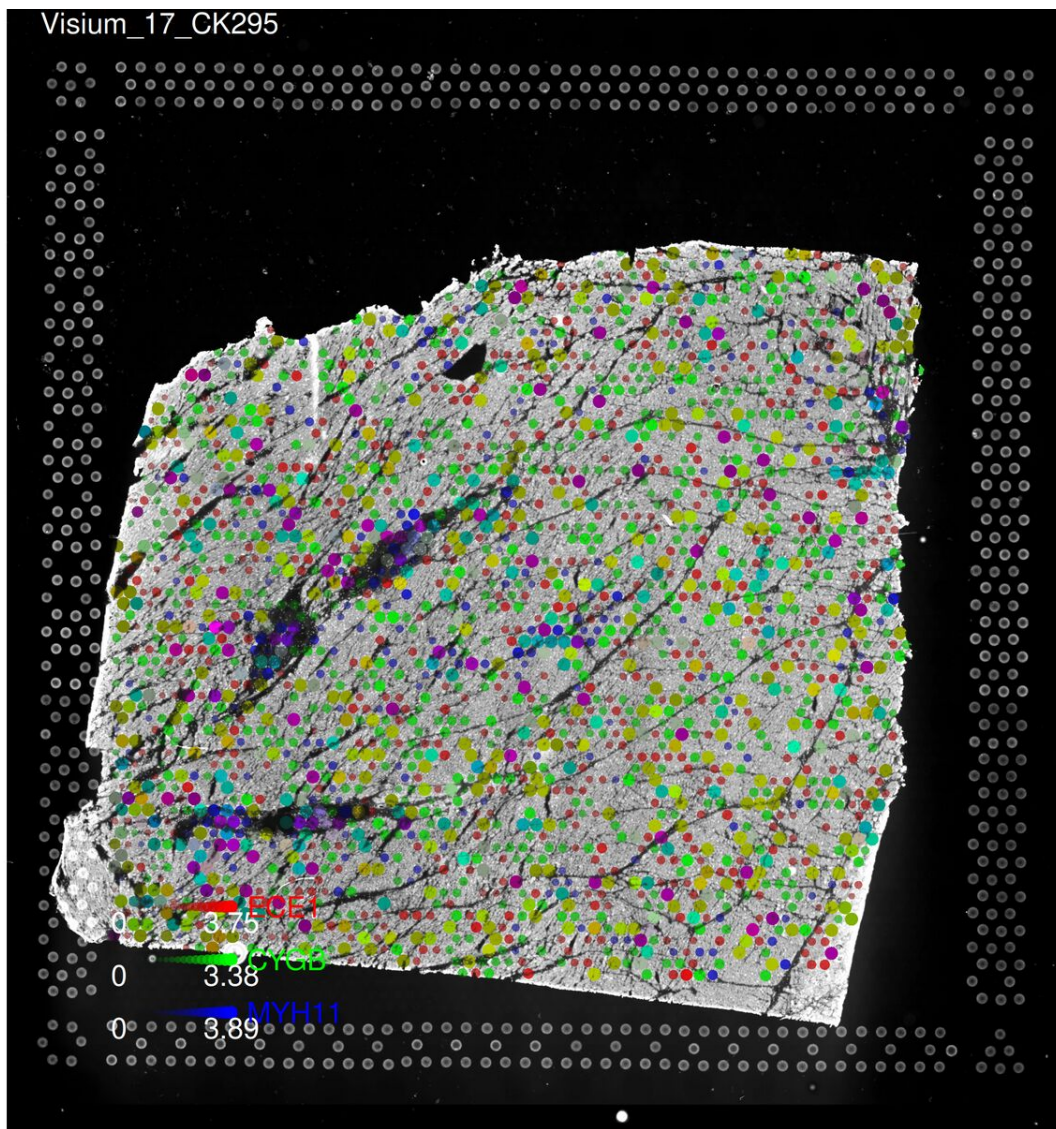
AKK002_Infarct_MI_acute_V19T05025_rep2

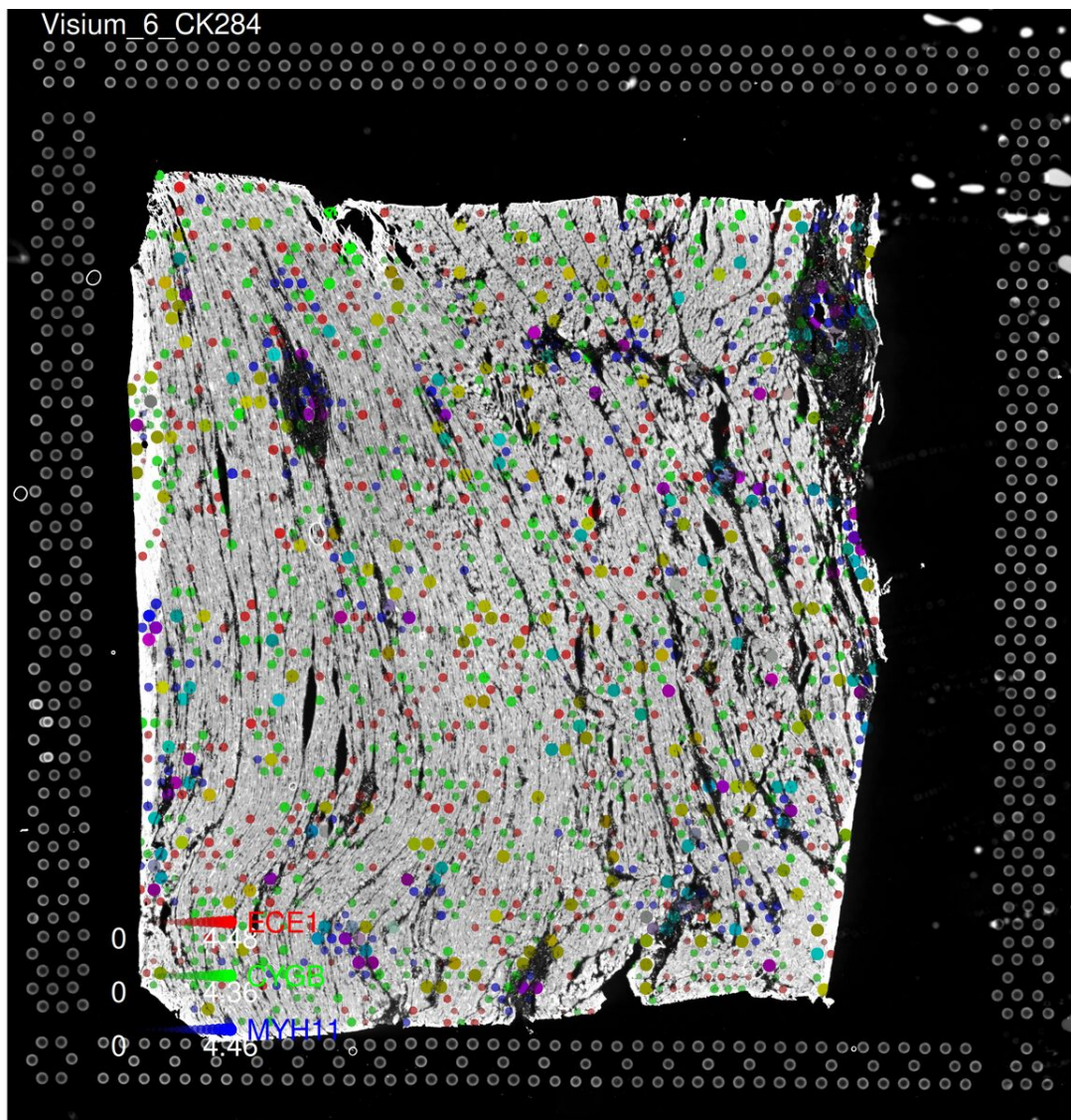


AKK002_Normal_MI_acute_V19T05026_rep1









1.3 Communication Scoring

```
[9]: ## We can pick up all edn gene expression across the plots.
# gc()

# genes_EDN.exp_list = lapply(
#   datasets,
#   function(i) {
#     x = anndata::read_h5ad(i)

#     if("n_counts" %in% colnames(x$obs)){
#       spot_depths = x$obs[, "n_counts"]
```



```

#           } else {
#               spot_depths = x$obs[, "nCount_Spatial"]
#           }
#               good_spots = which(spot_depths > 600)

#               gene.idx = match(c("EDN1", "EDN2", "EDN3", "EDNRA",
↪ "EDNRB", "ECE1", "ECE2"), x$var$feature_name)

#               genes_EDN_matrix = as.matrix(x[good_spots, gene.idx]$X)
#               colnames(genes_EDN_matrix) = c("EDN1", "EDN2", "EDN3", "EDNRA",
↪ "EDNRB", "ECE1", "ECE2")
#               x_metadata = x$obs[good_spots,]
#               return(list(genes_EDN_matrix, x_metadata))
#           }
#       )
# # saveRDS(genes_EDN.exp_list, "genes_EDN.exp_list.rds")

```

```
[10]: genes_EDN.exp_list = readRDS("genes_EDN.exp_list.rds")
```

1.3.1 Generating our comparison data - we focus on ECE1 and the EDNRA / EDNRB, due to the results in suspension

```

[11]: # Looking at the co-expression of "ECE1" and the receptors...
ligand = c("ECE1")
receptor = c("EDNRA")

fraction_of_positive_EDNRA = sapply(1:13, function(i) {
  a = rowSums(genes_EDN.exp_list[[i]][[1]][,ligand,drop=F]) > 0 &
↪ rowSums(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F]) > 0
  # sum(a > 0) / length(a)
  sum(a > 0) / length(a) * 100
})

mean_coexp_EDNRA = sapply(1:13, function(i) {
  l_exp = t(genes_EDN.exp_list[[i]][[1]][,ligand,drop=F]) / colMaxs(genes_EDN.
↪ exp_list[[i]][[1]][,ligand,drop=F])
  r_exp = t(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F]) /
↪ colMaxs(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F])
  a = colMeans(rbind(l_exp, r_exp))
  a = a[a > 0]
  # sum(a > 0) / length(a)
  mean(a)
})

# Looking at the co-expression of "ECE1" and the receptors...

```

```

ligand = c("ECE1")
receptor = c("EDNRB")

fraction_of_positive_EDNRB = sapply(1:13, function(i) {
  a = rowSums(genes_EDN.exp_list[[i]][[1]][,ligand,drop=F]) > 0 &
  ↪rowSums(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F]) > 0
  # sum(a > 0) / length(a)
  sum(a > 0) / length(a) * 100
})

mean_coexp_EDNRB = sapply(1:13, function(i) {
  l_exp = t(genes_EDN.exp_list[[i]][[1]][,ligand,drop=F]) / colMaxs(genes_EDN.
  ↪exp_list[[i]][[1]][,ligand,drop=F])
  r_exp = t(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F]) /
  ↪colMaxs(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F])
  a = colMeans(rbind(l_exp, r_exp))
  a = a[a > 0]
  # sum(a > 0) / length(a)
  mean(a)
})

# Looking at the co-expression of "ECE1" and the receptors...
ligand = c("ECE1")
receptor = c("EDNRA", "EDNRB")

fraction_of_positive = sapply(1:13, function(i) {
  a = rowSums(genes_EDN.exp_list[[i]][[1]][,ligand,drop=F]) > 0 &
  ↪rowSums(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F]) > 0
  # sum(a > 0) / length(a)
  sum(a > 0) / length(a) * 100
})

mean_coexp = sapply(1:13, function(i) {
  l_exp = t(genes_EDN.exp_list[[i]][[1]][,ligand,drop=F]) / colMaxs(genes_EDN.
  ↪exp_list[[i]][[1]][,ligand,drop=F])
  r_exp = t(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F]) /
  ↪colMaxs(genes_EDN.exp_list[[i]][[1]][,receptor,drop=F])
  a = colMeans(rbind(l_exp, r_exp))
  a = a[a > 0]
  # sum(a > 0) / length(a)
  mean(a)
})

```



```
[12]: # Using a method including overall communication between neighbouring spots
library("spdep")
```

Loading required package: spData

To access larger datasets in this package, install the spDataLarge package with: ``install.packages('spDataLarge', repos='https://nowosad.github.io/drat/', type='source')``

Loading required package: sf

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE

```
[13]: # Calculate Communication(s) for ECE1 / EDNRA or EDNRB
global_Ib_EDNRA = numeric()
global_Ib_EDNRB = numeric()
for(i in 1:13){
  samples2plot = sample_id_list[i]
  x.coords = image_coordinate_list[[samples2plot]][,1]
  y.coords = image_coordinate_list[[samples2plot]][,2]

  # pick up the counts and coordinates
  counts = t(genes_EDN.exp_list[[i]][[1]])
  coords = cbind(x.coords, y.coords)

  # calculate the nearest neighbours
  nb <- knn2nb(knearneigh(coords, k = 6)) # Forces the 6 nearest neighbors
  listw <- nb2listw(nb, style = "W")

  nb_with_self <- include.self(nb) # add the "self" spot, because we still
  ↪ expect it to be important
  listw_self <- nb2listw(nb_with_self, style = "W")

  # Ligand
  ligand = c("ECE1")
  l_exp <- as.numeric(colSums(counts[ligand,,drop=F]))

  # Calculate first for EDNRA
  receptor = c("EDNRA")
  r_exp <- as.numeric(colSums(counts[receptor,,drop=F]))

  # Calculate Bivariate Local Moran's
  res <- localmoran_bv(x = l_exp,
                      y = r_exp,
                      listw = listw_self,
                      nsim=100)
```

```

# Calculate the communication score using spatial lag model
r_lag <- lag.listw(listw_self, r_exp)

# add values to metadata
genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_stat = res[,"Ibvi"]
genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p = res[,"Pr(z != E(Ibvi))"]
genes_EDN.exp_list[[i]][[2]]$EDNRA_comm_score = l_exp * r_lag
genes_EDN.exp_list[[i]][[2]]$EDNRA_receptor_exp = r_exp
genes_EDN.exp_list[[i]][[2]]$EDNRA_ligand_exp = l_exp
genes_EDN.exp_list[[i]][[2]]$EDNRA_receptor_lag = r_lag

global_Ib_EDNRA = c(global_Ib_EDNRA, moran_bv(l_exp, r_exp,
↪listw=listw_self, nsim=100)$t0)

# Calculate second for EDNRB
receptor = c("EDNRB")
r_exp <- as.numeric(colSums(counts[receptor,,drop=F]))

# Calculate Bivariate Local Moran's
res <- localmoran_bv(x = l_exp,
                     y = r_exp,
                     listw = listw_self,
                     nsim=100)

# Calculate the communication score using spatial lag model
r_lag <- lag.listw(listw_self, r_exp)

genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_stat = res[,"Ibvi"]
genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_p = res[,"Pr(z != E(Ibvi))"]
genes_EDN.exp_list[[i]][[2]]$EDNRB_comm_score = l_exp * r_lag
genes_EDN.exp_list[[i]][[2]]$EDNRB_receptor_exp = r_exp
genes_EDN.exp_list[[i]][[2]]$EDNRB_ligand_exp = l_exp
genes_EDN.exp_list[[i]][[2]]$EDNRB_receptor_lag = r_lag

global_Ib_EDNRB = c(global_Ib_EDNRB, moran_bv(l_exp, r_exp,
↪listw=listw_self, nsim=100)$t0)

# combine the scores
genes_EDN.exp_list[[i]][[2]]$comm_score = genes_EDN.
↪exp_list[[i]][[2]]$EDNRA_comm_score + genes_EDN.
↪exp_list[[i]][[2]]$EDNRB_comm_score
}

```

Warning message in knn2nb(knearneigh(coords, k = 6)):
 “neighbour object has 2 sub-graphs”


```

[14]: alpha = 0.05

EDN_comms_df = data.frame(
  "sampleID" = sample_id_list,
  "x" = rep(c("Healthy", "MI_IZ", "MI_RZ"), c(4,5,4)),
  "comm_score_EDNRA" = sapply(1:13, function(i) {
    a = genes_EDN.exp_list[[i]][[2]]$EDNRA_comm_score
    # sum(a > 0) / length(a)
    mean(a, na.rm=T)
  }),
  "comm_score_EDNRB" = sapply(1:13, function(i) {
    a = genes_EDN.exp_list[[i]][[2]]$EDNRB_comm_score
    # sum(a > 0) / length(a)
    mean(a, na.rm=T)
  }),
  "comm_score" = sapply(1:13, function(i) {
    a = genes_EDN.exp_list[[i]][[2]]$comm_score
    # sum(a > 0) / length(a)
    mean(a, na.rm=T)
  }),
  "comm_score_EDNRA_significant" = sapply(1:13, function(i) {
    sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha |
    ↪genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_p < alpha
    a = genes_EDN.exp_list[[i]][[2]]$EDNRA_comm_score
    # sum(a > 0) / length(a)
    mean(a[sig.points], na.rm=T)
  }),
  "comm_score_EDNRB_significant" = sapply(1:13, function(i) {
    sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha |
    ↪genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_p < alpha
    a = genes_EDN.exp_list[[i]][[2]]$EDNRB_comm_score
    # sum(a > 0) / length(a)
    mean(a[sig.points], na.rm=T)
  }),
  "comm_score_significant" = sapply(1:13, function(i) {
    sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha |
    ↪genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_p < alpha
    a = genes_EDN.exp_list[[i]][[2]]$comm_score
    # sum(a > 0) / length(a)
    mean(a[sig.points], na.rm=T)
  })
),

```

```

"Ibvi_global_EDNRA" = global_Ib_EDNRA,
"Ibvi_global_EDNRB" = global_Ib_EDNRB,
"fraction_of_doublePositive_EDNRA" = fraction_of_positive_EDNRA,
"fraction_of_doublePositive_EDNRB" = fraction_of_positive_EDNRB,
"fraction_of_doublePositive_both" = fraction_of_positive,
"coexpression_value_EDNRA" = mean_coexp_EDNRA,
"coexpression_value_EDNRB" = mean_coexp_EDNRB,
"coexpression_value" = mean_coexp,
"fraction_of_significantSpots_EDNRA" = sapply(1:13, function(i) {
  sum(genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha) /
  length(genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p)
}),
"fraction_of_significantSpots_EDNRB" = sapply(1:13, function(i) {
  sum(genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_p < alpha) /
  length(genes_EDN.exp_list[[i]][[2]]$EDNRB_moran_p)
}),
"fraction_of_significantSpots" = sapply(1:13, function(i) {
  sum(genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
  exp_list[[i]][[2]]$EDNRB_moran_p < alpha) / length(genes_EDN.
  exp_list[[i]][[2]]$EDNRA_moran_p)
})
)[c(1,2,3,4,5,6,7,9,10,11,12,13),]
for(column in 3:ncol(EDN_comms_df)){EDN_comms_df[,column] =
  round(EDN_comms_df[,column],4)}

```

```

[15]: # write.table(EDN_comms_df, file="EDN_comms_df.csv", sep=",", quote=F, col.
  names=T, row.names=F)
EDN_comms_df = read.csv("EDN_comms_df.csv")
EDN_comms_df

```

A data.frame: 12 × 19

sampleID <chr>	x <chr>	comm_score_EDNRA <dbl>	con <d
AKK006_II_healthycontrol_V19T05022_rep2	Healthy	0.0419	0.0
Visium_1_CK279	Healthy	0.0273	0.0
Visium_3_CK281	Healthy	0.0609	0.0
Visium_4_CK282	Healthy	0.0282	0.1
Visium_9_CK287	MI_IZ	0.1102	0.1
Visium_7_CK285	MI_IZ	0.0744	0.2
Visium_20_CK298	MI_IZ	0.1328	0.1
AKK002_Infarct_MI_acute_V19T05025_rep2	MI_IZ	0.0600	0.2
AKK002_Normal_MI_acute_V19T05026_rep1	MI_RZ	0.0402	0.1
Visium_2_CK280	MI_RZ	0.0251	0.0
Visium_17_CK295	MI_RZ	0.0678	0.0
Visium_6_CK284	MI_RZ	0.0462	0.1

1.3.2 Statistical Testing of Communication scores

ANOVA / Parametric between MI, RZ and Healthy

```
[16]: comms_anova = aov(comm_score~x, data=EDN_comms_df)
summary(comms_anova)
```

```
          Df Sum Sq Mean Sq F value Pr(>F)
x           2 0.07248  0.03624    15.8 0.00114 **
Residuals    9 0.02064  0.00229
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
[17]: TukeyHSD(comms_anova)
```

```
Tukey multiple comparisons of means
95% family-wise confidence level

Fit: aov(formula = comm_score ~ x, data = EDN_comms_df)

$x
          diff          lwr          upr      p adj
MI_IZ-Healthy 0.17995 0.08541015 0.27448985 0.0012596
MI_RZ-Healthy 0.03620 -0.05833985 0.13073985 0.5552507
MI_RZ-MI_IZ   -0.14375 -0.23828985 -0.04921015 0.0054913
```

```
[18]: comms_anova = aov(comm_score_significant~x, data=EDN_comms_df)
summary(comms_anova)
```

```
          Df Sum Sq Mean Sq F value Pr(>F)
x           2 0.2235  0.11177    8.435 0.00864 **
Residuals    9 0.1192  0.01325
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
[19]: TukeyHSD(comms_anova)
```

```
Tukey multiple comparisons of means
95% family-wise confidence level

Fit: aov(formula = comm_score_significant ~ x, data = EDN_comms_df)

$x
          diff          lwr          upr      p adj
MI_IZ-Healthy 0.306800 0.07954804 0.53405196 0.0110882
MI_RZ-Healthy 0.038375 -0.18887696 0.26562696 0.8861689
MI_RZ-MI_IZ   -0.268425 -0.49567696 -0.04117304 0.0227642
```

Non-Parametric Test - Kruskal Wallis If we aren't convinced about the parametric assumptions the non-parametric test is also significant for MI vs Healthy

```
[20]: comms_kruskal = kruskal.test(comm_score~x, data=EDN_comms_df)
      comms_kruskal
```

Kruskal-Wallis rank sum test

data: comm_score by x

Kruskal-Wallis chi-squared = 7.7308, df = 2, p-value = 0.02095

```
[21]: comms_kruskal = conover.test(EDN_comms_df$comm_score, EDN_comms_df$x,
      ↪method="bonferroni")
```

Kruskal-Wallis rank sum test

data: x and group

Kruskal-Wallis chi-squared = 7.7308, df = 2, p-value = 0.02

Comparison of x by group
(Bonferroni)

Col Mean-		
Row Mean	Healthy	MI_IZ
-----+-----		
MI_IZ	-4.392841	
	0.0026*	
MI_RZ	-0.976187	3.416654
	0.5317	0.0115*

alpha = 0.05

Reject Ho if p <= alpha/2

```
[22]: comms_kruskal = kruskal.test(comm_score_significant~x, data=EDN_comms_df)
      comms_kruskal
```

Kruskal-Wallis rank sum test

data: comm_score_significant by x

Kruskal-Wallis chi-squared = 6.9615, df = 2, p-value = 0.03078

```
[23]: comms_kruskal = conover.test(EDN_comms_df$comm_score_significant,
      ↪EDN_comms_df$x, method="bonferroni")
```

Kruskal-Wallis rank sum test

data: x and group

Kruskal-Wallis chi-squared = 6.9615, df = 2, p-value = 0.03

Comparison of x by group
(Bonferroni)

Col Mean-		
Row Mean	Healthy	MI_IZ
MI_IZ	-3.806010	
	0.0063*	
MI_RZ	-1.024695	2.781315
	0.4984	0.0320

alpha = 0.05

Reject Ho if p <= alpha/2

plotting the results of the communication scores

[150]:

```
options(repr.plot.width = 10)
options(repr.plot.height = 10)

par(mar=c(5,6,1,6), cex=2)
plot(c(0.5,3.5), c(0,4), pch=NA, xlab="", ylab="spot communication score",
     xaxt="n")
axis(1, at=1:3, labels=c("H", "MI_RZ", "MI_IZ"), las=2)

condition_cols = rep(c("red","black","grey"), each=4)

rep(c(1,3,2), each=4)
plot.positions = rep(c(1,3,2), each=4)
i_list = c(1,2,3,4,5,6,7,9,10,11,12,13)

score2compare = "comm_score"
alpha=1

for(i in i_list){
  n = i_list[i]
  sig.points = genes_EDN.exp_list[[n]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
  exp_list[[n]][[2]]$EDNRB_moran_p < alpha
  points(
    plot.positions[i]+rnorm(length(genes_EDN.
  exp_list[[n]][[2]][,score2compare] [sig.points]),0,0.1),
    genes_EDN.exp_list[[n]][[2]][,score2compare] [sig.points],
    pch=".",
```

```

        col=c(Discrete12, "grey")[i],
        cex=4
    )
}
for(i in 1:12){
    points(rnorm(1, plot.positions[i], 0.1),EDN_comms_df[i,"comm_score"],
    pch=21, cex=1.2, bg=condition_cols[i], col="black")
}

pdf("figures/panels/results/Kuppe2022/communicationECE1_scatter.pdf", height=5,
    width=3)

plot(c(0.5,3.5), c(0,4), pch=NA, xlab="", ylab="spot communication score (p < 0.
    05 spots only)", xaxt="n")
axis(1, at=1:3, labels=c("H", "MI_RZ", "MI_IZ"), las=2)

score2compare = "comm_score"
alpha=0.05
for(i in 1:12){
    n = i_list[i]
    sig.points = genes_EDN.exp_list[[n]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
    exp_list[[n]][[2]]$EDNRB_moran_p < alpha
    points(
        plot.positions[i]+rnorm(length(genes_EDN.
    exp_list[[n]][[2]][,score2compare] [sig.points]),0,0.1),
        genes_EDN.exp_list[[n]][[2]][,score2compare] [sig.points],
        pch=".",
        col=c(Discrete12, "grey")[i],
        cex=4
    )
}
for(i in 1:12){
    points(rnorm(1, plot.positions[i], 0.
    05),EDN_comms_df[i,"comm_score_significant"], pch=21, cex=1.2,
    bg=condition_cols[i], col="black")
}
dev.off()
plot(c(0.5,3.5), c(0,4), pch=NA, xlab="", ylab="spot communication score (p < 0.
    05 spots only)", xaxt="n")
axis(1, at=1:3, labels=c("H", "MI_RZ", "MI_IZ"), las=2)

score2compare = "comm_score"
alpha=0.05
for(i in 1:12){
    n = i_list[i]

```

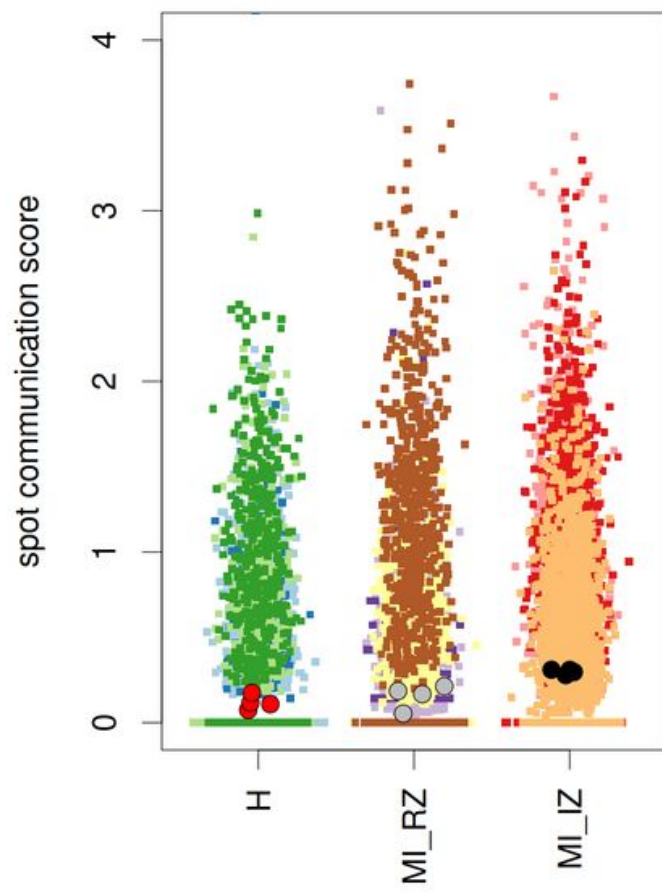
```

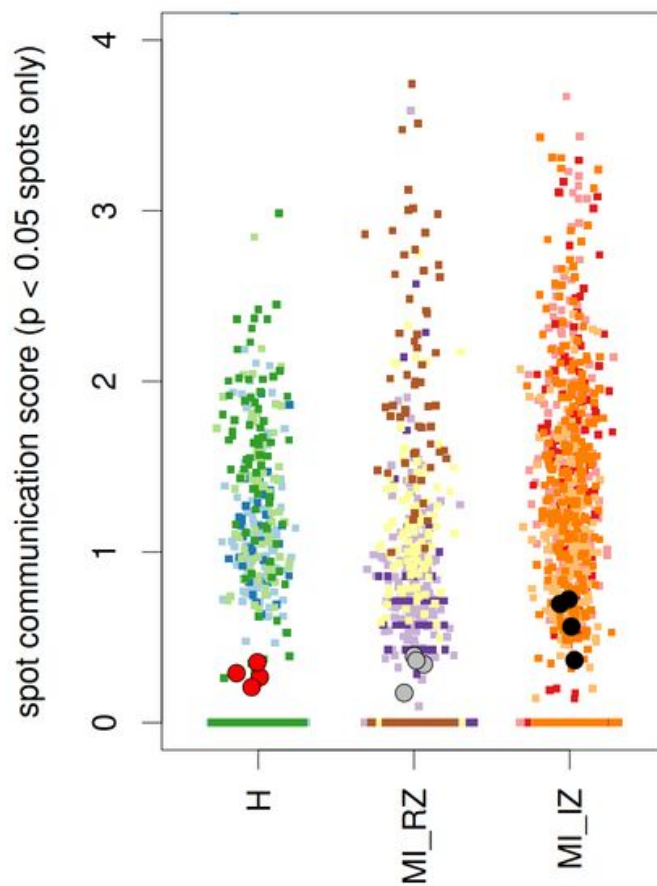
    sig.points = genes_EDN.exp_list[[n]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
    ↪exp_list[[n]][[2]]$EDNRB_moran_p < alpha
    points(
      plot.positions[i]+rnorm(length(genes_EDN.
    ↪exp_list[[n]][[2]][,score2compare] [sig.points]),0,0.1),
      genes_EDN.exp_list[[n]][[2]][,score2compare] [sig.points],
      pch=".",
      col=c(Discrete12, "grey")[i],
      cex=4
    )
  }
  for(i in 1:12){
    points(rnorm(1, plot.positions[i], 0.
    ↪05),EDN_comms_df[i,"comm_score_significant"], pch=21, cex=1.2,
    ↪bg=condition_cols[i], col="black")
  }

```

1. 1 2. 1 3. 1 4. 1 5. 3 6. 3 7. 3 8. 3 9. 2 10. 2 11. 2 12. 2

agg_record_555613782: 2





```
[160]: ## Looking at ECE1 expression vs communication score.
plot(c(0,3),c(0,1.5),pch=NA, xlab="ECE1 expression", ylab="EDNRA lag_
  ↪(surrounding 6 points)")

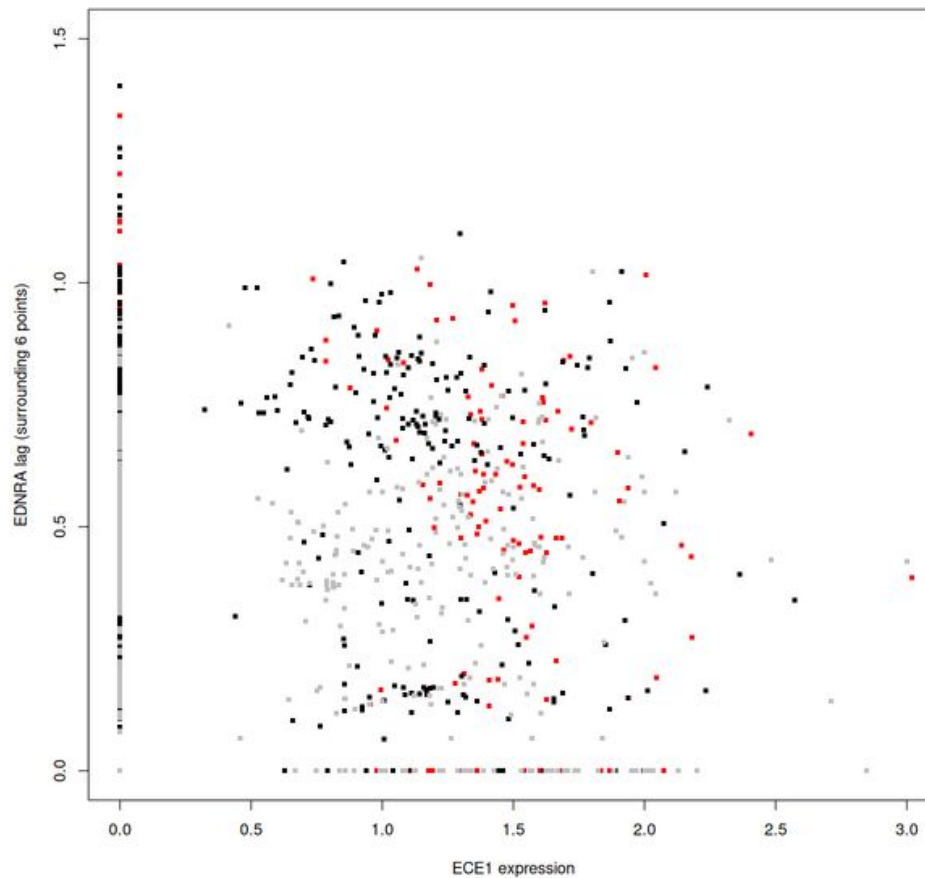
alpha=0.01
for(i in c(1,2,3,4,5,6,7,9,10,11,12,13)){
  sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
  ↪exp_list[[i]][[2]]$EDNRB_moran_p < alpha
  points(
    genes_EDN.exp_list[[i]][[1]][sig.points,"ECE1"],
    genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRA_receptor_lag"],
    pch=".",
    col=rep(c("red", "black", "grey"), c(4,4,4))[i],
    cex=5
  )
}
```

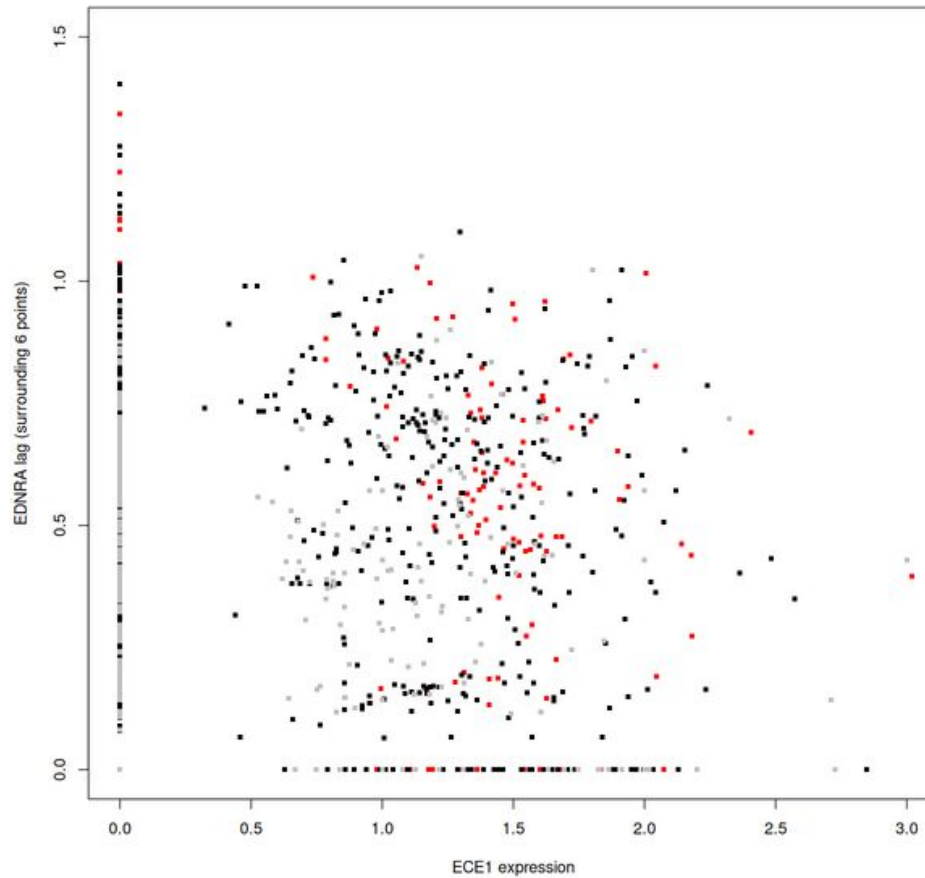
```

plot(c(0,3),c(0,1.5),pch=NA, xlab="ECE1 expression", ylab="EDNRA lag (surrounding 6 points)")

for(i in c(1,2,3,4,5,6,7,9,10,11,12,13)){
  sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
  exp_list[[i]][[2]]$EDNRB_moran_p < alpha
  points(
    genes_EDN.exp_list[[i]][[1]][sig.points,"ECE1"],
    genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRA_receptor_lag"],
    pch=".",
    col=rep(c("red", "black", "grey"), c(4,5,4))[i],
    cex=5
  )
}

```





```
[24]: # readying the plots - maximum scores across all data
max_scores = sapply(1:13, function(i){

  alpha= 0.05
  sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
  ↪exp_list[[i]][[2]]$EDNRB_moran_p < alpha

  score1 = genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRA_comm_score"]
  score2 = genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRB_comm_score"]

  red_range = max(score1)
  blue_range = max(score2)
  alpha_range = max(score1 + score2)
  return(c(red_range, blue_range, alpha_range))
})
```

```
[25]: max_scores
```

```

          1.280383  1.205611  2.039101  1.122437  1.954989  1.588962  1.760589  1.6290
A matrix: 3 × 13 of type dbl 2.106463  2.980344  1.595182  2.984197  2.770827  3.171274  2.056256  2.4906
          2.187471  4.174165  2.844744  2.984197  3.669418  3.295949  2.648087  3.7817

```

```
[26]: max_vals = rowMaxs(max_scores)
max_vals
```

```
1. 2.03910077263225 2. 3.47519045589865 3. 4.17416458972316
```

1.3.3 Communication Plots

```
[27]: # plot all communication plots, do not run again!
options(repr.plot.width = 20)
options(repr.plot.height = 20)

maxp2p = 0.01
p2p_mod = 1.6
for(i in c(1,2,3,4,5,6,7,9,10,11,12,13)){

#for(i in c(1)){
samples2plot = sample_id_list[i]
  pdf(paste("figures/panels/results/Kuppe2022/CommunicationECE1_", gsub("^.",
↪*datasets/|.h5ad", "", datasets[i]), sample_id_list[i], ".pdf", sep=""),
↪height=20, width=20)
  plotVisium1(
    rgb.samples_list = VisiumData1("", sample_id_list =
↪samples2plot),

    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent =
↪sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )

#for(sample_id in sample_id_list){
  x.coords = image_coordinate_list[[samples2plot]][,1]
  y.coords = image_coordinate_list[[samples2plot]][,2]
#}

alpha= 0.05

```

```

sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
→exp_list[[i]][[2]]$EDNRB_moran_p < alpha

max_pvals = rowMins(as.matrix(genes_EDN.
→exp_list[[i]][[2]][,c("EDNRA_moran_p", "EDNRB_moran_p")]))[sig.points]
max_pvals[max_pvals<maxp2p] = maxp2p

score1 = genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRA_comm_score"]
score2 = genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRB_comm_score"]
score3 = genes_EDN.exp_list[[i]][[1]][,"ECE1"]

score_combined = (score1 + score2)/max_vals[3]
#score_combined[score_combined > 1] = 1
score1 = score1 / max_vals[1]#max(score1)
score2 = score2 / max_vals[2]#max(score2)
score3 = score3 / max(score3)

#score_combined = normalise(score1 + score2)
#points(x.coords, y.coords, col=plotcols(EDN_comms_score), pch=16)
#points(x.coords, y.coords, col=plotcols(score1, col=c("black","red")),
→pch=16)
#points(x.coords, y.coords, col=plotcols(score2, col=c("black","blue")),
→pch=16)
#points(x.coords, y.coords, col=rgb(score1, 0, score2, score_combined),
→pch=16, cex=-log10(max_pvals)*0.5)
#points(x.coords, y.coords, col=rgb(0, score3,0, score3), pch=16, cex=2)
points(
  x.coords[sig.points],
  y.coords[sig.points],
  col=rgb(score1, 0, score2, score_combined),
  pch=16,
  cex=-log10(max_pvals)*p2p_mod
)

legend("topleft",
  legend=seq.log.basic(0.5,maxp2p, length=5),
  pt.cex=-log10(seq.log.basic(0.5,maxp2p, length=5))*p2p_mod,
  pch=16
)
n2p = 10
points(
  seq(50,200, length=n2p),
  rep(10, n2p),
  pch=15,
  col=rgb(seq(0,1, length=n2p),0,0,seq(0,1, length=n2p)),
  cex=2

```



```

)
text(210,10,signif(max_vals[1],2), adj=0)
points(
  seq(50,200, length=n2p),
  rep(30, n2p),
  pch=15,
  col=rgb(0,0,seq(0,1, length=n2p),seq(0,1, length=n2p)),
  cex=2
)
text(210,30,signif(max_vals[2],2), adj=0)
# points(
#   seq(50,200, length=n2p),
#   rep(50, n2p),
#   pch=15,
#   col=rgb(0,0,0,seq(0,1, length=n2p)),
#   cex=2
# )
dev.off()

plotVisium1(
  rgb.samples_list = VisiumData1("", sample_id_list =
↪samples2plot),
  light_or_dark = input$light_or_dark,
  plot_size = input$plot_size,
  mask_overlay = input$mask_overlay,
  min.col = input$min.col,
  opacity_count_dependent =
↪sum(input$opacity_count_dependent),
  opacity = input$opacity,
  dot_or_not = input$dot_or_not,
  sample_id_list = samples2plot
)

#for(sample_id in sample_id_list){
  x.coords = image_coordinate_list[[samples2plot]][,1]
  y.coords = image_coordinate_list[[samples2plot]][,2]
#}

alpha= 0.05
sig.points = genes_EDN.exp_list[[i]][[2]]$EDNRA_moran_p < alpha | genes_EDN.
↪exp_list[[i]][[2]]$EDNRB_moran_p < alpha

max_pvals = rowMins(as.matrix(genes_EDN.
↪exp_list[[i]][[2]][,c("EDNRA_moran_p", "EDNRB_moran_p")]))[sig.points]
max_pvals[max_pvals<maxp2p] = maxp2p

score1 = genes_EDN.exp_list[[i]][[2]][sig.points,"EDNRA_comm_score"]

```

```

score2 = genes_EDN.exp_list[[i]][[2]][sig.points, "EDNRB_comm_score"]
score3 = genes_EDN.exp_list[[i]][[1]][, "ECE1"]

score_combined = (score1 + score2)/max_vals[3]
#score_combined[score_combined > 1] = 1
score1 = score1 / max_vals[1] #max(score1)
score2 = score2 / max_vals[2] #max(score2)
score3 = score3 / max(score3)

#score_combined = normalise(score1 + score2)
#points(x.coords, y.coords, col=plotcols(EDN_comms_score), pch=16)
#points(x.coords, y.coords, col=plotcols(score1, col=c("black", "red")),
→pch=16)
#points(x.coords, y.coords, col=plotcols(score2, col=c("black", "blue")),
→pch=16)
#points(x.coords, y.coords, col=rgb(score1, 0, score2, score_combined),
→pch=16, cex=-log10(max_pvals)*0.5)
#points(x.coords, y.coords, col=rgb(0, score3, 0, score3), pch=16, cex=2)
points(
  x.coords[sig.points],
  y.coords[sig.points],
  col=rgb(score1, 0, score2, score_combined),
  pch=16,
  cex=-log10(max_pvals)*p2p_mod
)

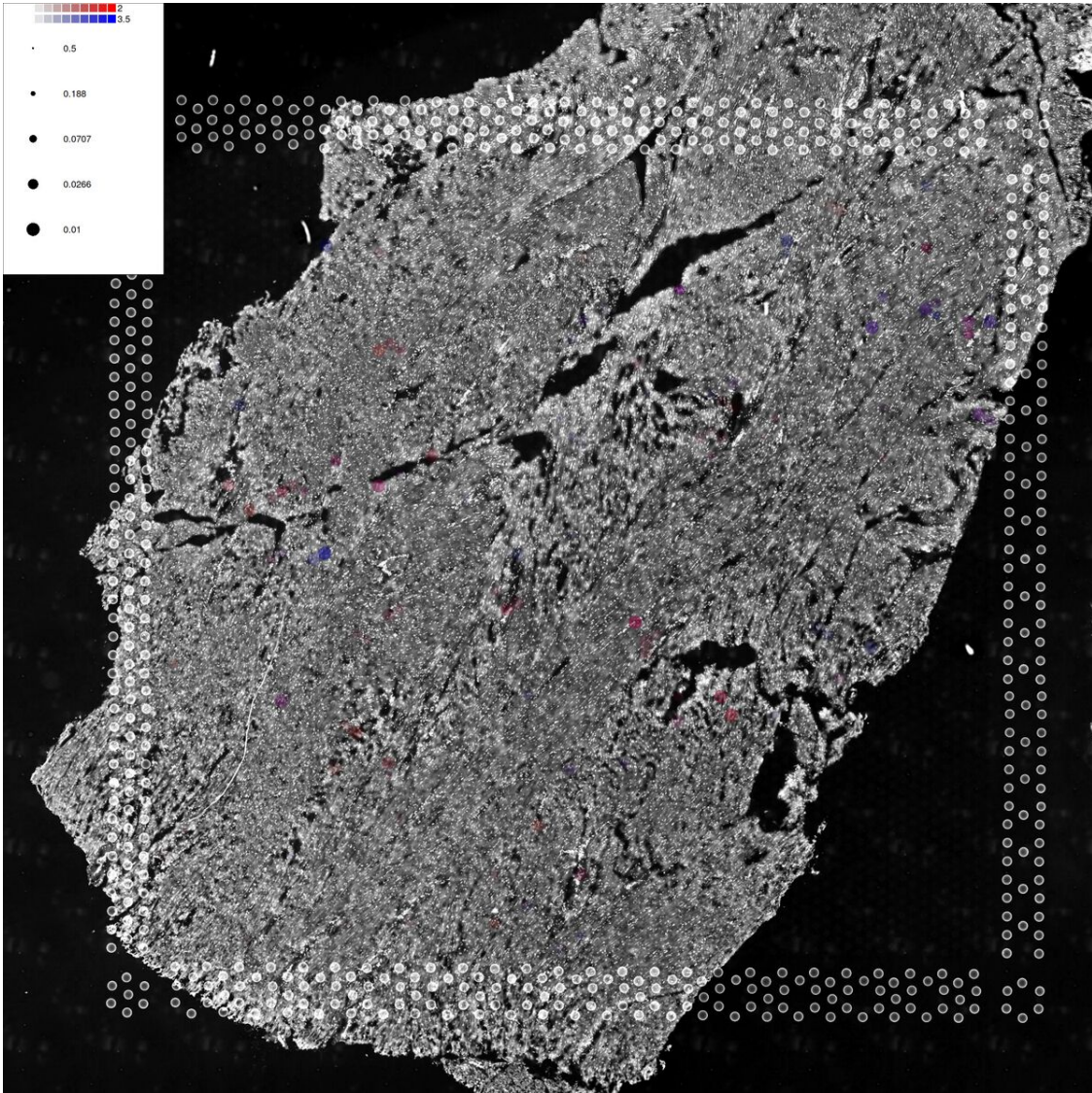
legend("topleft",
  legend=signif(seq.log.basic(0.5, maxp2p, length=5), 3),
  pt.cex=-log10(seq.log.basic(0.5, maxp2p, length=5))*p2p_mod,
  pch=16
)
n2p = 10
points(
  seq(50, 200, length=n2p),
  rep(10, n2p),
  pch=15,
  col=rgb(seq(0, 1, length=n2p), 0, 0, seq(0, 1, length=n2p)),
  cex=2
)
text(210, 10, signif(max_vals[1], 2), adj=0)
points(
  seq(50, 200, length=n2p),
  rep(30, n2p),
  pch=15,
  col=rgb(0, 0, seq(0, 1, length=n2p), seq(0, 1, length=n2p)),
  cex=2
)

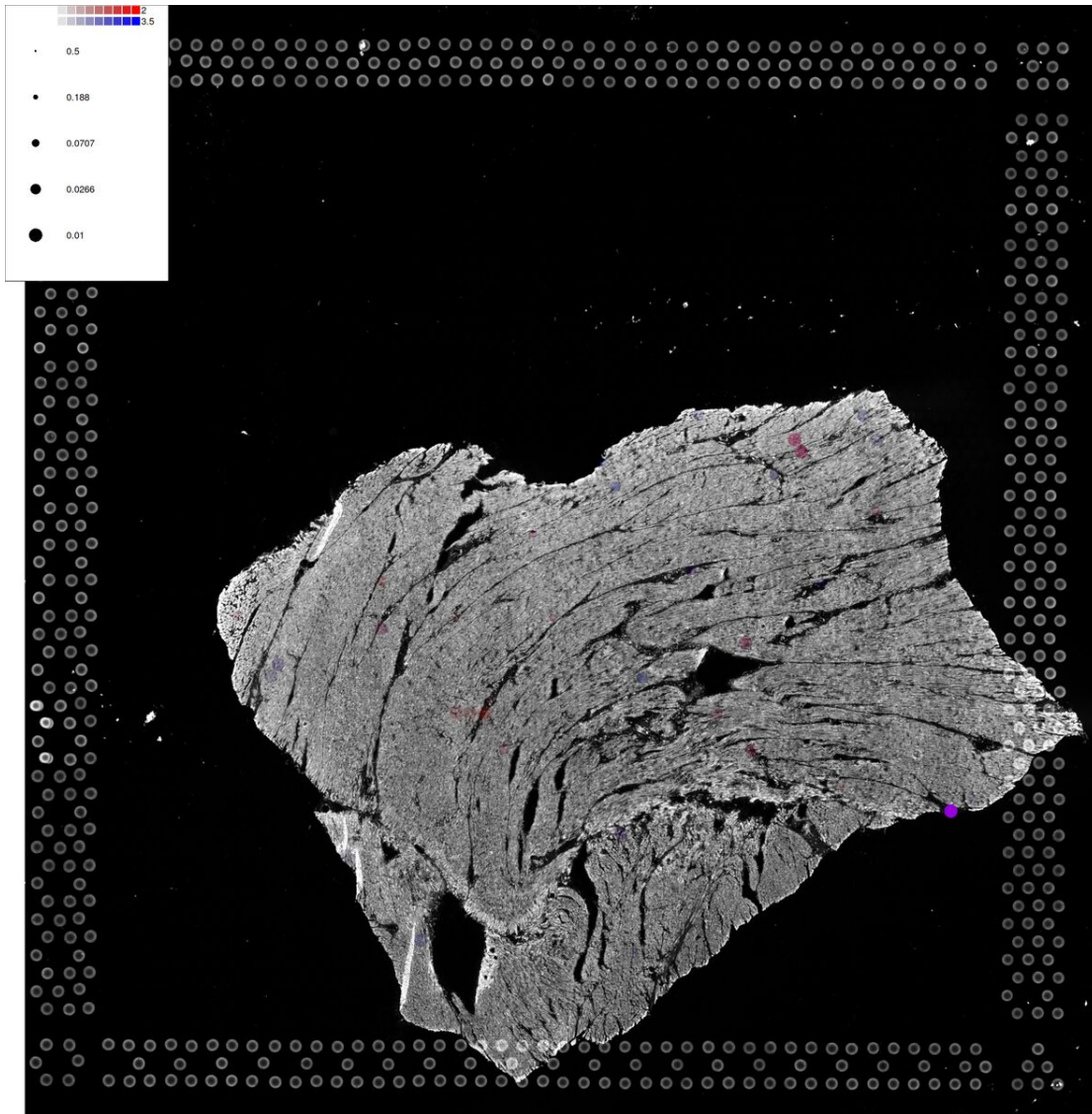
```

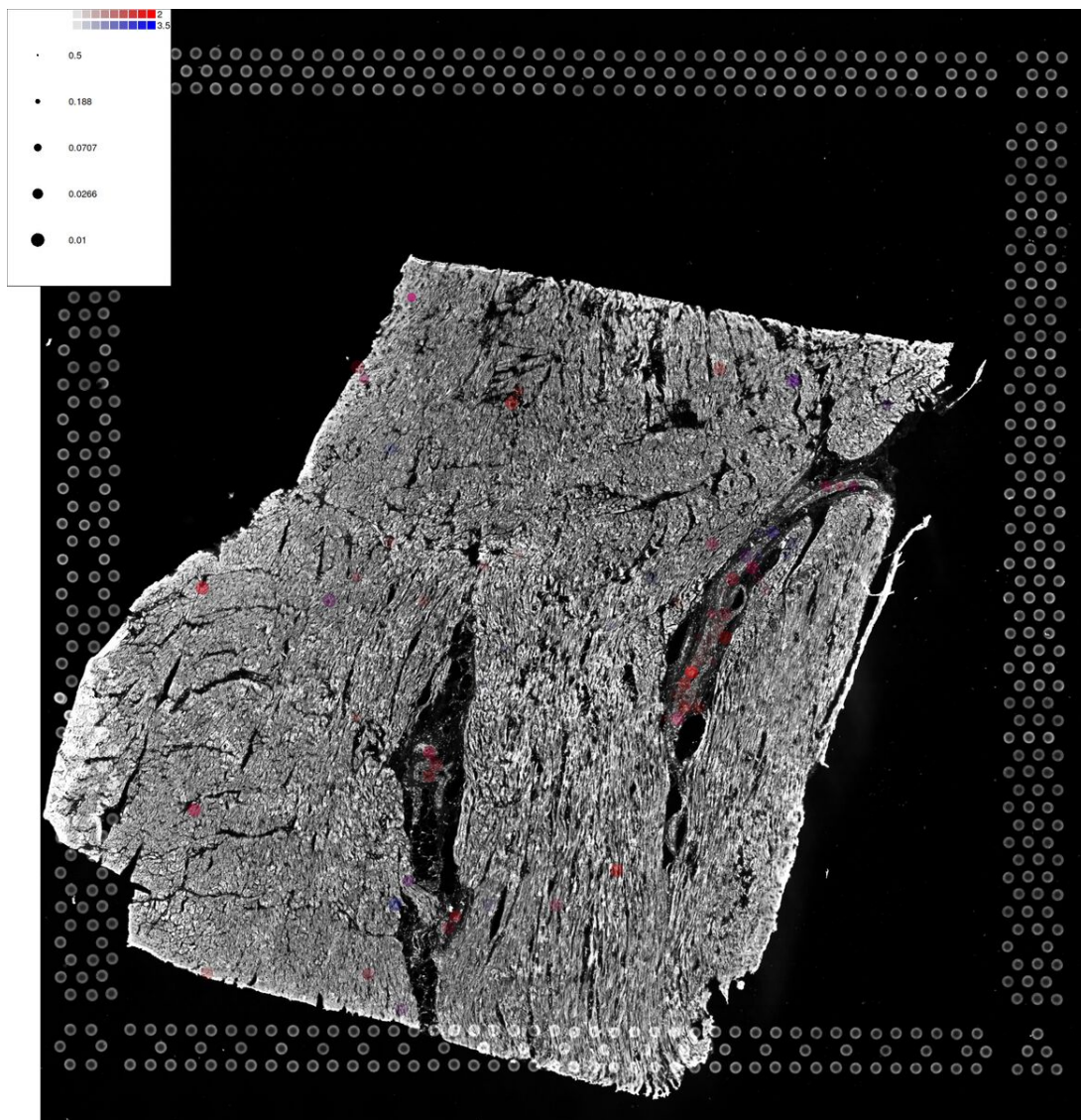
```

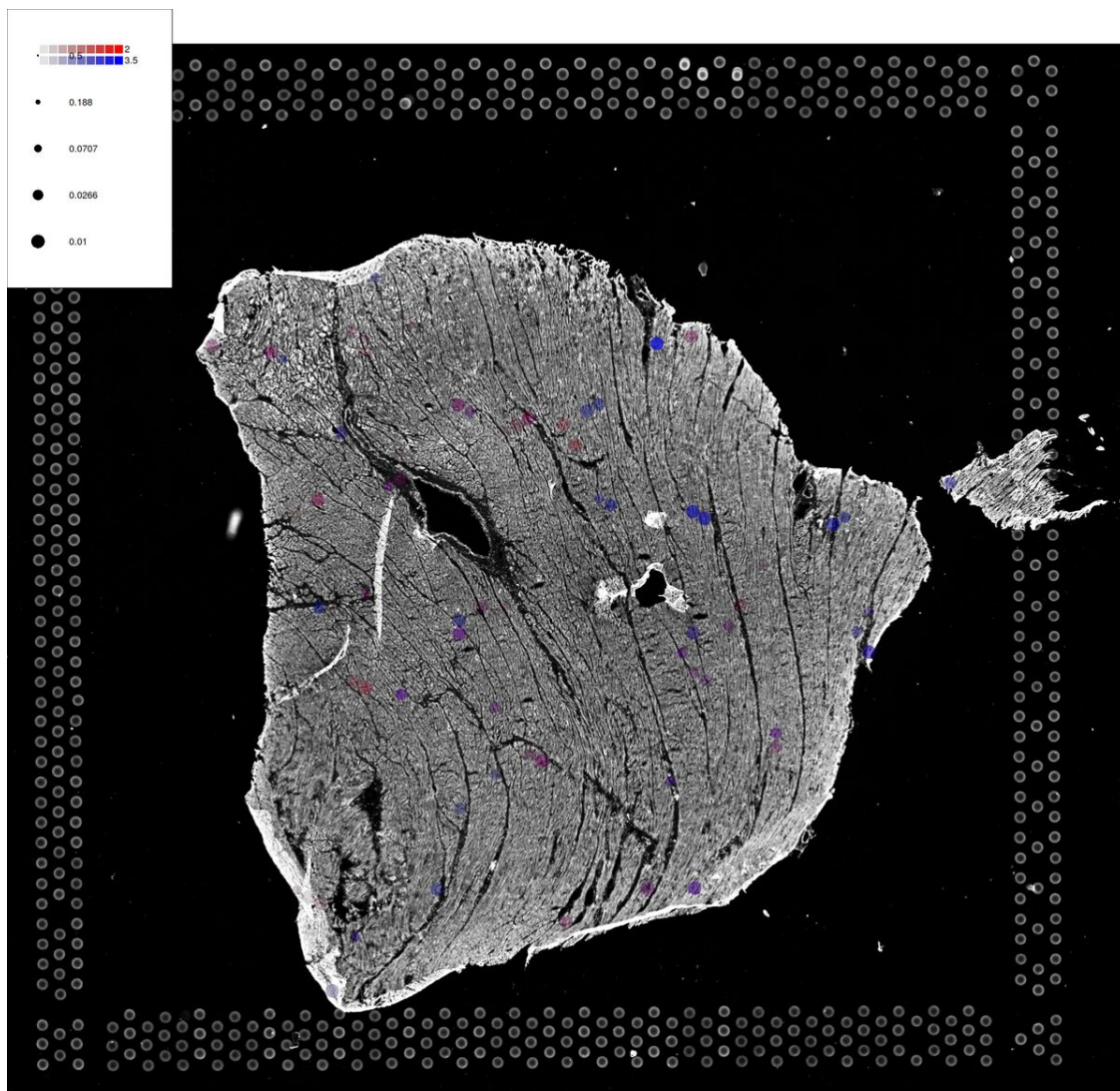
text(210,30,signif(max_vals[2],2), adj=0)
# points(
#   seq(50,200, length=n2p),
#   rep(50, n2p),
#   pch=15,
#   col=rgb(0,0,0,seq(0,1, length=n2p)),
#   cex=2
# )
}

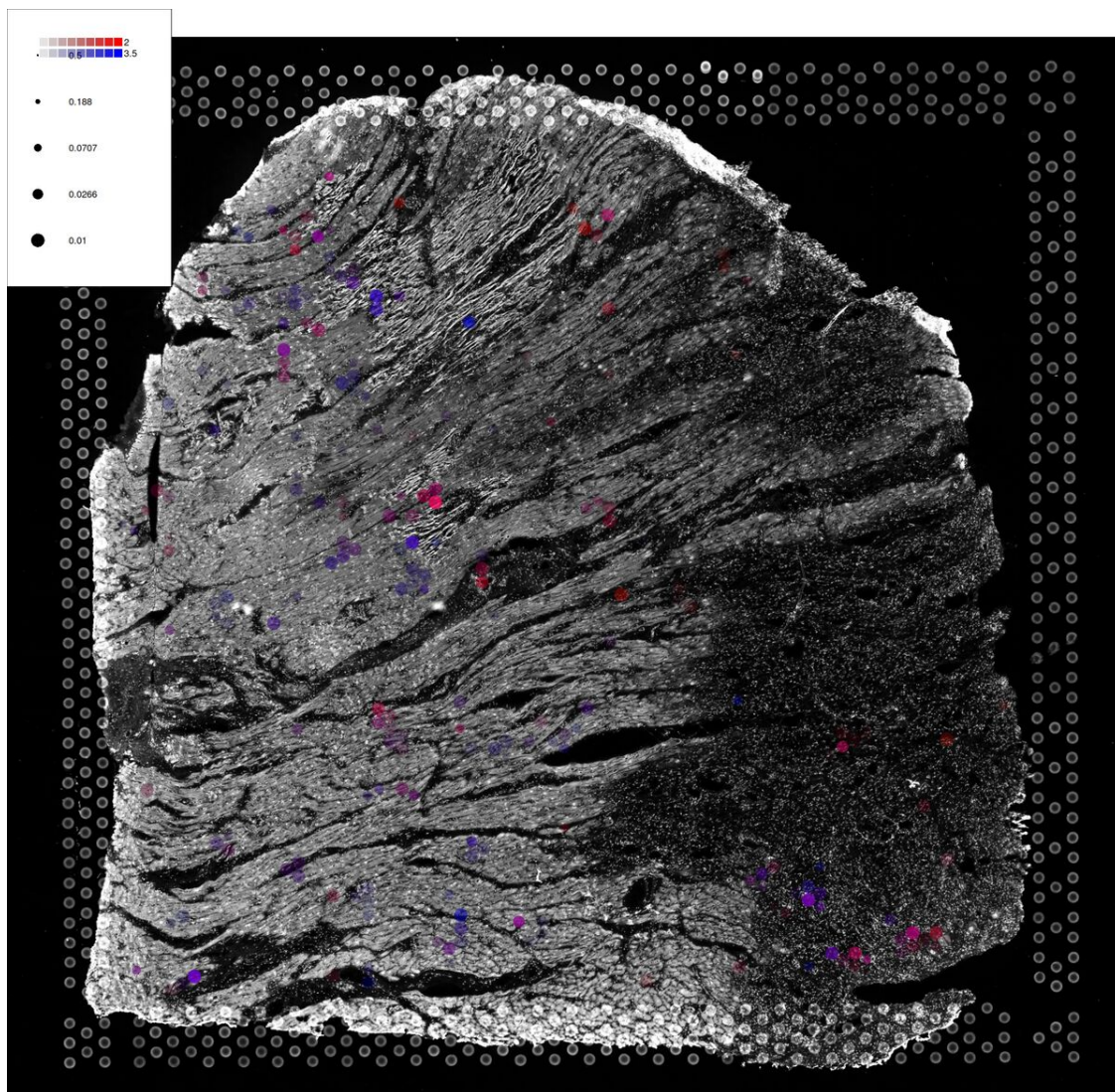
```

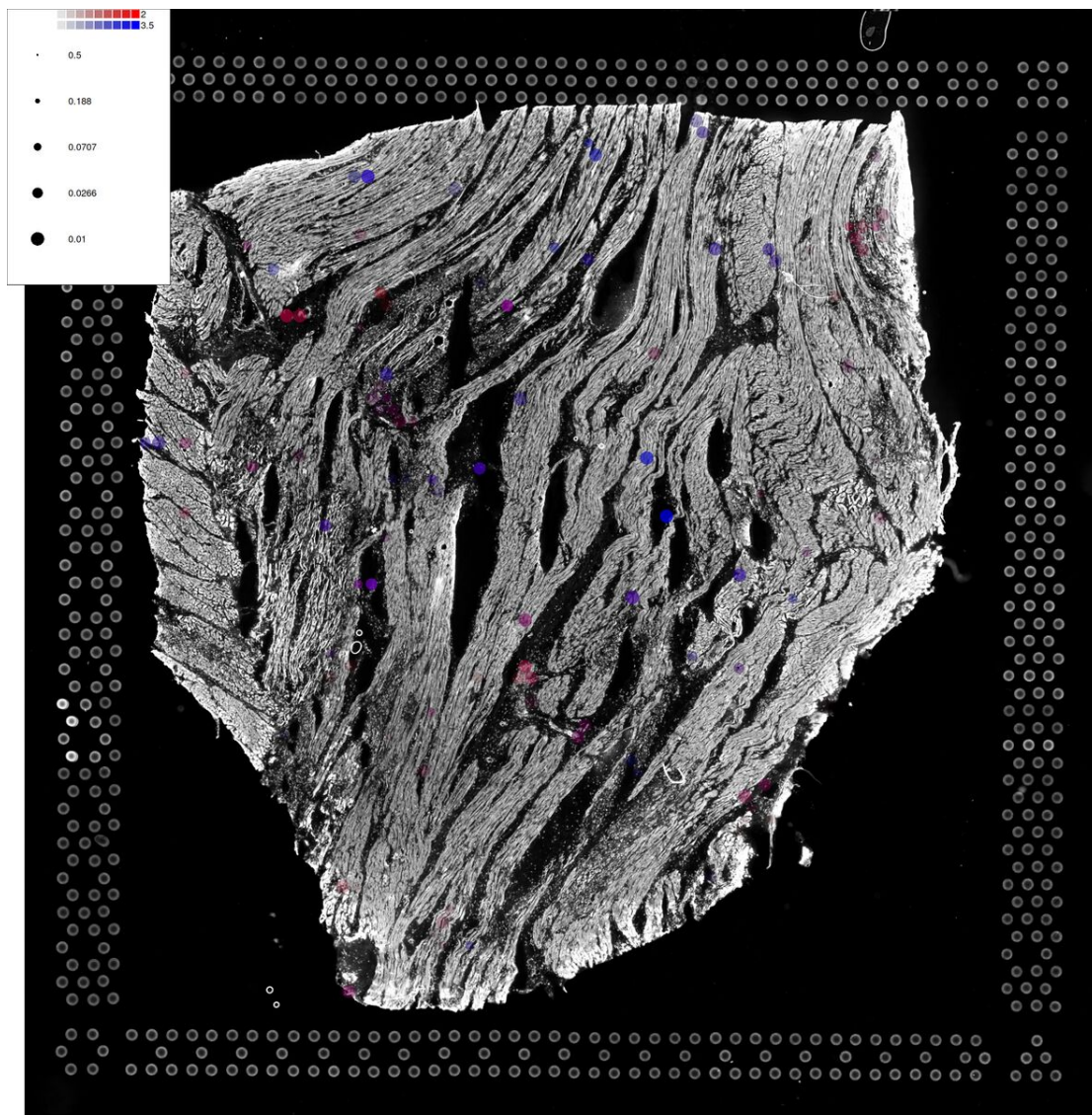


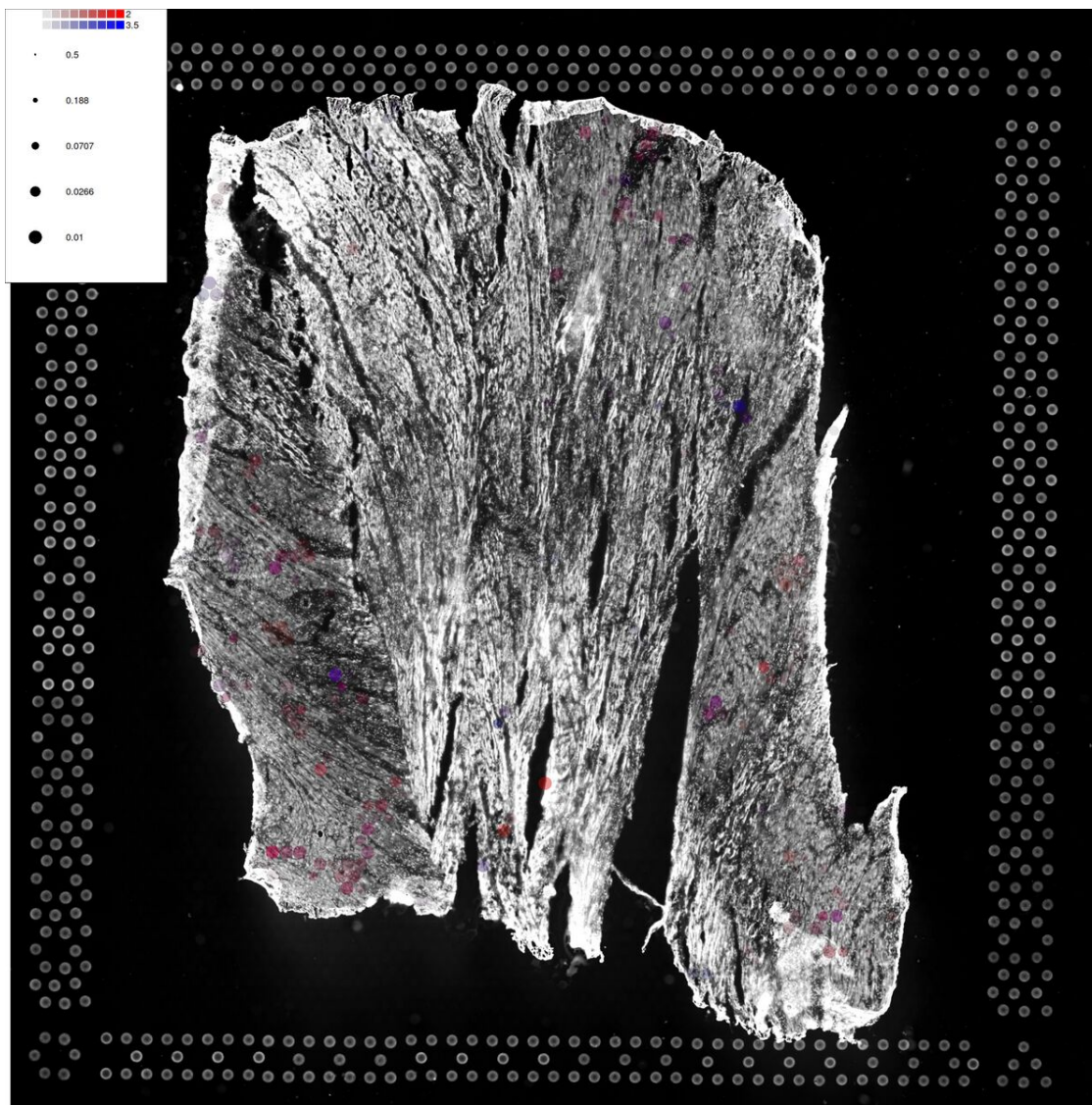


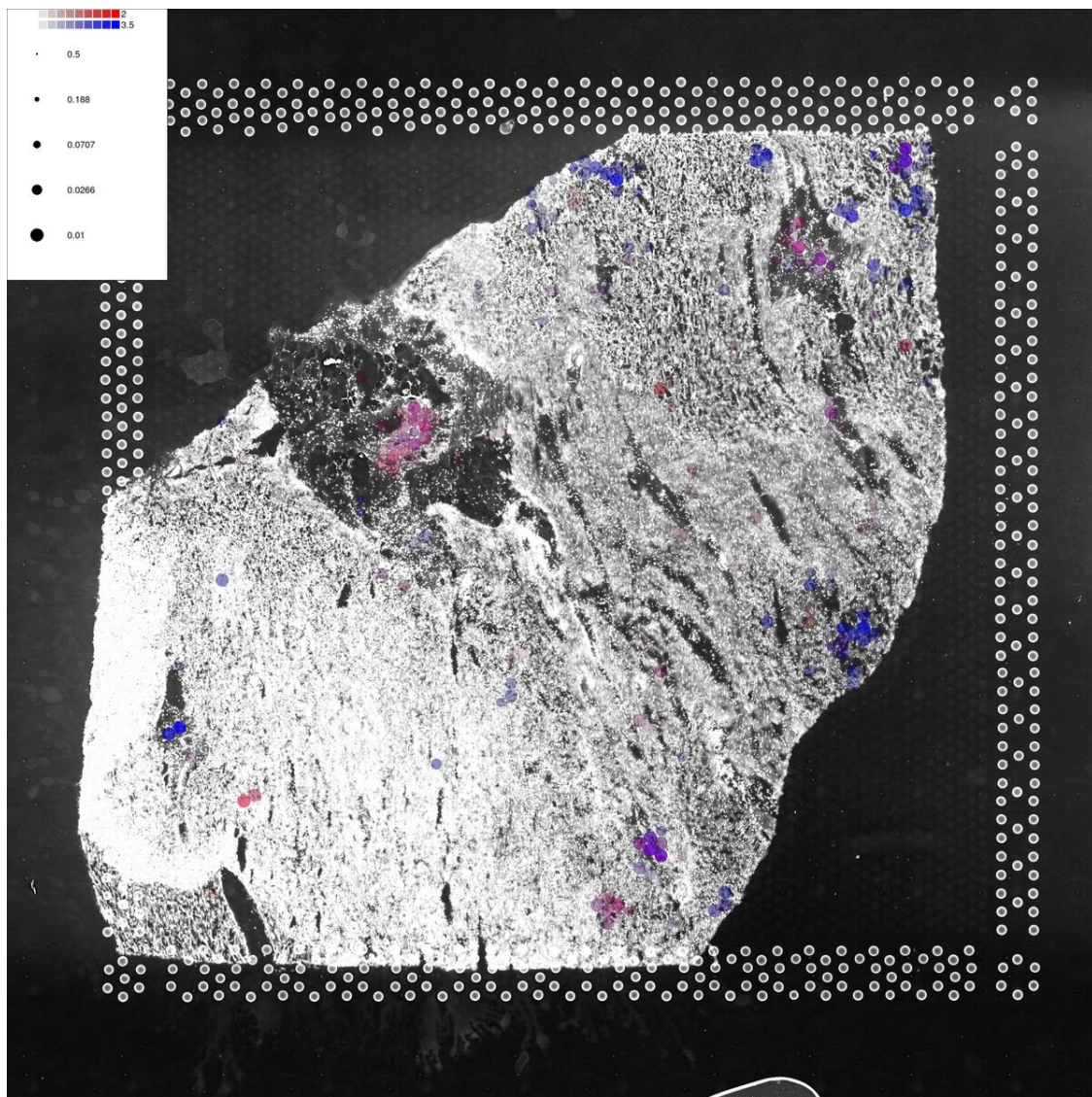


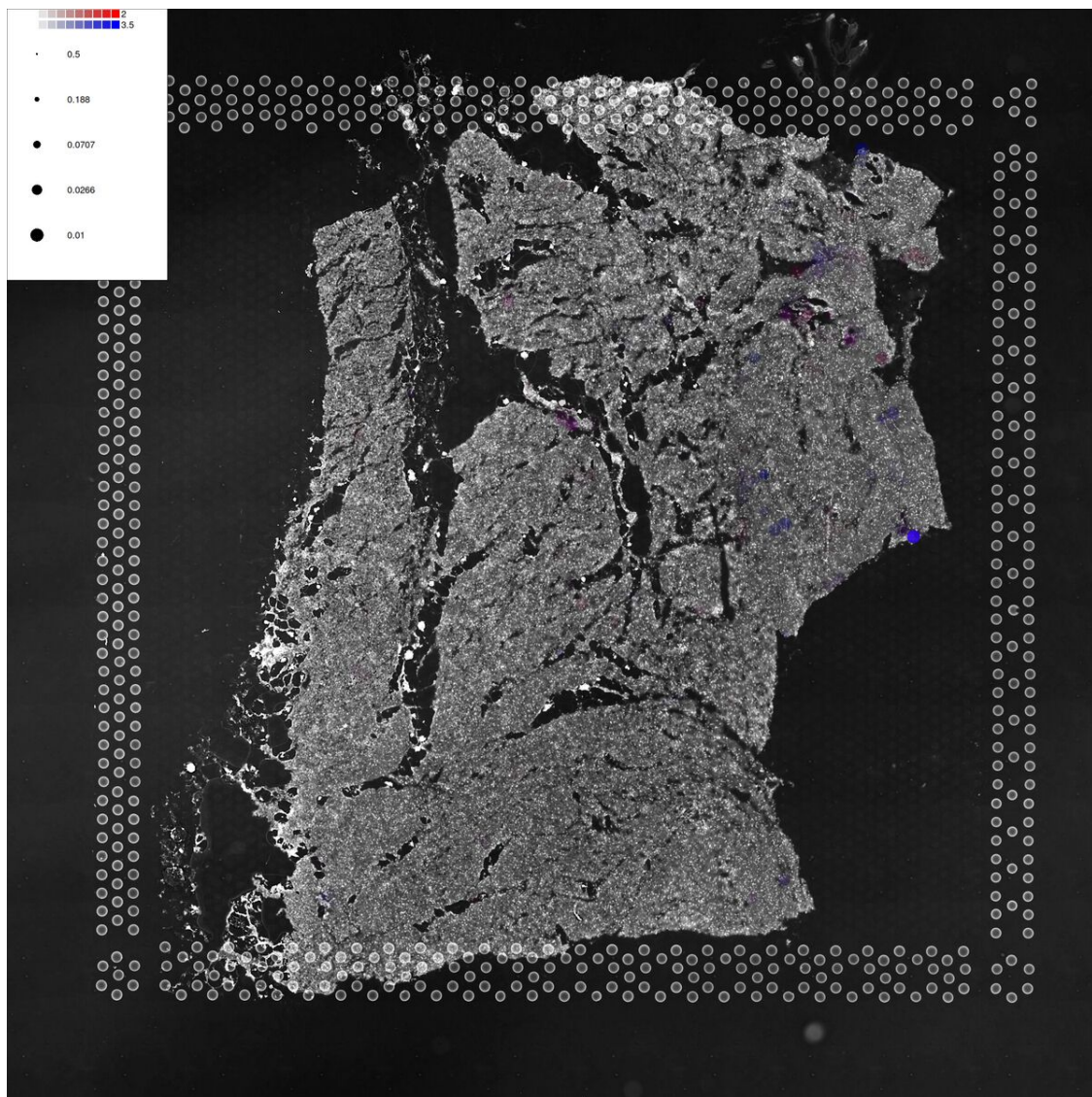


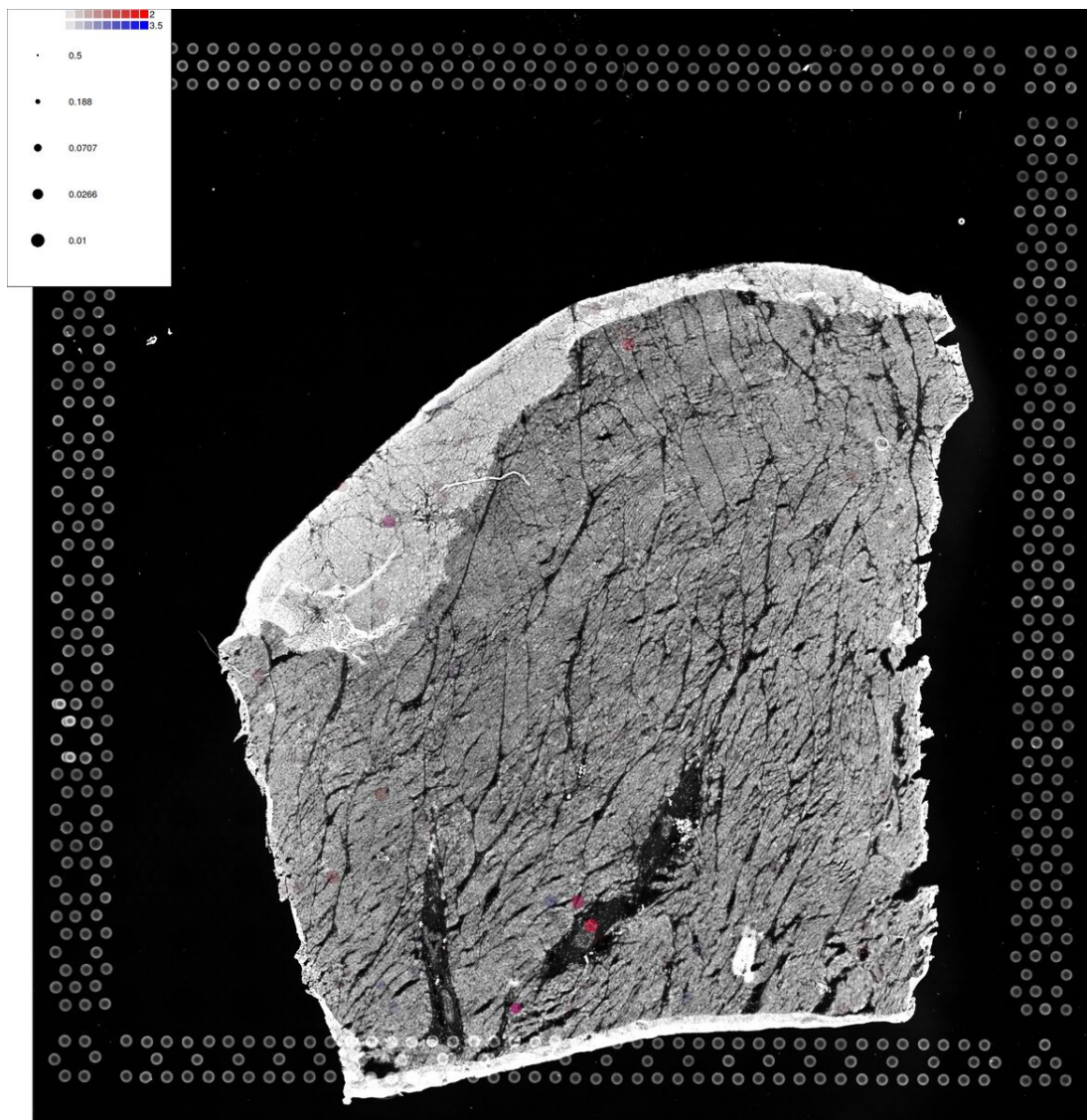


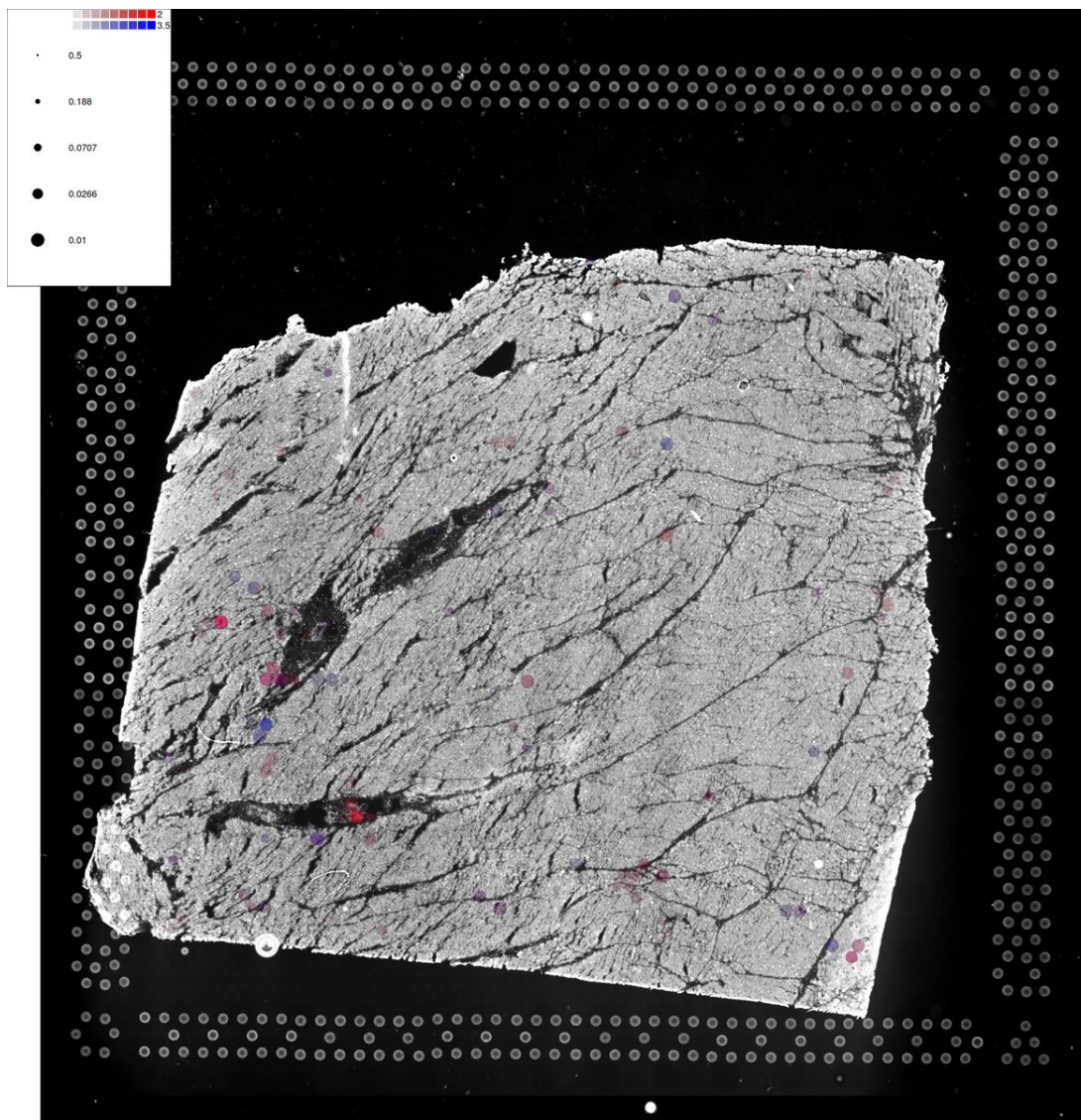


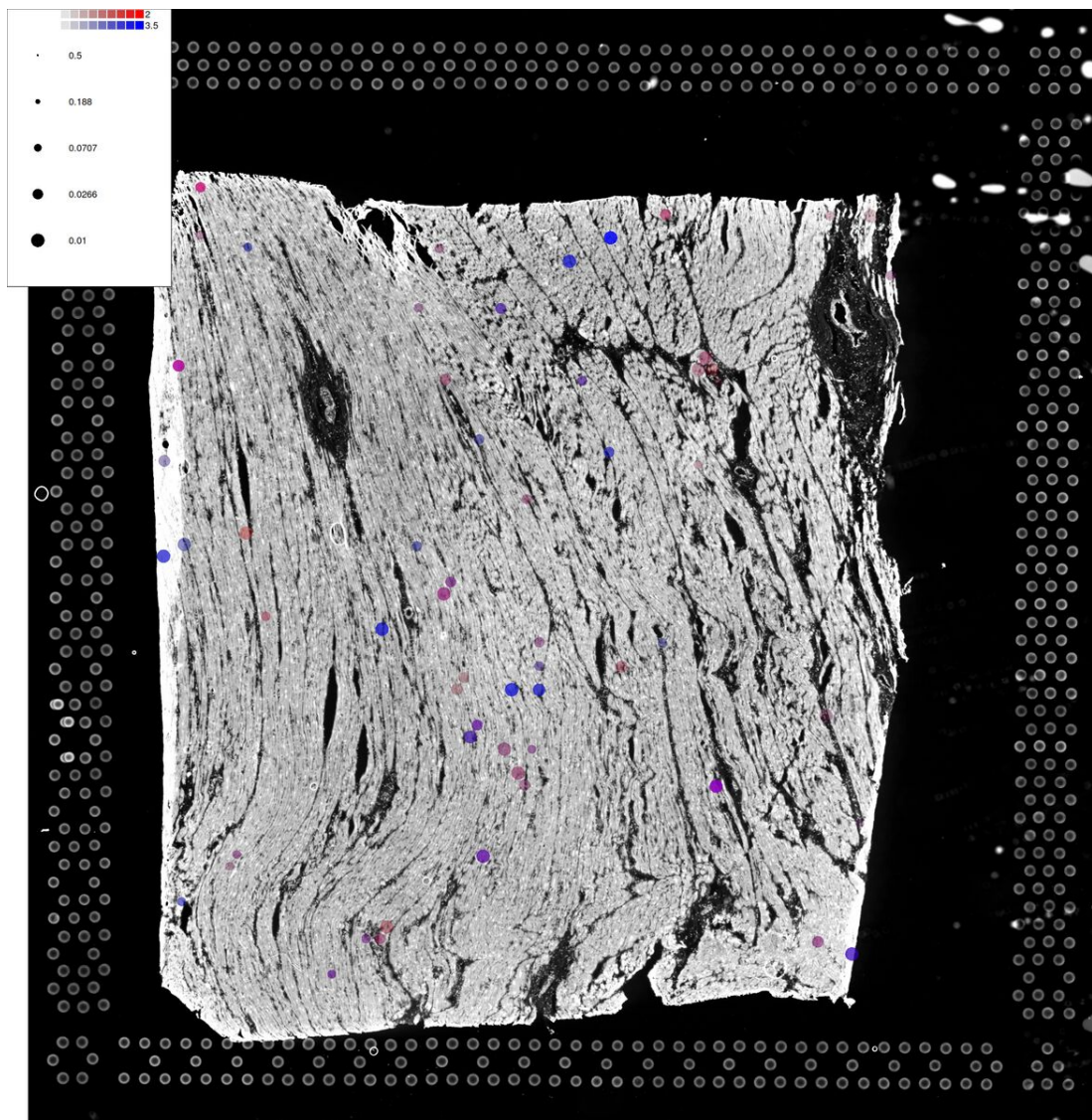












[]: